A FRESH LOOK AT SOFTWARE ENGINEERING WITH STATECHARTS

Karthika Venkatesan

Doctor of Philosophy Thesis

January 2024

International Institute of Information Technology, Bangalore

A FRESH LOOK AT SOFTWARE ENGINEERING WITH STATECHARTS

Submitted to International Institute of Information Technology, Bangalore
in Partial Fulfillment of
the Requirements for the Award of
Doctor of Philosophy

by

Karthika Venkatesan
PH2014004

International Institute of Information Technology, Bangalore
January 2024

Dedicated to

Ranganayaki & Venkatesan..

My Amma and Appa...

Thesis Certificate

This is to certify that the thesis titled **A Fresh Look at Software Engineering with Statecharts** submitted to the International Institute of Information Technology, Bangalore, for the award of the degree of **Doctor of Philosophy** is a bona fide record of the research work done by **Karthika Venkatesan**, **PH2014004**, under my supervision. The contents of this thesis, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

Prof. Sujit Kumar Chakrabarti

Bangalore,
The 7th of January, 2024.

A FRESH LOOK AT SOFTWARE ENGINEERING WITH STATECHARTS

Abstract

Statechart is a modelling notation that has wide acceptance in the industry. Statecharts provide a visual representation of the different states and transitions in a system, making it easier to understand and analyse complex specifications. They also allow for the decomposition of a system into smaller, more manageable modules, which promotes reusability and maintainability. Statecharts' global variables, however, pose a challenge to modularization, incremental development, and the specification of systems.

To meet this need, we have created StaBL, a state-based language, with the objective of adding local variables to statecharts. This involves formalising how local variables interact with key components of traditional statecharts, including states, transitions, hierarchy, entry and exit actions, history, and concurrency. Additionally, analysis methods and tools were developed to verify, simulate, and test the models written in our statechart variant.

Our work introduces local, parameter, and static classes of variables with custom types into traditional statecharts. The unique feature of the language is that it looks into and defines the idea of data passing between states, which makes regular state charts more expressive and modular. We present two variants of the language: StaBL, a state-based language with features including states, transitions, composite states, history, local variables, and custom types; and ConStaBL, a

state-based language with concurrency.

Our initial contribution presents a state-based language with local variables (StaBL), and its operational semantics with a specific focus on variables, history, and data passing. We present the design of a compiler frontend (a parser and a typechecker) and a simulator to assist in the writing and simulation-based validation of the models.

We present a translation-based approach to verifying models through symbolic execution. We transform StaBL models into imperative languages so that we can use the power of existing symbolic execution engines to find problems in the models, like undefined variable access, stuck specifications, and non-determinism.

Further, as we upgraded the language to include concurrency, we detailed the semantics that supports fine-grained interleaved execution. This new version comes with its own semantics and toolchain for simulation. In the case of concurrent fine-grained execution, all the possible outcomes must be explored. For that reason, we have worked out an exhaustive exploration algorithm on state charts, which ensures that all the interleavings are explored. We have demonstrated the use of fuzzing with a statechart. This has helped us to check the correctness of the simulator, replicate the random behaviour of the environment, and exhaustively simulate the model with random events and inputs.

In this work, we have created a variant of the statechart language, built a tool chain for simulating and verifying the models, and addressed the intricacies and detailing involved pertaining to the action language of statecharts. We tested our tools on several case studies of different scales and complexity to make sure that our language can aid in building modular specifications. This also showcases the effectiveness of the language and methods in terms of expressiveness, scalability, completeness, and correctness.

Acknowledgements

Swami Vivekananda once said, **Arise, Awake and Stop not till the goal is reached**. Years ago, I embarked on an overwhelming yet transformative PhD journey—a monumental odyssey filled with challenges and triumphs. The unwavering support of various individuals during this academic expedition not only sustained me but also grounded my aspirations. I extend my heartfelt gratitude to those who have been instrumental in shaping this experience.

First and foremost, to my supervisor and mentor, Prof. Sujit Kumar Chakrabarti, your guidance has been pivotal in navigating the intricacies of research. Your constant push to break intellectual boundaries has shaped my academic pursuits. Thank you for always believing in me through the ups and downs of this journey. I express my thanks to the Members of my Doctoral Advisory Committee Prof. Meenakshi D'Souza, and Prof. G. Srinivasaraghavan for their insights and guidance, which played a crucial role in the success of my research.

To my parents Ranganayaki and Venkatesan, who deserve special recognition for sowing the seeds of curiosity within me, and instilling the zest in me to look at everything from a research perspective. Their strength has been the bedrock on which I stand today. They are the primary source behind all my endeavours, for enabling me to recognise the inner strength to be who I am.

A heartfelt thank you to Dr. Sarat Chandra Babu (Advisor NCOE-DSCI and Former Executive Director, C-DAC Bangalore) for his belief in my abilities, unwavering trust, and invaluable fatherly guidance, providing a steady hand through the ups and downs of this long journey.

To my sisterly figures—Ms. Sreekala Nambiar, Ms. Supriti, and Ms. Lakshmi

Rajkannan —your belief and support have been constant sources of inspiration, guiding me through the complexities of this journey.

Special acknowledgement goes to the Executive Director Dr. SD Sudarsan, Centre for Development of Advanced Computing, Bangalore whose support and guidance significantly contributed to the success of my work. I thank my colleagues Dr. Sreekanth, Mr. Haribabu, Dr. Balaji, Dr. Saritha and Dr. Janaki for their encouragement and support. Special thanks to Dr. Ketaki Bapat(Retd) from the Office of Principal Scientific Advisor, GoI and Dr. Seshadri Srinivasan from KARE India for their heartfelt good wishes and guidance. I would also like to thank Prof. Debabrata Das, Director IIITB, Prof. Sadagopan, Former Director IIITB and Shridhar Shreenivasa Ratnam Commodore(Retd), Registrar, IIITB for their leadership and administrative support.

I extend my gratitude to well-wishers, lab mates, and family members for their unwavering encouragement and understanding, acting as constants in this dynamic journey. In the company of friends—Dumindu Nayanjith Silva, Karthik Venkatachalam, Sahana Raj, O. Aishwarya, Nikhila KN, Varsha Suresh, Meghana Varghese, Vishnu Raj, Karthik Subramanian, Deepti J. Anand, Ganesha—I found timely encouragement that kept my motivation intact during challenging times. Collaborative efforts with students like Harika Rajavaram, Shyam Singh, Sharad Mahajan, Mayank Senani, Advait Lonkar, Amogh Johri, Arpitha Malavalli, Nikitha AN, Bharath Sai Ellanti, Srinivasan Vijayaraghavan, Krishna Sharma and Kaveri Biswas enriched my research experience, fostering a sense of community.

This academic journey has been a collective effort, and I extend my gratitude to the entire universe for its role in shaping my research endeavours.

— Karthika Venkatesan

List of Publications

[A] Sujit Kumar Chakrabarti and Karthika Venkatesan. 2020. StaBL: Statecharts with Local Variables. In Proceedings of the 13th Innovations in Software Engineering Conference on Formerly known as India Software Engineering Conference (ISEC 2020). Association for Computing Machinery, New York, NY, USA, Article 6, 1–10. https://doi.org/10.1145/3385032.3385040

[B] Karthika Venkatesan and Sujit Kumar Chakrabarti, A fresh look on semantics of Concurrent State Based Language (ConStaBL), Journal of Computer Languages, vol. 79, p. 101270, 2024.

[C] Statechart specification for web applications, (poster presentation) in 14th ACM-IEEE International Conference on Formal Methods and Models for System Design (MEMOCODE 2016).

Contents

List of Figures

List of Tables

List of Abbreviations

CAS ..... Common Ancestors

CA ..... Collision Avoidance

CCA ..... Closest Common Ancestors

CFG ..... Control Flow Graph

CFGTree ..... Control Flow Graph Tree

ConStaBL ... Concurrent State based Language

CP ..... Control Point

CST ..... Configuration State Tree

ENode ..... External Node

EVA ..... Emergency Vehicle Avoidance

IIITB ..... International Institute of Information Technology Bangalore

INode ..... Internal Node

JP ..... Join Point

LUB ..... Least Upper Bound

StaBL ..... State based language

CHAPTER 1

INTRODUCTION

Statecharts [2] have garnered widespread acceptance within the industry as a modelling notation that excels in visually representing the states and transitions within a system. They allow for the visualisation of system behaviour at various levels of detail, making even large and complex specifications manageable and understandable.

Statecharts can serve as standalone behavioural descriptions or can be integral to a design that includes dataflow specification and functional decomposition. Statecharts have found extensive use in the design of reactive systems [2]. For example; encompasses vehicles, communication networks, computer operating systems, avionics, and other domains. Statecharts are compact and expressive. They offer a compositional and modular approach to system specification. This graphical representation facilitates the comprehension and analysis of complex specifications, leading to enhanced understanding and maintainability.

In our research, we extend the statechart language by incorporating local variables, and examining their interaction with various statechart elements such as hierarchy, concurrency, and action block execution. Our aim was to build a full-fledged toolchain for a subset of statecharts (atomic states, composite states, orthogonal states, inter-level transitions, and history), and to study its impact

by introducing variables with scopes and storage classes. We have detailed the operational semantics for our language and built a simulator, test engines, and verification mechanisms.

1.1 Background

David Harel introduced statecharts [2] in 1987 as a visual modelling language for complex reactive systems. Visually representing discrete event systems was the primary intent behind Harel's introduction of the language for statecharts. The inclusion of the notions of hierarchy, concurrency, and communication into state diagrams resulted in the development of statecharts [2] to present a system's behaviour at various refinement levels. Apart from the structure of statecharts, the work highlighted various interesting ideas, like to maintain history of a state in two ways - deep and shallow. This feature allows a state to return to its substate exited when it was previously active. For instance, in the digital watch example provided in Harel's paper, this feature is used to return the display of the watch to its previous state. When the display is toggled between setting an alarm and the initial screen from which it was accessed, and the display is restored to that initial screen. History can be cleared or retained. Various pseudo-states were defined to describe the system more effectively - initial, choice, history (deep and shallow) etc.

While there is no consensus on "one right way" to build semantics for statecharts [3], various approaches have been proposed to accurately capture system behaviour, given their ability to model real-time, event-driven systems. Over time, more than 100 variants [4–7] of statecharts have been proposed; they are distinct based on their features, semantics, and application domains.

These variants have aimed to address specific limitations or introduce new fea-

tures to enhance the expressiveness and effectiveness of statecharts in different domains. These works differ in either semantics, features or application domains. Some notable works include the introduction of timed statecharts [8–13], probabilistic statecharts [14–16], and hierarchical and concurrent statecharts [4–6, 17]. From finite state automata, state transition diagrams, extended hierarchical automata, these state based languages are commonly employed for modelling systems and applications in various domains, such as hardware systems, embedded systems [18], software [19], web applications, websites, and graphical user interfaces [20–22]. These extensions have significantly broadened the applicability of statecharts and have been widely adopted in various industries such as automotive, aerospace, and software engineering.

The main difference in semantics is based on its event processing mechanism and how it interact with other features in Statecharts [23, 24]. There were different type of semantics such as operational [18, 25–28], compositional [29, 30], denotational [31–33], or template [34] for statecharts.

However, certain limitations have been identified with respect to variable management within statechart models. While global variables are prevalent in statechart implementations, and some variants classify variables as input/output variables, these approaches do not restrict variable access by specific states or elements. This absence of constraints can result in unintended modifications of variables and pose challenges in identifying such issues, particularly as models grow in complexity. A review of existing analysis tools reveals a predominant focus on the translation of statecharts into alternative formalisms, such as hierarchical automata, NuSMV, and Promela [35] to verify the issues. Our focus was specifically on working with the statechart itself. This motivated us to develop comprehensive semantics for statecharts incorporating local variables. This semantics will be a foundation for future works on formal analysis of statecharts that leverage local

variables.

This section provided an overview of the relevant existing work. A detailed review of these works is presented in chapter 8.

1.2 Motivational Example

Let us consider this automated traffic light, which will record the number of times a signal goes red, green, or yellow (as shown in Figure FC1.1). We proceed by modeling a traffic signal junction, as shown in Figure FC1.2, that contains two signals.

If the signal junction is modeled as a statechart, then all the variables would have to be global (of both traffic lights). However, this may introduce a vulnerability where one traffic light has access to increment and decrement the count variables of another. For instance, traffic light 2 could change the rCount of traffic light 1.

Therefore, it is sufficient for the traffic lights to have their own local variables and to share only the variables that are necessary in the common state, as shown in Figure FC1.2. Here, since the junction should ensure that only one of the traffic lights is green at the same time, it just shares the active variables, whereas all other variables are local to the respective traffic lights.

With a model like this, we can ask questions such as whether S1 and S2 will ever be green at the same time (which could lead to accidents). However, for this to work, we need well-defined semantics for the variable locality that preserves the formal verification capabilities of statecharts, even as we modify the semantics of statecharts themselves.

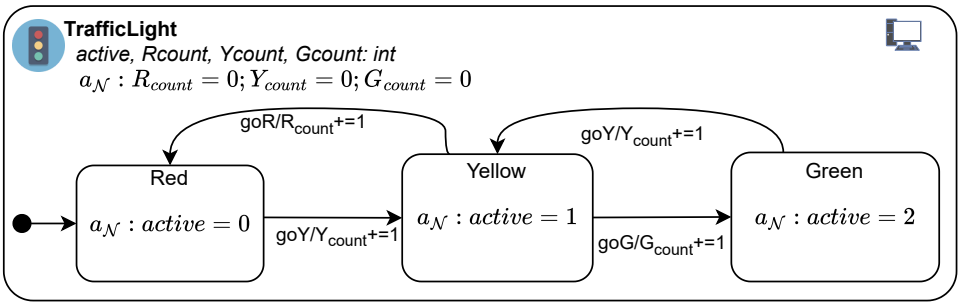


Figure FC1.1: Traffic light example

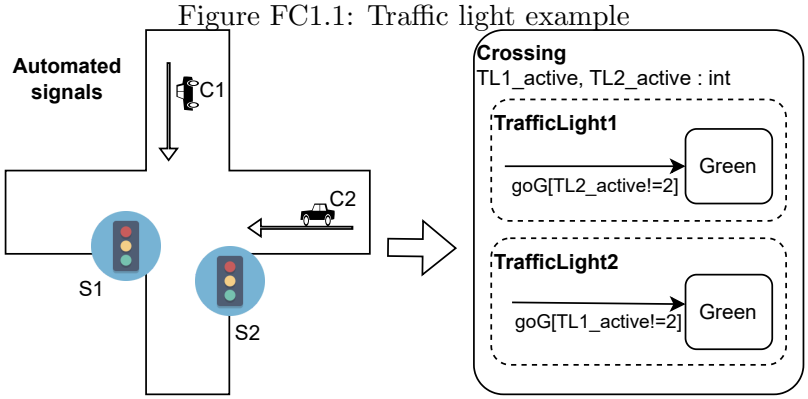


Figure FC1.2: Modelling a signal with traffic lights

1.3 Contributions

1. Develop a formal state-based modelling language with local variables that is amenable to analysis. We began with a non-concurrent statechart variant featuring local variables and subsequently introduced a concurrent variant, resulting in two distinct statechart versions: StaBL - a state-based language with local variables, and ConStaBL - StaBL with concurrency.

2. Redefine the operational semantics of the language focusing on the impact of local variables.

3. Detect common issues: conflicting transitions, stuck specification, undefined variable access, and configuration reachability.

4. Build a tool chain to implement the semantics.

5. Perform fuzz testing of statechart models and analysis.

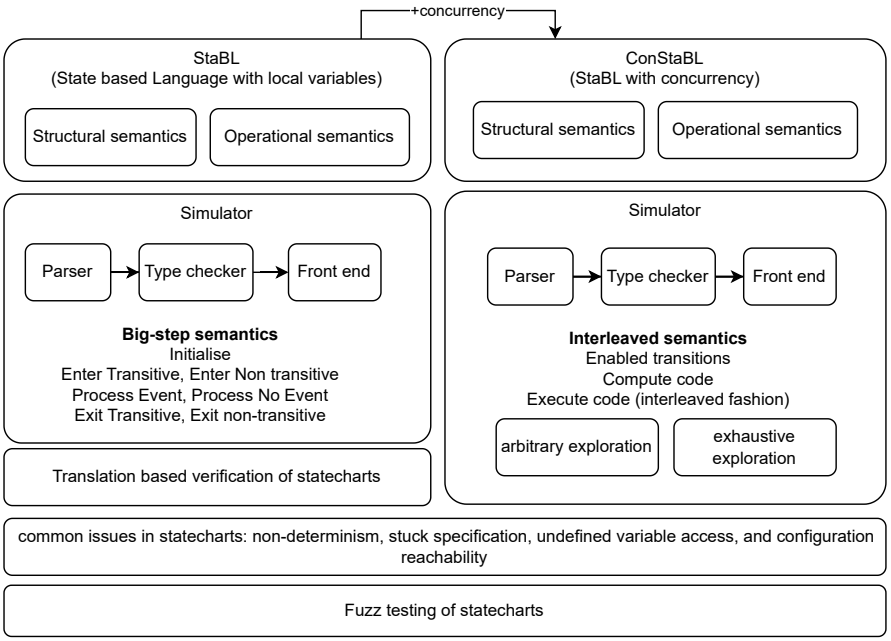


Figure FC1.3: Overview of work

6. Carry out exhaustive exploration in the case of concurrent statecharts for property checking.

The overview of the work undertaken is shown in Figure FC1.3. The work is primarily divided into two main parts, aligned respectively with StaBL and ConStaBL. Verification issues are applicable for both variants. The method of verification through translation has been employed specifically for StaBL, while fuzz testing has been applied to both variants.

1.3.1 State Based Language with local variables (StaBL)

StaBL, a state-based language, draws inspiration from traditional statecharts while introducing the novel concept of local variables. This integration of variable scoping within the statechart paradigm necessitates a redefinition of semantics to accommodate their access and implications. The language defines three distinct storage classes for local variables—local, parameter, and static—each with unique read and write scopes. A unique characteristic of StaBL is its formali-

sation of data passing between states through variables of varied storage classes, fostering enhanced expressiveness and modularity in comparison to conventional statecharts. To achieve this, the operational semantics have been revised with a specific focus on variables, their value histories, and data transmission mechanisms. This involves formalising the interactions between local variables and fundamental statechart components, including states, transitions, hierarchy, entry and exit actions, history, and concurrency. We propose a modularity metric specifically tailored for statecharts and demonstrate the increased modularity exhibited by models constructed in StaBL. A comprehensive toolchain for simulating and verifying these models has been developed, addressing the complexities and nuances associated with the action language within statecharts.

1.3.2 A concurrent State Based Language (ConStaBL)

This work details the effect of concurrency within the statechart modelling. We introduce ConStaBL, a concurrent variant of the StaBL language, defined to encompass concurrency constructs. The integration of concurrency necessitates a reevaluation of statechart execution behaviour, as orthogonal states mandate the simultaneous activation of all their substates. In this context, a single event can potentially trigger multiple transitions within an orthogonal composition, underscoring the need for concurrent execution of action blocks associated with these transitions. We present a formalisation of operational semantics from the interleaving perspective. We address the intricacies of forks and merges required for preserving the interleaved execution when transitions originate from or terminate in nested orthogonal states. This distinction necessitates a fine-grained redesign of operational semantics to ensure coherence and accuracy. Delving into both structural and operational semantics of concurrent statecharts, this work places a particular emphasis on action block interleaving, offering a novel perspective that

contributes to a deeper understanding of interleaved execution behaviour within complex systems.

1.3.3 Fuzz testing of statecharts

Fuzz testing, a widely adopted technique for software assessment, involves the generation of arbitrary inputs to uncover potential vulnerabilities and defects. In the context of statecharts, which model system behaviour using visual representations of states, transitions, and events, fuzz testing involves systematic generation of randomised inputs to simulate diverse scenarios and validate model behaviour. This approach enables the identification of issues such as erroneous state transitions, unanticipated behaviours, and deadlock situations that may arise within statechart models. It proves particularly valuable for comprehensive testing of complex systems employing state-based languages like ConStaBL, which often exhibit intricate concurrency behaviours. We have harnessed fuzzing techniques to evaluate both the performance and utility of our statechart simulator as a tool for model exploration. This involved subjecting the simulator to a substantial volume of events generated through fuzzing, as well as processing models of varying sizes. To assess coverage, fuzzing was used to create diverse event sequences, enabling the evaluation of reachability between randomised model configurations. Additionally, fuzzing was employed to verify crucial statechart properties, including conflicting transitions and the presence or absence of desired/undesired configurations.

1.3.4 Exhaustive exploration of statecharts

While ConStaBL semantics function through the exploration of arbitrarily selected interleavings, a more comprehensive approach to ensuring property correctness involves exhaustive exploration. This technique involves a systematic analysis

of all possible statechart executions, guaranteeing the robustness of properties under verification across all potential scenarios. To achieve this, we propose an algorithm for exhaustive exploration, outlining state maintenance and processing procedures, bounded by maximum exploration depth imposed on statecharts.

1.3.5 Translation based verification of statecharts

In the context of large-scale system models, the efficacy of simulation techniques in pinpointing potential issues becomes impeded. To address this challenge, we propose the adoption of symbolic execution, a method that facilitates reasoning about system behaviour without requiring concrete inputs. Instead, symbolic values are employed to represent inputs, and as diverse execution paths are explored, constraints on these symbolic values are gathered. These constraints identify the paths where the issue occurs. We instrument statechart-specific properties and propose a translation approach that harnesses these engines for property verification within statecharts. We use the existing symbolic execution engines such as KLEE [36] (for C code) and JPF-Symbc [37] (for Java code). The translator transforms hierarchical statecharts (in the StaBL language variant) into flattened models, ensuring that variable reinitialisation is handled in accordance with variable scope. Upon completion of the flattening and translation process, symbolic execution techniques are systematically applied to exhaustively traverse all feasible paths and states within the system. This approach enables the detection of potential issues that may stem from conflicting transitions, stuck specifications, or undefined variable access.

1.4 Thesis outline

The rest of the thesis is organised as follows:

- Chapter 2, introduces the basic structures of our statechart formalism and forms an introduction to all the terminologies that are common to both StaBL and ConStaBL variants.
- Chapter 3, details StaBL [38, 39] - the state-based language with local variables and the operational semantics of the language.
- Chapter 4, details ConStaBL [40] and its operational semantics.
- Chapter 5 presents our approach to employing fuzzing on the statechart model to verify the issues.
- Chapter 6 presents an exhaustive exploration procedure that is needed for concurrent statecharts for exploration of all interleavings.
- Chapter 7, provides an automated translation and verification of issues through symbolic execution.
- Chapter 8 presents the existing literature on statecharts that highlights the usage of variables and action language.
- Finally, Chapter 9 presents the future work and conclusion.
- The appendix A presents the types used in our statechart models and appendix B presents the extended version of example provided in the exhaustive exploration chapter 6.

CHAPTER 2

LANGUAGE FUNDAMENTALS

This chapter describes the fundamentals of the statechart language (StaBL[1] [2]and ConStaBL[3]). We begin by introducing the essential constructs of the language and gradually add the StaBL and ConStaBL-specific features. The major difference in our adaptation of Harel's statecharts [2] is the introduction of local variables.

The introduction of local variables in our version of statechart is a significant enhancement. By allowing the use of local variables, our language becomes more expressive and flexible, enabling the modelling of more complex systems. The structural semantics presented here lay the foundation for understanding the overall behaviour of our language, while the subsequent chapters delve into the intricacies of StaBL and ConStaBL-specific semantics. Our language is a subset of the traditional statechart language that lends itself to formal analysis.

[1]The content of this chapter is based on: Sujit Kumar Chakrabarti and Karthika Venkatesan. "Stabl: Statecharts with local variables." In Proceedings of the 13th Innovations in Software Engineering Conference on Formerly Known as India Software Engineering Conference, ISEC 2020, doi: https://doi.org/10.1145/3385032.3385040

[2]The content of this chapter is based on: Venkatesan, Karthika, and Sujit Kumar Chakrabarti. "StaBL-State Based Language for Specification of Web Applications." arXiv preprint arXiv:1901.02188 (2019). doi: https://doi.org/10.48550/arXiv.1901.02188

[3]The content of this chapter is based on: Venkatesan, Karthika, and Sujit Kumar Chakrabarti. "ConStaBL–A Fresh Look at Software Engineering with State Machines." arXiv preprint arXiv:2307.03790 (2023), doi:https://doi.org/10.48550/arXiv.2307.03790 and "A fresh look on semantics of Concurrent State Based Language (ConStaBL)", Journal of Computer Languages, vol. 79, p. 101270, 2024. doi:https://doi.org/10.1016/j.cola.2024.101270

StaBL [38, 39] takes inspiration from well-known imperative languages like C and Python (mutable variables, instructions, lexical scoping etc.), but borrows significantly some important aspects from Statecharts: *viz.*, states, transitions, hierarchy and action language. StaBL is a language with constructs supporting programming using primarily imperative style. These imperative elements make its syntax approachable to a developer. The state-based structure also makes it easy to visualise a StaBL program in a pictorial form akin to Statecharts. We present our language both informally through examples as well as through a formal description of some salient aspects of its semantics.

StaBL and ConStaBL are the two variants we use to refer to the language in this thesis. StaBL serves as the basis for the language for statecharts incorporating local variables, while ConStaBL incorporates concurrency to improve upon StaBL. The key concepts outlined for StaBL are predominantly transferable to ConStaBL; In the thesis, StaBL and ConStaBL are used for our version of the Statechart language. The term "Statechart" with a capitalization denotes the traditional language for statecharts. The terms "statechart" or "statecharts" represents the model written in the language "Statechart". Additionally, in statechart models are also referred as "model".

2.1 Abstract syntax

A statechart model consists of five components: a set of states (S), a set of transitions (T), a set of events (E), a set of types ($\mathbb{T}$) and a set of functions ($\mathbb{F}$). Here, let us consider a representative set of $S : \{sc, s, s', s'', ...\}$. We proceed by defining the state tuple.

2.1.1 State

A ***state*** s is a *tuple* of $(n, p, C, I, \mathcal{V}_l, \mathcal{V}_p, \mathcal{V}_s, a_{\mathcal{N}}, a_{\mathcal{X}}, \tau, h)$.

n Name of the state s, an unique identifier.

p The parent state of s (for a root state of type *statechart*, parent is denoted as $\diamond$(undefined)).

C Set of all substates of a s.

I Set of initial substates of s. I is a subset of C ($I \subseteq C$).

$\mathcal{V}_l$, $\mathcal{V}_p$, $\mathcal{V}_s$ Set of variables declared within the state s with storage class *local*, *parameter* and *static* respectively. The union of all these types of variables is denoted as $\mathcal{V}$.

$a_{\mathcal{N}}, a_{\mathcal{X}}$ Entry and exit actions[4].

τ is the type of the state. $\tau \in \{statechart, atomic, composite, shell\}$.

h is a boolean value that specifies if the history[5] should be maintained by the state.

Throughout the discussion, we have references to states being atomic and non-atomic. Here, s is atomic ($s.C = \phi$) and non-atomic (i.e, composite, shell or statechart) ($s.C \neq \phi$). The set of all states S is the union of the set of atomic states (S_A) and of non-atomic states (S_{NA}): $S = S_A \cup S_{NA}$.

Also, please note that we use *dot* notation to refer to the values in the state tuple. For instance, for a state s, the set of initial states is denoted as $s.I$.

[4] written in an appropriate action language

[5] functionality of history is detailed in section 2.1.6

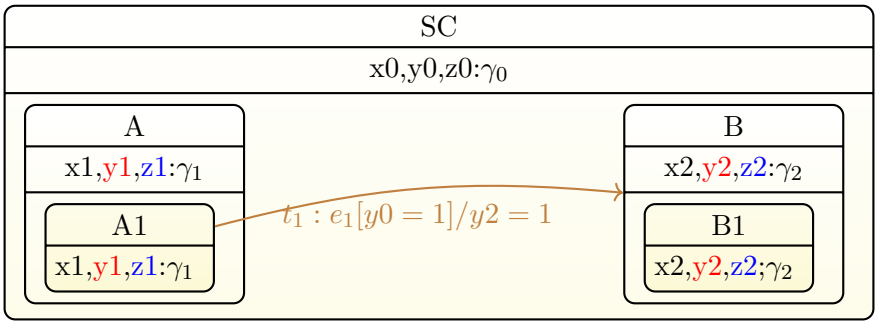


Figure FC2.1: Example for statechart

Also, a state s is uniquely identified by the namespace given by its containment[6] from root state *sc*. In figure FC2.1, the state *A1* can also be referred to as *SC.A.A1*. The state tuple for the state A is $(A, SC, A1, A1, x_1, y_1, z_1, a_{\mathcal{N}} : \{\}, a_{\mathcal{X}} : \{\}, composite, false)$. More examples with detailed discussion are given in section 2.2.

2.1.2 Transition

A ***transition*** is a *tuple* of (n, p, s, d, e, g, a). A transition is annotated as $e[g]/a$ on the arrow that connects the *source*(s) and *destination*(d) states. For a transition t,

n is the unique identifier. Name of the transition.

p is the parent state (a state that encloses both source and destination of the transition)

e is the event/trigger.

g is the guard - boolean expression or boolean value

a is the action code.

[6] more on containment is in section 2.4.1

In figure FC2.1 transition $t1$, the event is $e1$, guard is $y0 = 1$ and transition action is $y2 = 1$.

2.1.3 State types

There are 4 types of states in a statechart model.

1. Statechart - The root state of the model is of type *statechart*. A statechart model has only one state of the *statechart* type, which is itself. I will contain one of its substates.

2. Composite - This type of state can contain multiple substates, which is responsible for the hierarchical notion of statechart models. But at most, only one of its substates is active at any point in time during execution. For a composite state, I is a singleton set with one of its substates.

3. Atomic - This type of state cannot have substates($C = \emptyset$). For an atomic state, I is *empty*.

4. Shell- The states contained within *shell* execute concurrently. A *shell* state bears resemblance to an *AND* state [41] [42], but it distinguishes itself in terms of how transitions are defined as described in Chapter 4. The substates of the *shell* state are of type *composite*, aka. *regions*, and all or none of them will be active at a given point in time during execution, so I will contain all of its substates.

In figure FC2.1,

atomic The states A1 and B1 are atomic. A1 is $\{A1, A, \{\}, \{\}, x_1, y_1, z_1, a_{\mathcal{N}} : \{\}, a_{\mathcal{X}} : \{\}, atomic, false\}$.

composite The states A and B are composite. A is $\{A, SC, A1, A1, x_1, y_1, z_1, a_{\mathcal{N}} : \{\}, a_{\mathcal{X}} : \{\}, composite, false\}$. If history was enabled then the last element of the tuple will be *true*.

statechart The state SC is of type statechart. SC is $\{SC, nil, A, B, A, x_0, y_0, z_0, a_{\mathcal{N}} : \{\}, a_{\mathcal{X}} : \{\}, statechart, false\}$

The *shell* state is detailed in the chapter 4.

2.1.4 Variables

A variable v is a *tuple* of (n, p, t, st), described by its name n, the state p in which it is declared, type $t \in \mathbb{T}$ and storage class specifier $st \in \{local, parameter, static\}$. For any state s, we partition $s.\mathcal{V}$ into $s.\mathcal{V}_l$ (local variables), $s.\mathcal{V}_p$ (parameter variables) and $s.\mathcal{V}_s$ (static variables). Hence, $s.\mathcal{V} = s.\mathcal{V}_l \cup s.\mathcal{V}_p \cup s.\mathcal{V}_s$.

2.1.5 Variable types

In its current state, StaBL has provision to use basic types like `int`, `boolean` and `string` with standard operations on these types. It also supports generic types like `map`, `list` and `set` with standard functions associated with these types (e.g. `get_map` and `put_map` – all of which are generic – for `map`). Generic containers[7] allow us to define a variety of container objects without sacrificing type safety.

1. All basic types and containers are *native types*. All other types are *user defined types*.

2. All types are globally declared, whether native or user-defined. However,

[7] More details on the polymorphic containers and functions with examples are detailed in Appendix A

variables can be declared anywhere in the model of all types, approximately like it is in C.

3. Containers and functions are polymorphic. The variable declarations for such objects must be instantiated at the time of declaration.

As mentioned above, all types are globally visible. More details on the polymorphic containers and functions with examples are detailed in Appendix A.

2.1.6 History

Let us consider $s, s' \in S \bullet \{i, s', s''\} \prec_1 s$, where i is the initial state. When a state s is marked with history, the last active substate s' of s is preserved by the execution engine/ simulator. As soon as s is re-entered, s' is entered instead of the initial state i. History is detailed in the operational semantics section 3.2.

2.1.7 Action language

Although, a language like Java or C++ is very well-suited as the action language in writing Statechart models, a purely object-oriented style imposes syntactic overheads (e.g. class definitions) which are not desirable while specifying functionality at a high level. On the other hand, a language like C with no support for parametric polymorphism does not meet the basic standards of type safety needed for writing a robust and analysable model. Hence, we decided to stick to a simple imperative structure with support for parametric polymorphism. Features like pointers, dynamic memory allocation and inheritance are too low level to be relevant at the specification level and have been left out. Code can be inserted in three places: firstly, as entry/exit code inside states; secondly, as action code on transitions and thirdly, as guard in transition.

The action language in StaBL is not strictly a part of the statechart language. However, in StaBL, variables whose scoping is the central point of interest here, are a part of the action language. Hence, in StaBL, action language partially enters the core part of the language. The action language $A(\mathcal{E}, \mathcal{S}, \mathbb{T})$ can be seen as comprising sets $\mathcal{E}$, $\mathcal{S}$ and $\mathbb{T}$. $\mathcal{E}$ is the set of expressions which evaluate to a value without side-effects. $\mathcal{S} = I \cup Seq$ is the set of statements – program phrases whose execution produces side-effects – which are either instructions (I) or sequences(Seq). Instructions, in turn, are either assignments $Assign(v, e)$ where v is a variable and $e \in \mathcal{E}$; if statements $If(c, s_1, s_2)$ where c is a boolean condition, and $s_1, s_2 \in \mathcal{S}$; loop blocks $Loop(c, body)$ where c is a boolean condition and $body \in \mathcal{S}$ is a statement. The set of sequences Seq comprises sequences of the form $i; s$ which is a recursive composition of an instruction $i \in I$ and statement $s \in \mathcal{S}$. Finally, $\mathbb{T}$ is the set of all types – basic, user-defined – in the action language (as discussed in section 2.1.5). For all $s \in S$ and $t \in T$, we can say that $s.a_{\mathcal{N}}$, $s.a_{\mathcal{X}}, t.a \in A.\mathcal{S}$ and $t.g \in A.\mathcal{E}$.

2.2 An example StaBL model

Fig. FC2.2 shows a statechart model in StaBL. The model is intended to illustrate the language features of StaBL and not directly correspond to a real modelling scenario. A model of a website is presented in section 3.1.1 where the operational semantics of StaBL are detailed.

Example 1. *In Figure FC2.2, the state SC is of type statechart. S*1 *and S*2 *are composite.* S_{11}, S_{12}, S_{21} *and* S_{22} *are atomic. State* S_{22} *has history tag (H). The*

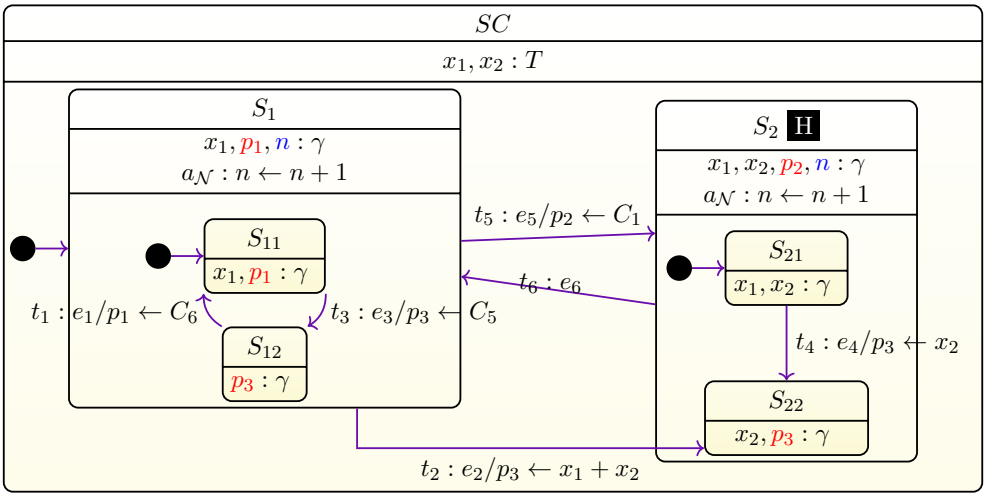


Figure FC2.2: StaBL model example

tuple of each of these states are:

$$SC : (n = SC, p = \emptyset, C = \{S_1, S_2\}, I = S_1, \mathcal{V}_l = \{x_1, x_2\}, \mathcal{V}_p = \emptyset, \mathcal{V}_s = \emptyset,$$
$$a_{\mathcal{N}} : \{\}, a_{\mathcal{X}} : \{\}, \tau = statechart, h = false)$$

$$S_1 : (n = S_1, p = SC, C = \{S_{11}, S_{12}\}, I = S_{11}, \mathcal{V}_l = \{x_1\}, \mathcal{V}_p = \{p_1\}, \mathcal{V}_s = \{n\},$$
$$a_{\mathcal{N}} : \{n \leftarrow n + 1\}, a_{\mathcal{X}} : \{\}, \tau = composite, h = false)$$

$$S_2 : (n = S_2, p = SC, C = \{S_{21}, S_{22}\}, I = S_{21}, \mathcal{V}_l = \{x_1, x_2\}, \mathcal{V}_p = \{p_2\}, \mathcal{V}_s = \{n\},$$
$$a_{\mathcal{N}} : \{n \leftarrow n + 1\}, a_{\mathcal{X}} : \{\}, \tau = composite, h = true)$$

$$S_{11} : (n = S_{11}, p = S_1, C = \emptyset, I = \emptyset, \mathcal{V}_l = \{x_1\}, \mathcal{V}_p = \{p_1\}, \mathcal{V}_s = \{\},$$
$$a_{\mathcal{N}} : \{\}, a_{\mathcal{X}} : \{\}, \tau = atomic, h = false)$$

$$S_{12} : (n = S_{12}, p = S_1, C = \emptyset, I = \emptyset, \mathcal{V}_l = \{\}, \mathcal{V}_p = \{p_3\}, \mathcal{V}_s = \{\},$$
$$a_{\mathcal{N}} : \{\}, a_{\mathcal{X}} : \{\}, \tau = atomic, h = false)$$

$$S_{21} : (n = S_{22}, p = S_2, C = \emptyset, I = \emptyset, \mathcal{V}_l = \{x_1, x_2\}, \mathcal{V}_p = \{\}, \mathcal{V}_s = \{\},$$
$$a_{\mathcal{N}} : \{\}, a_{\mathcal{X}} : \{\}, \tau = atomic, h = false)$$

$$S_{22} : (n = S_{22}, p = S_2, C = \emptyset, I = \emptyset, \mathcal{V}_l = \{x_2\}, \mathcal{V}_p = \{p_3\}, \mathcal{V}_s = \{\},$$
$$a_{\mathcal{N}} : \{\}, a_{\mathcal{X}} : \{\}, \tau = atomic, h = false)$$

The model in Fig. FC2.2 illustrates the presence of variables with three different storage classes: local, parameter and static. we have colour-coded the declaration of local variables with black, parameter variables with red, and static variables with blue in figure FC2.2: S_1 *has three variables, namely,* x_1, p_1 *and* n. *Note that* S_{11} *has two variables* x_1 *and* p_1 *declared in it. Just as in statically scoped programming languages,* $S_{11}.x_1$ *does not conflict with* $S_1.x_1$. $\gamma \in \mathbb{T}$ *is the type of the variable.*

2.3 Variable scopes in action blocks

Data being defined in one component and being used in another is an essential feature of applications. Without this feature, it would be impossible to specify interesting applications.

Example 2. *When a user logs into a web application, she is taken to her dashboard possibly displaying many other details about her already stored in the database of the application. Hence, on transitioning from the login page to the dashboard, what gets displayed is dependent on the user ID that is entered in the login page. How do we achieve this effect in a Statechart model?*

A simple alternative would be to use global variables. However, this has all the disadvantages of a global variable, and could become a hurdle in creating large models.

Making state variables strictly local to their respective states is often too restrictive. States in a statechart model often interact with each other through variable values. One option would be to lift the scope of such variables (which are used in multiple states) up to a common super state. This is useful if the number of states using a variable is large. However, in most cases, such variables are used by a pair of states as a data passing mechanism from a source state to

a destination state of a transition being taken. In such cases, scope lifting again affects modularity of the model by potentially making the variable visible in a large number of states which do not need to see the variable.

In StaBL, we achieve inter-state visibility of local variables by defining the following scoping rules. The visibility and access of variables is restricted by its storage classes - local, parameter and static.

1. Local : The variables are local to a specific state in which they are declared. The outgoing transitions are allowed to read these variables of the source state. *Local* variables have lifetimes strictly aligned with the time the machine spends within the containing state or any of its substates.
2. Parameter : special type of local variables which are writable in the action code of incoming transitions to that state.
3. Static : special type of local variables that outlive the time spent within the containing state and retain their value until redefined later

All local and static variables have scope within the containing state and all its substates. These storage classes facilitates values to be transferred from a source state of a transition to its destination in a controlled way (interstate data flow mechanism).

As mentioned earlier, the scoping rules for variables in StaBL are driven by two conflicting requirements:

1. to restrict visibility of variables to within states to achieve modularity;
2. to allow restricted data-flow between states.

The scoping rules of StaBL are summarised follows:

1. Visibility of a variable in any part of the model are defined in terms of the predicates R (read) and W (write). Variables with R access are allowed to appear in expressions – in the transition guards, on the RHS of assignments, etc.; and those with W access are allowed as l-values, viz. in LHS of assignments.

2. All variables with local and static storage classes of a state have RW (i.e. $R \wedge W$) access in itself and all its descendent states.

3. All variables with parameter storage class of a state have W access in incoming transition along with RW access similar to a local variable.

4. On a transition t, all local/static variables of the source state $t.s$ have RO access(i.e. Read-Only: $R \wedge \neg W$), and all parameter variables of the destination state $t.d$ have W access. All local/static variables of $t.s \sqcup t.d$ and its ancestor states have RW access, but variables present in all the ancestors of $t.s$ or $t.d$ up until $t.s \sqcup t.d$[8] are inaccessible on t. Local variables of descendent states of $t.s$ and $t.d$ are inaccessible on t.

Please note, that the statement "state/transition has read or write access" refers to the state action blocks ($a_{\mathcal{N}}$ and $a_{\mathcal{X}}$) and transition action (a) having the respective read/write access. The variables are looked upon in their environments as defined in section 2.4.4.

Example 3. *In Figure FC2.2 any action code within S_{11}, $S_{11}.x_1$ will shadow $S_1.x_1$. In any action code in S_1, $S_{11}.x_1$ is not visible. In state S_1, we have a static variable n. n is declared to be of type T, and has been initialised to 0 (the default value of its type as it is a static variable). In the entry code of S_1, n gets incremented by 1 (possibly to keep track of the number of times S_1 state is entered). A static variable n is defined in state S_2. Note that, scoping rules for*

[8] $\sqcup$ - closest common ancestor detailed in section 2.4.3

static variables are identical to those of local variables. Hence, $S_1.n$ and $S_2.n$ are not related to each other. Parameter variables are similar to static variables in terms of their lifetimes. However, they have a slightly different scope definition: apart from being visible within the containing state and its substates, parameters are (write-)visible on the incoming transitions. A local variable is (read-)visible in outgoing transition. Consider the transition $S_{21} \rightarrow S_{22}$, the variable x_2 being used in the action code of e_4 is $S_{21}.x_2$. The variable p_3 being set there is $S_{22}.p_3$. Consider $S_1 \rightarrow S_{22}$, The variable x_1 being used in the action code is $S_1.x_1$; whereas x_2 is $SC.x_2$.

2.4 Terminologies and Definitions

Now, we introduce some definitions and notations to help a succinct and consistent presentation of further concepts:

2.4.1 Containment

Containment (denoted by $\prec$) is function between two states s_1, s_2, denoted by $s_1 \prec_i s_2$, where i is the level of containment. When $i = 1$, s_2 is the *parent* of s_1 and s_1 is *child/substate* of s_2. When $i = *$, s_2 is the *ancestor* of s_1 and s_1 is *descendent* of s_2 at an arbitrary level. Containment relation is partial-order, not reflexive (i.e., $\forall s \in S, (s \not\prec_* s)$) and transitive (i.e., $\forall s_1, s_2, s_3 \in S, (s_1 \prec_* s_2) \wedge (s_2 \prec_* s_3) \implies (s_1 \prec_* s_3)$). For instance, $s_i \prec_1 s_j$ means that s_i is immediately contained in s_j, i.e. $s_i \in s_j.C$.

Only a few type substates are valid. There are 4 state types: *statechart*, *shell*, *composite* and *atomic*. The root state of a model must be of type *statechart*, and it can have substates of any type but itself. A *composite* state must have substates

of type *composite*, *shell*, or *atomic*. A shell state must have substates of type *composite* only. An *atomic* state cannot have substates.

2.4.2 Common ancestors.

For a set of states, $S' = \{s_1, s_2..., s_n\}$, Common ancestors $CAS(S')$ is the set of states from S that are an *ancestor* of all the states in S'.

$$CAS(\{s_1, s_2, ...s_n\}) = \{s \in S | s_1 \prec_* s \wedge s_2 \prec_* s \ \wedge ....s_n \prec_* s\}$$

2.4.3 Closest common ancestors

Closest common ancestors(CCA)/ Lowest upper bound (LUB) (denoted by $\sqcup$.) is a set of states, $S' = \{s_1, s_2..., s_n\}$, $CCA(S')$ is an element s from $CAS(S')$ such that, it is not the ancestor of any other common ancestor $s' \in CAS(S')$ (i.e, $\nexists s' \in CAS(S') \setminus s | s \succ_* s'$).

$$\sqcup(\{s_1, s_2, .., s_n\}) = s | s \in CAS(s_1, s_2...s_n) \setminus s \wedge \nexists s' \in CAS(s_1, s_2, ..s_n) \wedge s' \prec_* s$$

CCA for a set of states $s_1, s_2, .., s_n$ can also be denoted by $s_1 \sqcup s_2...\sqcup s_n$ [9]. Further, CCA of set of transitions is the CCA of the source and destination states of those transitions (i.e., $CCA(\{t_1, t_2..., t_n\}) = \sqcup(\{t_1.s, t_1.d, t_2.s, t_2.d, ..., t_n.s, t_n.d\})$).

2.4.4 Environment

An environment can be visualised as a linked list of declaration lists, shown as $D^1 :: ... :: D^{n-1} :: D^n :: \phi$ where D^1, D^2, ... D^n are declaration lists, D^1 being the

[9] infix short-form of $\sqcup(\{s_1, s_2, ..., s_n\})$

Table TC2.1: Environments for states and transitions

	State s	**Transition** t
σ_R	$\sigma(s)$	$\sigma(t.s \sqcup t.d) \cup t.s.\mathcal{V}_l \cup t.s.\mathcal{V}_s$
σ_W	$\sigma(s)$	$\sigma(t.s \sqcup t.d) \cup t.d.\mathcal{V}_p$

declaration list of the closest surrounding state. ϕ represents an empty declaration list, Operator '::' represents the list construction operator.

A lookup of a variable v in an environment σ, denoted by $\sigma[v]$, consists of searching for v's declaration in all the declaration lists in σ starting from the left end. v gets bound with the first matching declaration in this order.

Two environment types are defined as follows: Read environment σ_R is where a name which is *used* in an expression (i.e. used as r-value) is looked up in. Write environment σ_W is where a name which is *defined* in an expression (i.e. used as l-value) is looked up in. The environments for the states and transitions are defined as summarised in table TC2.1.

Example 4. *In Figure FC2.2,* $S_1 \prec_1 S_{11}$ *means* S_1 *is the parent of* S_{11} *and* $substates(S_1) = \{S_{11}, S_{12}\}$. *Also,* $\mathcal{SC}$ *is the ancestor of states like* S_{11}, S_{12}, S_{21} *and* S_{22} *(it is also the ancestor of all the states contained within these states). Here,* $S_1.\tau = S_2.\tau = composite$ *and* $SC.\tau = statechart$. *all other states are of type atomic. The common ancestor function behaves in the same way for shell state as well. The example specific to shell state is presented in chapter 4 that details ConStaBL language(StaBL with concurrency).*

$$CAS(\{S_{11}, S_{12}\}) = \{S_1, SC\} \text{ and } CAS(\{S_1, S_{21}\}) = \{SC\}$$
$$CCA(\{t_2\}) = \sqcup(\{S_{11}, S_{12}\}) = S_1$$
$$CCA(\{t_2, t_3\}) = \sqcup(\{S_1, S_{22}, S_{11}, S_{12}\}) = SC$$

2.5 Typechecking

The local variables in StaBL and their scoping rules are implemented in the StaBL typechecker. During typechecking of the model, all names are looked up in a (type) environment ($\mathbb{T}_\sigma$).

t_{Assign}

$$\frac{\mathbb{T}_{\sigma_W} \vdash vname : \gamma \qquad \mathbb{T}_{\sigma_R} \vdash e : \gamma}{\mathbb{T}_{\sigma_R}, \mathbb{T}_{\sigma_W} \vdash vname \leftarrow e \longrightarrow (nil, OK)}$$

Shown above is an example of a typechecking rule: t_{Assign}, the rule for an assignment statement. The LHS of the statement, *vname*, is looked up in $\mathbb{T}_{\sigma_W}$ while the the RHS e is typechecked in $\mathbb{T}_{\sigma_R}$. If both typecheck to some type $\gamma \in \mathbb{T}$, then the typechecking of the assignment statement succeeds, giving a *nil* type and OK to indicate success. *nil* type means no type suggestion. If the type does not match then the required *type* is suggested.

2.5.1 Typechecking polymorphic types

Type checking of a polymorphic structure type declaration and a polymorphic function declaration[10] proceed somewhat similarly to each other. The fields (whether fields in a structure or parameters in a function declaration) are identical in their roles, from type-checking perspective. The rules for typechecking a polymorphic type declaration are informally outlined as follows:

[10]The details on polymorphic structures and functions are provided in appendix A

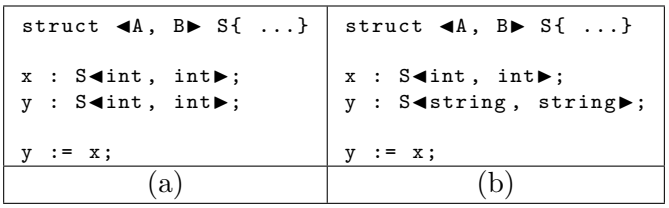


Figure FC2.3: Using variables of polymorphic types: (a) Correct usage; (b) Incorrect usage.

2.5.1.1 Polymorphic Type Declarations.

- Each field should reference a known type.
- If a referenced type is a polymorphic type, then the number of type arguments should be equal to the number of type parameters of the declared polymorphic type.
- Each type should get bound to its declared type. In case of polymorphic type, the type argument should get bound to a known type or a type variable corresponding to the respective type parameter of the enclosing type, or a known polymorphic type instantiated appropriately with known types or known type parameters, recursively.

2.5.1.2 Variable instantiation with polymorphic types.

When variables are declared using polymorphic types, they must be completely instantiated, i.e. they may not have any type variables as arguments to the declared type. The type must supply an appropriate number of type arguments.

Example 5. *In code fragments involving reconciliation between types, e.g. equality or assignment, the type checking is done by doing structural equality test between types.*

Fig. FC2.3 shows a sample code of how variables of polymorphic types can

be used in expressions and instructions. In the left sub-figure, `x` *and* `y` *are of structurally identical types, i.e.* `S ◄int, int ►`*. This code will typecheck. On the other hand, the code fragment in the right sub-figure will not type check as the type arguments cause the polymorphic structure* `S` *to instantiate two dissimilar types for* `x` *and* `y`*.*

2.6 Summary

StaBL has been primarily developed to explore the utility of local variables in Statechart and for static verification of statechart models.

CHAPTER 3

STABL - STATE BASED LANGUAGE

3.1 Introduction

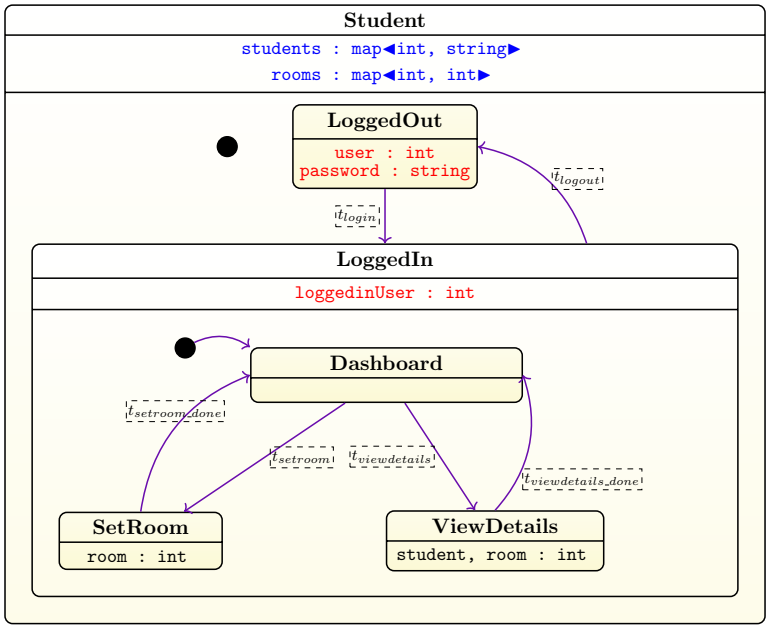
State **B**ased **L**anguage (StaBL)[1] [2]inspired by Statecharts - a language with various formal semantics [4, 43]. StaBL [38] was developed with the motive of enhancing its characteristics, such as modularity and scalability, through controlled data flow. The presence of variables with different storage classes has necessitated a reexamination of its semantics. In this chapter, we describe the operational semantics of StaBL, along with a modularity metric, and present our experience in using it in various applications. We have implemented simulator as per our semantics.

3.1.1 A simple website in StaBL

Figure FC3.1 shows an example model written in StaBL. We have shown the logical structure of the model in Figure FC3.1(a) (in our current implementation,

[1]The content of this chapter is based on: Sujit Kumar Chakrabarti and Karthika Venkatesan. "Stabl: Statecharts with local variables." In Proceedings of the 13th Innovations in Software Engineering Conference on Formerly Known as India Software Engineering Conference, ISEC 2020, doi: https://doi.org/10.1145/3385032.3385040

[2]The content of this chapter is based on: Venkatesan, Karthika, and Sujit Kumar Chakrabarti. "StaBL-State Based Language for Specification of Web Applications." arXiv preprint arXiv:1901.02188 (2019). doi: https://doi.org/10.48550/arXiv.1901.02188



(a)

Transition	Code
t_{login}	**trigger** : eLogin **guard** : get_map◀int, string▶(students, Student.LoggedOut.user) : = Student.LoggedOut.password **action** : Student.LoggedIn.loggedinUser←Student.LoggedOut.user
t_{logout}	**trigger** : eLogout **action** : Student.LoggedOut.user←0 : Student.LoggedOut.password←""
$t_{setroom}$	**trigger** : eSetRoom
$t_{viewdetails}$	**trigger** : eViewDetails
$t_{setroom\_done}$	**trigger** : eDoneSetting
$t_{viewdetails\_done}$	**trigger** : eDoneViewing

State	Code
Student	**entry** : students ← put_map◀int, string▶ (students, 1, "p1") : students ← put_map◀int, string▶(students, 2, "p2")
LoggedOut	**entry** : user ← input◀int▶() : password ← input◀string▶()
SetRoom	**exit** : rooms ← put_map◀int, int▶(rooms, loggedinUser, room)
ViewDetails	**entry** : student ← loggedinUser; : room ← get_map◀int, int▶(rooms, loggedinUser)

(b)

Figure FC3.1: StaBL model for simple website

we use a linear concrete syntax akin to regular programming languages; but a pictorial view is more readable to an unaccustomed reader). The model describes a simple web application named **Student** with two main states: **LoggedOut** and **LoggedIn**. A transition t gets enabled when the machine is the source state of t, the event on its trigger takes place and its guard evaluates to true.

LoggedIn internally has sub-states **SetRoom** and **ViewDetails**. StaBL states are all OR states, which means that at any time the state machine is allowed to be in only one configuration involving a leaf state and all its ancestors. The AND states are detailed in chapter 4 (as a result of considerable alterations in semantics, the details of AND states are presented as a distinct piece). All these features are inspired by traditional Statecharts. The corresponding statechart in textual format is available in appendix C.

Now, we briefly discuss some of the design choices that were considering while creating StaBL and present the features which are distinct from traditional Statecharts.

Local variables and Scoping. States may have local variables, e.g. in Figure FC3.1, **LoggedOut** has two local variables `user` and `password`. Local variables are important to ensure a scalable development in the same manner as in the case of programming languages. Variable declarations can occur at two types of places: within states and within action code blocks (both in transition actions and in entry/exit blocks of states). StaBL has 3 storage classes of local variables - static, param and local. `students` and `rooms` are of storage class static.

Data passing. Many applications are characterised by data-flow between the components. The precise mechanisms may vary from normal client side shared variables to message passing through the server. Hence, it is required to permit controlled access to local variables of states from outside it. `loggedInUser` is of

parameter type which is set in the action code of incoming transition t_{login}.

3.2 Terminologies

In this section, we present the terminologies that are needed for explaining the operational semantics of the StaBL language. The presence of local, parameter and static variables influences the semantics. A simulator has been implemented complying to this semantics, that can be used for simulating the models written in StaBL language.

3.2.1 Execution state

The execution state – which we will refer to as *e-state* as opposed to *state*, the structural construct of a statechart model – is a 4-tuple $(\sigma, \mathbb{C}, \mathcal{H}, \mathbb{V})$. Here, σ is the value environment, $\mathbb{C}$ is the configuration, $\mathcal{H}$ is the state history and $\mathbb{V}$ is the value history.

The execution of a statechart model can be thought of as a sequence of values of the *e-state* effected by various execution events, e.g. entry to and exit from a state, activation of a transition etc.

3.2.2 Value environment

Value environment(σ) defines the mapping between variables and their value. We sometimes represent an environment like a list, e.g. $\sigma = [x_1 \mapsto v_1, x_2 \mapsto v_2, ...]$ to indicate that σ binds x_1 to v_1, x_2 to v_2 and so on. The value of a variable x is bound to is extracted by $\sigma[x]$. If x is not bound in σ, $\sigma(x) = \diamond$, the undefined value. If a list has two bindings to the same variable, the former prevails, and the

latter is shadowed. We use $\sigma[x \mapsto v]$ or $(x \mapsto v) :: \sigma$ to represent an environment which binds x to the value v and is identical to σ for all other variables.

Value environments are also used as type environments during type checking, wherein the values are types. For the entire model, we maintain a global value environment, again denoted by σ, that maps all variables to their current value during execution.

In Figure FC3.1, The value environment after the initialisation phase the entry action of *Student* and its default state *LoggedOut* is executed. The corresponding σ_0 is composed of $\sigma_{Student.a_{\mathcal{N}}}$ (the value environment obtained by executing the entry action of **Student** state) and $\sigma_{LoggedOut.a_{\mathcal{N}}}$ (the value environment obtained by executing the entry action of **LoggedOut** state). Thus, the σ_0 obtained after the initialisation of statechart model is, $\sigma_0 = [user \mapsto v1, password \mapsto v2] :: [students \mapsto (1 \mapsto p1, 2 \mapsto p2)]$.

3.2.3 Configuration

Configuration ($\mathbb{C}$) is the sequence of states $[s_1, s_2, .., s_n]$ where $s_1 \in S_A$, for $\forall i = \{1, ..., n-1\}$, $s_n = sc$ and $s_i \prec_1 s_{i+1}$. This represents the group of hierarchical states which are active at the moment. if $\mathbb{C}$ does not contain an atomic state then the configuration is unstable. As the states are entered or exited transitively, configuration is updated(as shown in rules in Figure FC3.3, Figure FC3.4 and Figure FC3.5)

In Figure FC3.1, the initial configuration is $[LoggedOut, Student]$. On transition t_{login}, the configuration changes to $[Dashboard, LoggedIn, Student]$. Here, *Dashboard* and *LoggedOut* are atomic states.

3.2.4 State history

At any point of time during the execution of the statechart model, the state history ($\mathcal{H}$) (or *history* in short) maps a state to its child state which was exited most recently. In all other cases (viz. for atomic states and for non-atomic states which have not been entered), for any state s, $\mathcal{H}[s] = \diamond$.

In Figure FC3.1, if the state $LoggedIn$ has history marker(i.e., $LoggedIn.h = true$). Initially, when the state has not been entered, $\mathcal{H}[LoggedIn] = \diamond$. Once the state is exited, then depending on the last active substate, $\mathcal{H}[LoggedIn]$ is updated to either of $Dashboard$, $SetRoom$ or $ViewDetails$.

3.2.5 Value history

At any point of time during the execution of the statechart model, the value history ($\mathbb{V}$) maps a variable to the value it held at the time of exiting the state in which it is defined. For states which have not been entered yet, for all variables $x \in s.\mathcal{V}$, $\mathbb{V}[x] = \diamond$.

In Figure FC3.1, the value history of $LoggedIn$ state is updated, $\forall x \in LoggedIn.\mathcal{V}$, $\mathbb{V}[x] = \sigma[x]$.

3.2.6 Environments

Much of the type checking in StaBL pertains to the action language which in turn is identical to any well-known imperative programming language. However, the interstate dataflow feature of StaBL brings about an important difference w.r.t. typical lexically scoped languages. The difference is in how the value/type environments are determined when the code in any action code is being executed/type

checked.

When an action is being executed/type checked, we look up for value/type bindings to variables in two different environments: the read environment (σ_R) and the write environment (σ_W). For each element like state and transition, σ_R and σ_W are projections of the value environment σ using variable lists V_R (variables that can be read) and V_W (variables that can be written to). For a state s, $V_R = V_W$, and is the list of all variables declared in it, in its parent state and so on all the way up to the outermost state. $V_R(s) = V_W(s) = s_1.\mathcal{V} :: s_2.\mathcal{V} :: ... :: s_n.\mathcal{V}$. Here, for $i \in \{1...n-1\}$, $s_i \prec_1 s_{i+1}$, $s_1 = s$ and $s_n = sc$. For a transition t, $V_R(t) = t.s.\mathcal{V} \cup V_R(t.s \sqcup t.d)$ and $V_W(t) = t.d.\mathcal{V}_p \cup V_W(t.s \sqcup t.d)$. Note that V_R and V_W are static features of a state/transition that stay constant for a model. For instance, in Figure FC3.1, for the state $LoggedIn$ $V_R(LoggedIn) = V_W(LoggedIn) = LoggedIn.\mathcal{V} \cup Student.\mathcal{V}$. The transition t_{logout}, $V_R(t_{logout}) = V_R(LoggedIn)$ and $V_W(t_{logout}) = LoggedOut.\mathcal{V}_p \cup V_W(Student)$.

On the other hand, σ_R and σ_W for state/transition are dynamic values, since σ is dynamic. For an element (state/transition) el, σ_R and σ_W are then computed as follows:

$$\sigma_R(el) = \sigma|_{V_R(el)} = [\forall x \in V_R(el),\ x \mapsto \sigma[x]]$$

$$\sigma_W(el) = \sigma|_{V_W(el)} = [\forall x \in V_W(el),\ x \mapsto \sigma[x]]$$

For any piece of action code, we define the following functions:

1. $use(a)$ gives the set of variables which are used/read from in a.
2. $def(a)$ gives the set of variables which are defined/updated/written to in a.

For example, for the action code a: $x \leftarrow y+1$, $def(a) = \{x\}$ and $use(a) = \{y\}$. All variables in $use(a)$ are required to be bound in σ_R and all variables in $def(a)$

in σ_W.

The execution of an action code a can be represented using the following rule:

$$\textsc{Execute-Action}\quad \frac{\forall x \in def(a)\ \sigma_W[x] \neq \diamond \qquad \forall x \in use(a)\ \sigma_R[x] \neq \diamond}{\sigma \vdash a \longrightarrow (\diamond, \sigma')}$$

That is, when all variables in $def(a)$ ($use(a)$) are bound in σ_W (σ_R), executing an action a under environment σ would lead to a modified global value environment σ'. For a, $\sigma_{R/W} = \sigma|_{V_{R/W}(el)}$ where a is an action code on element el (i.e. $(el \in S \implies a = el.a_{\mathcal{N}} \vee a = el.a_{\mathcal{X}})$ and $(el \in T \implies a = el.a)$).

Local variables tend to make the coupling between the core statechart model and the action language tighter. However, σ_R and σ_W act as the interface between the statechart model and the action language. Execution (or typechecking) for every individual fragment of action code is done by invoking the simulation (or typechecking) routine of action language with σ_R and σ_W passed as parameters. While the StaBL simulator (semantic analyser) maintains the environments, it is able to freely communicate with the action language simulator (typechecker) at the relevant parts without having to know its details through σ_R and σ_W. This allows maintaining loose coupling between the core statechart model and the action language.

3.3 Operational Semantics

The operational semantics explained in the remainder of this section have been summarised in Figure FC3.3, Figure FC3.4 and Figure FC3.5. We use the well-known rules of inference to present the big-step operational semantics. The big step is denoted by $\longrightarrow$. Certain rules in this chapter also contain rules that are in small-step semantics but are combined as a big-step. The e-state under which all execution steps happen is a 4-tuple $\sigma, \mathbb{C}, \mathcal{H}, \mathbb{V}$ as mentioned earlier. The result of a big step is a 4-tuple comprising updated values of the 4 components of e-state. In the remaining subsections of this section, we highlight the important aspects of these semantics. In particular, we describe the evolution of scope and value history, both associated with the idea of local variables, while running a statechart model. The architecture of the StaBL simulator built based on the semantics described in this section is shown in Figure FC3.2.

3.3.1 History

When the statechart model enters a state s after having exited it earlier, there are two aspects of the e-state at the time of the previous exit from s which influence the future execution of the statechart model. One is which substate of s the chart was in when s was exited previously (recorded in the state history environment $\mathcal{H}$). The other is the values of the variables at that time (recorded in the value history environment $\mathbb{V}$).

For a non-atomic state, the history will record the last substate exited. When s is entered as the destination of a transition or one of its descendants, the substate of s which will be entered is $\mathcal{H}[s]$. Note that $\mathcal{H}$ will contain the appropriate substate based on whether $s.h = true$. For atomic states, $\mathcal{H}[s] = \diamond$.

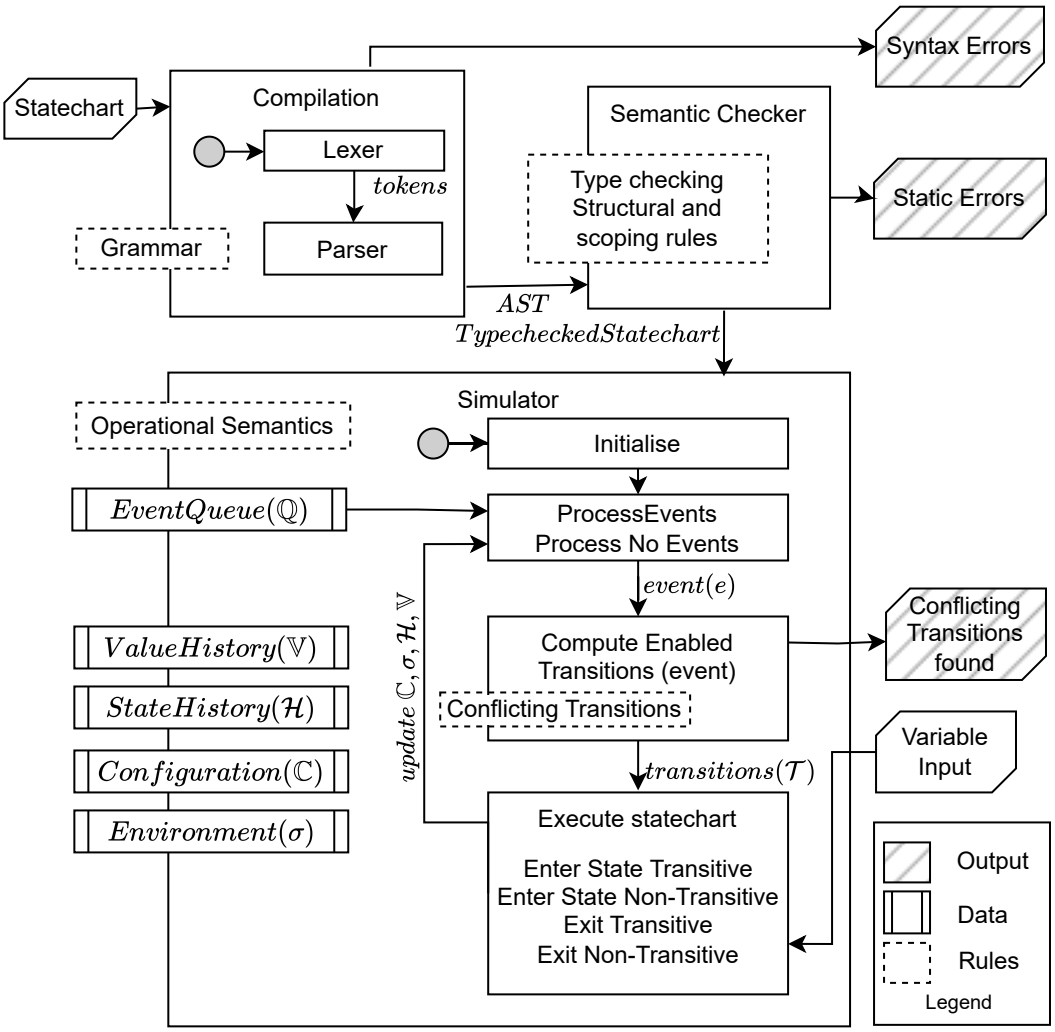


Figure FC3.2: StaBL architecture

When a state s is exited, the value of all its variables ($v \in s.\mathcal{V}$) is written back into the value history environment $\mathbb{V}$. When a state is entered, the value of its static variables is fetched from $\mathbb{V}$. If the values of parameter variables $x \in s.\mathcal{V}_p$ are not set along the incoming transition (e.g. when the destination state of an incoming transition is a substate of s), even if they are fetched from $\mathbb{V}$.

While σ, $\mathbb{C}$ and $\mathcal{H}$ are environments capturing the traditional ideas of Statechart language, $\mathbb{V}$ exists completely as a result of introducing local variables.

3.3.2 State transitions

State transition may happen when a statechart model consumes (or processes) an event. Events in StaBL are all external. On arrival of an event $e \in E$, the chart may make a state transition. If any of the outgoing transitions from the current configuration has the same event as e ($t.e = e$), and its guard evaluates to true ($\sigma_R(t) \vdash t.g \longrightarrow true$), that transition gets is said to be activated. When a transition t fires, the statechart model exits its current state s such that $s \prec t.s \vee s = t.s$ and enters another state d such that $d \prec t.d \vee d = t.d$. The code executed along the way is $s_1.a_{\mathcal{X}}$, $s_2.a_{\mathcal{X}}$, ..., $s_n.a_{\mathcal{X}}$, $t.a$, $d_1.a_{\mathcal{N}}$, $d_2.a_{\mathcal{N}}$, ..., $d_i.a_{\mathcal{N}}$, $t.d.a_{\mathcal{N}}$, $d_{i+1}.a_{\mathcal{N}}$, ..., $d_m.a_{\mathcal{N}}$. Here $[s_1, s_2, ..., s_n, ...]$ is the original configuration where $s_n \prec_1 t.p$. Likewise, $[d_m, d_{m-1}, ..., d_i, d_j = t.d, ..., d_1]$ is the resultant configuration where $d_1 \prec_1 t.p$ and $d_j = t.d$. For $[d_m, d_{m-1}, ..., d_j]$ part of the configuration, for $j \leq k < m$, $d_{k+1} = \mathcal{H}[d_k]$. In other words, every substate of any state s in the resulting configuration down from $t.d$ is d_k's last exited substate, if d_k has a history tag, or is d_k's initial substate $d_k.i$ as explained in section 3.3.1.

3.3.3 State entry and exit

The machine enters any state d as a result of a transition t being taken. This phenomenon can be further be divided into two cases:

1. when $(t.d \preceq d) \wedge (d \preceq t.s \sqcup t.d)$. In other words, $d \in [d_1, d_2, ..., d_j]$ where $t.d \prec_1 d_j \prec_1 ... \prec_1 d_1 \prec_1 (t.s \sqcup t.d)$. In this case, entry to d happens in a non-transitive way (Enter-State-Non-Transitive), i.e. only the variable initialisation and configuration update happen; history is not referred.

2. when $d \preceq t.d$. In other words, $d \in [d_{j+1}, d_{j+2}, ..., d_m]$ where $d_m \in S_A$ and $d_m \prec_1 t_{m-1} \prec_1 ... \prec_1 d_{j+1} \prec_1 d_j = t.d$. Once the machine enters $t.d = d_j$, it must descend all the way up to an appropriate atomic descendent state of $t.d$. This is done by making a transitive entry (Enter-State-Transitive) into $t.d$. That is, after $t.d$ is entered, the machine enters $\mathcal{H}[t.d]$, then $\mathcal{H}[\mathcal{H}[t.d]]$ and so on.

An important side-effect of a state entry is on the value environment σ. When entering a state s, all local variables bindings are set up as per their respective storage-class. For example, local variables ($x \in s.\mathcal{V}_l$) are initialised to their default values; parameter variables defined during the incoming transition are initialised to that value. All other variables (parameters not initialised on the transition and static variables) are initialised with their last value (if available), or with their default value otherwise, both are available in the value history $\mathbb{V}$.

When exiting a state s (Exit-State), the current values of all parameters and static variables of the state are written back to the value history and the parent state history is updated with s.

Entering and Exiting States

ENTER-STATE-NON-TRANSITIVE

$$\frac{\begin{array}{c} x_1, x_2, ..., x_p \in s.\mathcal{V} \\ \forall x_j \in \{x_1, ..., x_p\},\ \mathbb{V}[x_j] = v_j \qquad \sigma \vdash s.a_{\mathcal{N}} \longrightarrow (\diamond, \sigma') \qquad \mathbb{C}' = s :: \mathbb{C} \end{array}}{\sigma, \mathbb{C}, \mathcal{H}, \mathbb{V} \vdash enter_{NT}(s) \longrightarrow (\sigma', \mathbb{C}', \mathcal{H}, \mathbb{V})}$$

ENTER-STATE-TRANSITIVE

$$\frac{\begin{array}{c} x_1, x_2, ..., x_n \in s.\mathcal{V}_l \qquad x_{n+1}, ..., x_{n+m} \in s.\mathcal{V}_p \cup s.\mathcal{V}_s \\ \forall x_j \in \{x_{n+1}, ..., x_{n+m}\},\ \mathbb{V}[x_j] = v_j \qquad \sigma \vdash s.a_{\mathcal{N}} \longrightarrow (\diamond, \sigma') \\ \mathbb{C}' = s :: \mathbb{C} \qquad \mathcal{H}[s] = s' \qquad \sigma', \mathbb{C}', \mathcal{H}, \mathbb{V} \vdash enter_T(s', []) \longrightarrow (\sigma'', \mathbb{C}'', \mathcal{H}, \mathbb{V}) \end{array}}{\sigma, \mathbb{C}, \mathcal{H}, \mathbb{V} \vdash enter_T(s, [x_1 \mapsto v_1, ..., x_n \mapsto v_n]) \longrightarrow (\sigma'', \mathbb{C}'', \mathcal{H}, \mathbb{V})}$$

EXIT-STATE

$$\frac{\begin{array}{c} x_1, x_2, ..., x_n \in s.\mathcal{V}_p \cup s.\mathcal{V}_s \\ \sigma[x_1] = v_1, \sigma[x_2] = v_2, ..., \sigma[x_m] = v_m \\ \sigma \vdash s.a_{\mathcal{X}} \longrightarrow (\diamond, \sigma') \\ \mathbb{C} = s :: \mathbb{C}' \qquad \mathcal{H}' = \mathcal{H}[s.p \mapsto s] \text{ if } s.h = true \\ \mathcal{H}' = \mathcal{H} \text{ if } s.h = false \\ \mathbb{V}' = \mathbb{V}[x_1 \mapsto v_1, x_2, \mapsto v_2, ..., x_m \mapsto v_m] \end{array}}{\sigma, \mathbb{C}, \mathcal{H}, \mathbb{V} \vdash exit(s) \longrightarrow (\sigma', \mathbb{C}', \mathcal{H}', \mathbb{V}')}$$

Figure FC3.3: Operational Semantics of StaBL: Entering and Exiting States

3.3.4 Event processing and run

A StaBL statechart model processes one event every clock tick. Regardless of whether it results in a state transition to take place (PROCESS-EVENT-SUCCESS) or not (PROCESS-EVENT-FAILURE), the event thus processed is considered consumed. If multiple transitions are enabled (i.e. $t.s \in \mathbb{C} \wedge e = t.e \wedge t.g = true$), then the transition to be taken is chosen randomly. Most Statecharts implementations follow their specific semantics in such cases. We find a random choice the least restrictive. We call such a situation as *conflicting transitions* – something that may correspond to an issue/defect in the model, hence that should be brought to the notice of the creator of the model, rather than imposing a default semantics.

The complete run (RUN) of a statechart model is comprised of initialisation (INIT) followed by processing a sequence of events (PROCESS-EVENTS). The run halts when an empty event sequence is presented (PROCESS-EVENTS-EMPTY).

For instance, in Figure FC3.1, the initial big step tuple is $(\emptyset, \emptyset, \emptyset, \emptyset)$. Let $[eLogin, tSetRoom, tLogout]$ be the event list to be processed.

After INIT rule in Figure FC3.4, the default state **LoggedOut** is transitively entered by executing the entry actions of all states from the root state **Student**. The maps `students` and `rooms` are initialised to default value when entering the state **Student**(as we described in the section 3.3.3. Then, the entry action of the state is executed, which add two values into the map. Similarly the variables *user* and *password* are initialised when entering the state **LoggedOut**. Then, when executing the entry action, variables *user* and *password* are taken as input (let us assume the input was $username \leftarrow 1, password \leftarrow p1$) and the environment is updated. The resulting tuple is $\sigma_m, \mathbb{C}_m, \mathcal{H}, \mathbb{V}$. Here, σ_m is $[user \mapsto 1, password \mapsto p1] :: [students \mapsto (1 \mapsto p1, 2 \mapsto p2)]$. $\mathbb{C}_m$ is $[LoggedOut, Student]$. $\mathcal{H}$ is $[Student \mapsto LoggedOut]$. $\mathbb{V}$ is σ_m. The actual σ after completion of the step, will contain all the updates that happened through the execution, however for the sake of comprehensiveness only the latest valuation is shown here as they shadow the earlier valuations, same holds good for the value environment as well. Here, INIT rule is used. The further execution proceeds with rule PROCESS-EVENTS.

After $process(eLogin)$, the *guard* is checked (i.e, if the inputted *username* and *password* is present in the *students*). As it evaluates to true, the corresponding exit of **LoggedOut**, transition action and entry action code of **LoggedIn** and **Dashboard** are executed. As the entry action, the $loggedinUser \leftarrow \sigma[user]$ is executed, the big step completes with the configuration $[Dashboard, LoggedIn, Student]$. The resulting tuple is $\sigma', \mathbb{C}', \mathcal{H}', \mathbb{V}'$. Here, σ' is updated with the valuation of new

variables, $[loggedinUser \mapsto 1] :: [user \mapsto 1, password \mapsto p1] :: [students \mapsto (1 \mapsto p1, 2 \mapsto p2)]$. $\mathbb{C}'$ is $[Dashboard, LoggedIn, Student]$. $\mathcal{H}'$ is $\mathcal{H}$. $\mathbb{V}'$ is $[user \mapsto 1, password \mapsto p1]$. Here Enter-State-Transitive and Process-event-success rules are used.

After $process(tSetRoom)$, there is no exit action in $Dashboard$ and entry action in $SetRoom$. The variable in the $SetRoom$ is initialised to default value and added to the σ - $[room \mapsto 0] :: [loggedinUser \mapsto 1] :: [user \mapsto 1, password \mapsto p1] :: [students \mapsto (1 \mapsto p1, 2 \mapsto p2)]$. The new $configuration$ is $[SetRoom, LoggedIn]$. All other valuations remain the same. Here Enter-State-Non-Transitive and Process-event-success rules are used.

After $process(eLogout)$, two states are exited, $SetRoom$ and $LoggedIn$. The respective exit action, entry actions and transition action is executed. The $rooms$ map is updated with $\sigma[user] \mapsto \sigma[room]$ (i.e, $1 \mapsto 0$). The $user$ and $password$ is reset in the transition action and the new $user$ and $password$ is taken as input when the state is entered. If the state $LoggedIn$, had history set to true, then $\mathcal{H}$ is updated with $\mathcal{H}[LoggedIn] = SetRoom$. Here, Enter-State-Non-Transitive and Process-event-success rules are used.

When, processing an event, if there is no transition becomes enabled - the rule Process-event-failure is used. Then, as all events are processed, the event list becomes empty - it is shown in rule process-empty-events. *Halt* denotes that the execution is halted/ stopped.

Transitions and Run - Part 1

INIT

$$\frac{\begin{array}{c} s_1, s_2, ..., s_n \in S_{NA} \\ t_1, t_2, ..., t_m \in S \quad t_1 \in S_A \quad t_m.\tau = statechart \quad t_1 = t_2.I \quad t_2 = t_3.I \\ ... \quad t_{m-1} = t_m.I \quad \mathcal{V}_I = \{x_1, ..., x_n\} = \text{set of initialised variables in the model} \\ \mathcal{V}_U = \{x_{n+1}, ..., x_{n+m}\} = \text{set of uninitialised variables in the model} \\ \forall x \in \mathcal{V}_U,\ x \mapsto init(type(x)) \\ \mathbb{C} = [] \quad \mathcal{H} = \{s_1 \mapsto s_1.I, s_2 \mapsto s_2.I, ...s_n \mapsto s_n.I\} \\ \sigma = \mathbb{V} = [x_1 \mapsto i_1, ..., x_n \mapsto i_n, x_{n+1} \mapsto i_{n+1}, ..., x_{n+m} \mapsto i_{n+m}] \\ \sigma, \mathbb{C}, \mathcal{H}, \mathbb{V} \vdash enter_T(t_m) \longrightarrow (\sigma_m, \mathbb{C}_m, \mathcal{H}, \mathbb{V}) \end{array}}{\phi, \phi, \phi, \phi \vdash initialise() \longrightarrow (\sigma_m, \mathbb{C}_m, \mathcal{H}, \mathbb{V})}$$

TRANSITION

$$\frac{\begin{array}{c} \mathbb{C} = [s_1, s_2, ..., s_i, ..., s_n, t.s \sqcup t.d, ..., sc] \\ \mathbb{C}' = [d_m, d_{m-1}, ..., d_j, ..., d_1, t.s \sqcup t.d, ..., sc] \quad sc.\tau = statechart \\ s_i = t.s \quad d_j = t.d \quad \forall k = [j, j+1, ..., m-1]\ \mathcal{H}[D_k] = D_{k+1} \\ \sigma_{s_1}, \mathbb{C}_{s_1}, H_{s_1}, H_{vs_1} \vdash exit(s_1) \longrightarrow (\sigma'_{s_1}, \mathbb{C}'_{s_1}, \mathcal{H}'_{s_1}, \mathcal{H}'_{vs_1}) \\ ... \quad \sigma'_{s_{n-1}}, \mathbb{C}'_{s_{n-1}}, \mathcal{H}'_{s_{n-1}}, \mathcal{H}'_{vs_{n-1}} \vdash exit(s_n) \longrightarrow (\sigma'_{s_n}, \mathbb{C}'_{s_n}, \mathcal{H}'_{s_n}, \mathcal{H}'_{vs_n}) \\ \sigma_{d_1}, \mathbb{C}_{d_1}, H_{d_1}, H_{vd_1} \vdash enter_{NT}(d_1) \longrightarrow (\sigma'_{d_1}, \mathbb{C}'_{d_1}, \mathcal{H}'_{d_1}, \mathcal{H}'_{vd_1}) \\ ... \quad \sigma'_{d_{j-1}}, \mathbb{C}'_{d_{j-1}}, \mathcal{H}'_{d_{j-1}}, \mathcal{H}'_{vd_{j-1}} \vdash enter_T(d_j) \longrightarrow (\sigma'_{d_j}, \mathbb{C}'_{d_j}, \mathcal{H}'_{d_j}, \mathbb{V}_{d_j}) \\ \sigma' = \sigma'_{d_j} \quad \mathbb{C}' = \mathbb{C}'_{d_j} \quad \mathcal{H}' = \mathcal{H}'_{d_j} \quad \mathbb{V}' = \mathbb{V}_{d_j} \end{array}}{\sigma, \mathbb{C}, \mathcal{H}, \mathbb{V} \vdash fire(t) \longrightarrow (\sigma', \mathbb{C}', \mathcal{H}', \mathbb{V}')}$$

Figure FC3.4: Operational Semantics of StaBL: Transitions and Run - Part 1

3.4 Modularity metrics

3.4.1 Modularity

Modularity in statecharts refers to the design principle that allows the decomposition of a complex statechart into smaller, manageable sub-statecharts or modules. This approach enhances the readability, maintainability, and reusability of the statechart models by encapsulating specific functionality within distinct modules. Each module operates semi-independently, allowing designers to focus on one part of the system without being overwhelmed by the entire complexity. Modularity facilitates easier updates and debugging, as changes in one module can

Transitions and Run - Part 2

PROCESS-EVENT-SUCCESS

$$\frac{T' \subseteq T \qquad \forall t \in T'\ t.s \in \mathbb{C} \wedge (e = t.e) \wedge (\sigma_R(t) \vdash t.g = true) \qquad t' = random(T') \qquad \sigma, \mathbb{C}, \mathcal{H}, \mathbb{V} \vdash fire(t') \longrightarrow (\sigma', \mathbb{C}', \mathcal{H}', \mathbb{V}')}{\sigma, \mathbb{C}, \mathcal{H}, \mathbb{V} \vdash process(e) \longrightarrow (\sigma', \mathbb{C}', \mathcal{H}', \mathbb{V}')}$$

PROCESS-EVENT-FAILURE

$$\frac{\nexists t \in T \text{ such that } t.s \in \mathbb{C} \wedge (e = t.e) \wedge (\sigma_R(t) \vdash t.g = true)}{\sigma, \mathbb{C}, \mathcal{H}, \mathbb{V} \vdash process(e) \longrightarrow (\sigma', \mathbb{C}, \mathcal{H}, \mathbb{V})}$$

PROCESS-EVENTS

$$\frac{\sigma, \mathbb{C}, \mathcal{H}, \mathbb{V} \vdash process(e_1) \longrightarrow (\sigma', \mathbb{C}', \mathcal{H}', \mathbb{V}') \qquad \sigma', \mathbb{C}', \mathcal{H}', \mathbb{V}' \vdash run([e_2, ..., e_n]) \longrightarrow (\sigma'', \mathbb{C}'', \mathcal{H}'', \mathbb{V}'')}{\sigma, \mathbb{C}, \mathcal{H}, \mathbb{V} \vdash process([e_1, e_2, ..., e_n]) \longrightarrow (\sigma'', \mathbb{C}'', \mathcal{H}'', \mathbb{V}'')}$$

PROCESS-EMPTY-EVENTS

$$\frac{}{\sigma, \mathbb{C}, \mathcal{H}, \mathbb{V} \vdash process([]) \longrightarrow Halt}$$

RUN

$$\frac{\phi, \phi, \phi, \phi \vdash initialise() \longrightarrow (\sigma, \mathbb{C}, \mathcal{H}, \mathbb{V}) \qquad \sigma, \mathbb{C}, \mathcal{H}, \mathbb{V} \vdash process([e_1, ..., e_n]) \longrightarrow (\sigma', \mathbb{C}', \mathcal{H}', \mathbb{V}')}{\phi, \phi, \phi, \phi \vdash run([e_1, e_2, ..., e_n]) \longrightarrow (\sigma', \mathbb{C}', \mathcal{H}', \mathbb{V}')}$$

Figure FC3.5: Operational Semantics of StaBL: Transitions and Run - Part 2

be made with minimal impact on others. Moreover, it supports hierarchical statecharts, where states can contain nested sub-statecharts, promoting a structured and scalable approach to modelling dynamic systems. This decomposition aligns with principles of software engineering, ensuring that complex behaviours are broken down into more understandable and manageable pieces. By breaking down a complex system into smaller statecharts representing well-defined functionalities, the overall model becomes easier to understand and manage. As a system evolves and new functionalities are added, modular statecharts make it easier to scale the model. New statecharts can be seamlessly integrated with existing ones, with-

out major rework of the entire system. This allows the statechart to effectively represent the growing complexity of the system it models.

3.4.2 Effects of local variables on modularity

Local variables aid in achieving core aspects of modularity: loose coupling, reduced complexity, improved maintainability, and reusability. By keeping data local, modules operate more independently, contributing to a well-structured and scalable system.

Encapsulation Local variables confine data access within a module, making them visible and usable only within that module's scope. This restriction prevents unintended modifications by external code, thereby promoting loose coupling and ensuring data integrity.

Reduced Complexity Local variables contribute to isolating functionality within a module. By keeping data specific to the module's tasks, the overall logic becomes clearer and less error-prone. Analogous to well-organized rooms, local variables help maintain designated spaces within a module, avoiding clutter and confusion.

Improved Maintainability The self-contained nature of local variables means changes within a module are less likely to impact other system parts. This enhances the ease of understanding, modifying, and debugging individual modules without causing unintended side effects.

Promotes Reusability Modules that utilize only local variables are more self-sufficient and independent of external data. This characteristic facilitates their reuse in different contexts without modification, akin to reusable tools,

where a local variable functions like a built-in wrench specific to the tool's purpose.

Local variables allow for controlled data sharing within a specific function or block. We can pass necessary data as arguments to the function, and it can operate on that data without affecting variables in other parts of the program. This reduces the risk of accidental modifications and makes the code easier to understand. The scope of the variables - local, parameter and static helps to achieve the same. Thus, we proceed by defining how these variables are accessed in the model to show the modularity benefits in the system models written in StaBL language.

3.4.3 Scope score

As for the enhancement in modularity accrued out of local variables and inter-state data flow, we measured it using a novel metric called *scope score*. We introduce the scope score and how we used it to measure modularity gains in StaBL.

3.4.4 Size.

The size of the model is determined by the standard Statechart size metrics like - NSS: Number of Simple States, NCS: Number of Composite States, NE: Number of Events, NT: Number of Transitions, NA: Number of actions(inclusive of entry, exit actions), NV: Number of Variables

3.4.5 Scope score for a variable.

For any given variable v in the model *spec* declared in a state s, We define,

- $N_S(s)$ as the number of states transitively contained in s, the state in which the variable is declared.

$$N_S(s) = 1 + \sum_{c \in states(s)} N_S(c)$$

- $N_T(s)$ and the number of transitions transitively contained in s.

$$N_T(s) = |T(s)| + \sum_{c \in states(s)} N_S(c)$$

  where, $T(s)$ is $\{t\} \in T$ such that $t.s \sqcup t.d = s$ (the set of transitions directly contained in s).

We measure the following values:

1. $scope_S$: Number of states in which the variable is visible. $scope_S(v) = N_S(s)$
2. $scope_T$: Number of transitions in which the variable is visible. $scope_T(v) = N_T(s)$
3. $Wscope$: The number of regions in which the variable v can be used as l-value in an expression. $Wscope(v) = scope_S(v) + scope_T(v)$. If the variable is of the parameter type, then $|T_{in}(s)|$ is also added to $Wscope$ where T_{in} is the set of all transitions t such that $t.d = s$.
4. $Rscope$: The number of regions in which the variable v can be used as r-value in an expression. $Rscope(v) = scope_S(v) + scope_T(v)$. If the variable is of the type local or static, then $|T_{out}(s)|$ is also added to $Rscope$ where T_{out} is the set of all transitions t such that $t.s = s$.
5. N_W: The number of regions in which the variable v is actually written to (defined).

6. N_R: The number of regions in which the variable v is actually read (used).

7. Here, *region* refers to either state or transition. For instance, even if $a_{\mathcal{N}}$ and $a_{\mathcal{X}}$ access the variable, the number of regions is based on the state that accesses the variable so in this case, N_{access}=1.

and then we determine the *ScopeScore* by the following formulas; Scope score of a variable gives the measure of actual usage of the variable in the model.

$$ScopeScore(v) = \frac{N_W}{Wscope} + \frac{N_R}{Rscope}$$

$$ScopeScore(spec) = \sum_{v \in spec} ScopeScore(v)$$

and the modularity increases when a variable's $Wscope$ and $Rscope$ is small. Hence, the *ScopeScore* is directly proportional to modularity.

$$modularity \propto ScopeScore(spec)$$

3.5 Implementation and experiments

We have implemented a compiler for StaBL [3]that takes a statechart model written in StaBL and generates a control flow graph for the same. Along with local variables and their static semantics, we have incorporated a sophisticated action language (with features like composite data types, function calls, and parametric polymorphism) in this compiler. The compiler parses and type checks the input model and generates its abstract syntax tree. On demand, the compiler further flattens the model into an imperative program which can then be analysed/verified using existing verification tools. This compiler is the basic infrastructure we have

[3]`https://github.com/sujitkc/statechart-verification`

Table TC3.1: Empirical Evaluation of Case study

Specification	**Size**						**ScopeScore**	
	NSS	NCS	NE	NT	NA	NV	StaBL	Statechart
HMS	43	15	58	64	76	102	89	1.687
CS	18	3	25	32	37	46	67	1.416
eHealth	19	5	21	32	24	27	46	0.814
LMS	13	4	11	25	17	31	37	0.708
Sample spec in Figure FC3.1	4	2	6	7	5	8	5.833	1.583

used to conduct the case studies discussed below. We are currently extending the toolchain to incorporate further verification support. We have implemented a simulator for StaBL as an extension of our StaBL toolchain as a realisation of the operational semantics discussed in this chapter.

We have written numerous small (< 10 states) statechart models of applications with StaBL, and have developed a complete model of 4 applications - Curfew System(CS), Hostel Management System(HMS), eHealth System, Library Management System(LMS). We found some advantages in using StaBL as a modelling language for these applications. Many of these advantages, e.g. type safety due to polymorphic type, shifting the focus of Statechart modelling to the action language and making it tightly integrated with the overall modelling notation etc. are abstract and were noticed through experience. The results on use of these metrics on Statechart and StaBL models are presented in Table TC3.1. The sub-columns under the column header **Size** present the size measures of our case-studies as per metrics reported in earlier works.

The main result of our experiment is tabulated under **ScopeScore** column header. Due to the facility of local variables, the $Wscope$ and $Rscope$ in StaBL model is significantly lower than in traditional Statecharts models which do not allow local variables. This manifests in the scope scores of StaBL models (with local variables) which are consistently and significantly higher than that of Statecharts (without local variables). This higher scope score is a significant indicator of

higher modularity of StaBL models as compared to traditional Statecharts. As the containment depth of variables increase in large models, we observe high-margin in *ScopeScore* of StaBL and Statecharts for larger models as compared to smaller ones.

3.6 Summary

We have presented StaBL, a Statechart variant with local variables, through its abstract syntax and operational semantics. In our description, we have emphasised on the impact of local variables on the language where relevant. As it turns out, addition of local variables, though an apparently simple addition to the language, interacts intricately with other features of the language. We have implemented a translator for StaBL and are implementing a simulator following the operational semantics presented in this chapter. We have done a number of modelling case-studies and translated the models into their imperative counterparts. Our general experience corroborates with the expectation that having local variables helps develop more modular models which are easier to build incrementally and maintain subsequently. We have performed experiments to examine the impact of local variables on the modularity of models. For this, we have introduced a novel modularity measure which we call scope score. Scope score measures for statechart models with local variables are consistently better w.r.t. models will all global variables. This indicates better modularity obtained due to the use of local variables.

CHAPTER 4

CONSTABL - CONCURRENT STATE BASED LANGUAGE

In this chapter, we present ConStaBL[1](**Con**current **Sta**te **B**ased **L**anguage) - a StaBL variant that includes concurrency [40, 44]. In all reported work on statechart semantics (detailed discussion on this presented in chapter 8, the modelling semantics is treated separately w.r.t. the action language. However, we believe that to fully treat the concurrency semantics of statecharts, action language cannot be treated as an entirely external feature. Hence, we take a fresh look at the operational semantics of ConStaBL by integrating action language as a part of modelling semantics. For instance, cyber-physical systems (CPS) integrate software and physical processes for real-time control, like in autonomous vehicles and medical devices. While advantageous, concurrency - using multiple processing threads - introduces a vulnerability: concurrency bugs. These bugs, like data races and deadlocks, arise from multiple threads accessing shared resources or interacting with physical components without proper coordination. Imagine an autonomous vehicle experiencing a data race, resulting in erroneous sensor readings or incorrect steering commands. A deadlock could prevent a critical control

[1]The content of this chapter is based on: Venkatesan, Karthika, and Sujit Kumar Chakrabarti. "ConStaBL–A Fresh Look at Software Engineering with State Machines." arXiv preprint arXiv:2307.03790 (2023), doi:https://doi.org/10.48550/arXiv.2307.03790 and "A fresh look on semantics of Concurrent State Based Language (ConStaBL)", Journal of Computer Languages, vol. 79, p. 101270, 2024. doi:https://doi.org/10.1016/j.cola.2024.101270

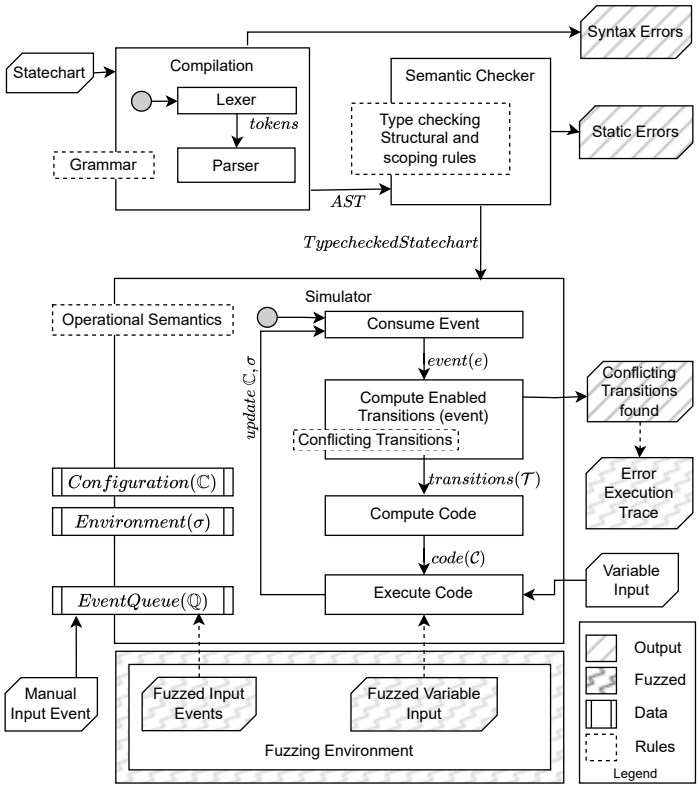


Figure FC4.1: Architecture of ConStaBL Simulator

system from responding to a safety-critical event. The potential consequences of concurrency bugs in CPS are far more severe than traditional software bugs, highlighting the urgent need for robust detection tools to ensure safety and reliability in the design stages.

4.1 Introduction

An overview of the ConStaBL simulator is presented in Figure FC4.1. ConStaBL statecharts have integrated action language and variable scoping. The parsing phase verifies syntax problems and produces an abstract syntax tree (AST). The semantic checker detects static errors in the statechart model, such as typing, and scope errors. The type-checked statechart can be executed through the simula-

tion engine. The execution begins by initialising the statechart to its default state and subsequently consuming an event from the event queue. During execution, processing an event begins with the identification of transitions that are *triggered* by an *event* for which the *guard* evaluates them to be *true*. Afterwards, the execution moves from the source state to the destination state, passing through several intermediate configurations. This is done by running the appropriate action blocks for the states and transitions that have been identified. The process involves generating code for the identified transitions and executing this code in an interleaved manner. Environment (σ) and configuration ($\mathbb{C}$) are updated during execution. The event and input variables are prompted manually during simulation. The execution stops and reports when a *conflicting transitions* are encountered (details on *Conflicting transitions* are presented in section 4.3.4.1); otherwise, execution proceeds as an event occurs. The fuzzing environment is detailed in the chapter 5.

Through this work, we make the following contributions:

1. Concurrent StaBL (ConStabl): an extension of StaBL, a state-based specification language with local variables, to include concurrency.
2. Operational semantics of ConStaBL.
3. Simulator for ConStaBL.

4.2 Structural semantics

All the semantic rules of StaBL are applicable for ConStaBL too. Apart from that we also specifically add the criteria for the inter level transitions for the shell states.

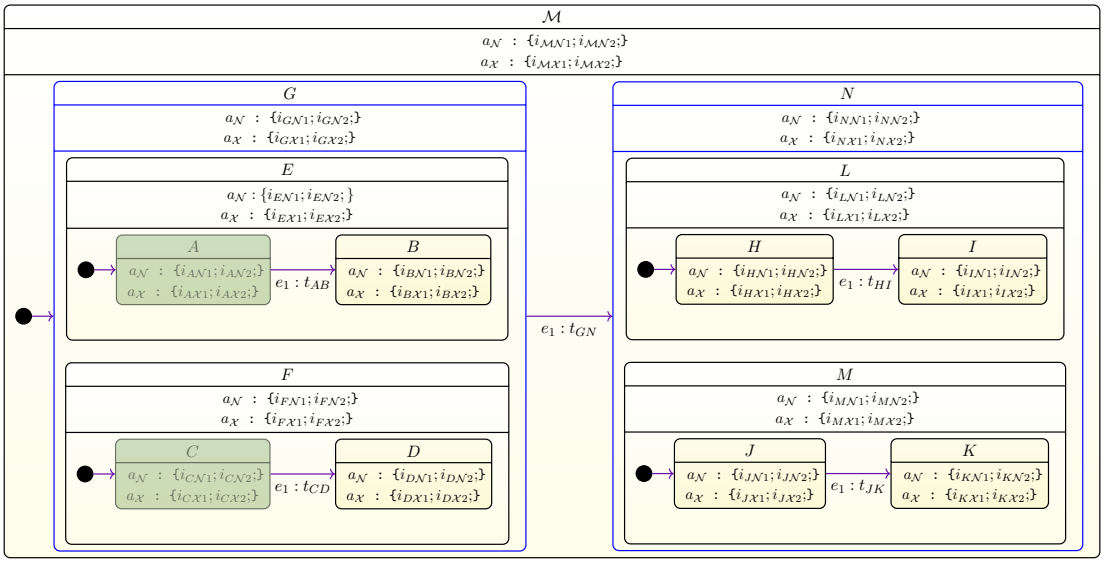


Figure FC4.2: A simple statechart with two shell states - G and N. The configuration is $\{A, C\}$

4.2.1 Inter level transitions.

When the parents of the source and destination of a transition are not the same, it is an inter level transition (i.e., $\exists t \in T | t.s.p \neq t.d.p$).

1. There can be no inter level transitions between the regions of a shell state (i.e., $\nexists t \in T | \sqcup (\{t.s, t.d\}).\tau = shell$).

2. There can be no transition between a descendent and an ancestor (i.e., $\nexists t \in T, (t.s \prec_* t.d) \vee (t.d \prec_* t.s)$).

3. The state of type statechart cannot have any incoming or outgoing transitions (i.e., $\nexists t \in T, t.s.\tau \vee t.d.\tau = statechart$).

These rules were conscious decisions to avoid the inherent conflicts that may arise due to such specifications.

1. We do not permit inter level transitions between regions of a shell state for the simple reason that if a shell state is part of an active configuration, then

all of its substates must also be active in order for the shell state as a whole to be active. If the inter level transition between regions is allowed, then it will deactivate a region and move to another region, which will violate the initial necessary condition of an active shell state. By disallowing inter-level transitions between regions, we guarantee that all substates within a shell state are active simultaneously, preserving the integrity of the configuration.

2. The transition between descendant and ancestor is not permitted. As per our present semantics, if a region has an outgoing transition to its parent shell state, it has the potential to trigger reset behaviour on all other regions, substates, and the like.

Example 6. *In Figure FC4.2,* $\mathcal{M} \prec_1 G$ *means* $\mathcal{M}$ *is the parent of* G *and* $substates(\mathcal{M}) = \{G, N\}$. *Also,* $\mathcal{M}$ *is the ancestor of states like* E, L, M *and* F *(it is also the ancestor of all the states contained within these states). Here,* $E.\tau = F.\tau = composite$ *and* $G.\tau = shell$. *To differentiate shell states, it has been marked with blue outline.*

$$CA(\{A, D\}) = \{G, \mathcal{M}\} \text{ and } CA(\{A, J\}) = \{\mathcal{M}\}$$
$$CCA(\{t_{AB}\}) = \sqcup(\{A, B\}) = E$$
$$CCA(\{t_{AB}, t_{CD}\}) = \sqcup(\{A, B, C, D\}) = G$$

4.3 Operational semantics

In this section, we explain the operational semantics of ConStaBL. When an *event* is consumed, the following four major stages are executed by the simulator. A simulation processes the *events* from the event queue($\mathbb{Q}$) as shown in the algorithm 1.

1. **Find Enabled Transitions:** In this step, we identify the transitions that are enabled for a specific event. If conflicts arise among these transitions, they are reported. (detailed in section 4.3.4.1)

2. **Compute Code:** If no conflicts exist, we proceed to identify the code that must be executed during the simulation step. This is achieved by constructing the corresponding transition state tree. (detailed in section 4.3.4.2)

3. **Code Execution:** Subsequently, we execute the code that has been identified in the previous step, and we update the environment(σ) accordingly. (detailed in section 4.3.4.3)

4. **Update Configuration:** Following the code execution, the configuration is transitioned from the source configuration to the destination configuration. Additionally, the event responsible for triggering this transition is removed from the event queue. (detailed in section 4.3.4.5)

4.3.1 Terminologies

We now introduce the terminologies and definitions that will be used throughout the work to present the operational semantics. In order to succinctly compute the code from the statechart structure, we use an intermediate *tree* data structure and define the operations on it: *subtree*, *sliced subtree*, and *tree map*.

Tree. $\textsc{tree} : node \times tree\ set \rightarrow tree$ is function such that $\textsc{tree}(n, S)$ creates a tree rooted at node n with all trees in set S as subtrees of n.

Subtree. $\mathbb{T} : node \times tree \rightarrow tree$ is a function such that $\mathbb{T}(n, tr)$ gives the subtree rooted at n in a tree tr.

$$\mathbb{T}(n, tr) = \textsc{tree}(n, \{\mathbb{T}(c, tr) \forall c \in childnodes(n)\})$$

Sliced subtree. $\widehat{\mathbb{T}}$: *node* $\times$ *node set* $\times$ *tree* $\rightarrow$ *tree* is a function such that, for any node n, the sliced subtree $\widehat{tr} = \widehat{\mathbb{T}}(n, C)$ w.r.t. to node set C is a subtree of n such that each path in $\widehat{tr}$ from its root goes to a member of C. Here, C is a set of nodes that contains some leaves of $\mathbb{T}(n, tr)$ possibly along with other nodes not in $\mathbb{T}(n, tr)$.

Treemap. *treemap* : $(\alpha \rightarrow \beta) \times \alpha$ *tree* $\rightarrow \beta$ *tree* is a function such that *treemap*(f, t) gives a tree t' that is identical in shape as the t except that the value on each node of t' is $v' = f(v)$ where v is the value on the corresponding node of t.

Example 7. *Let us consider the statechart shown in the Figure FC4.2, the tree representation of the statechart is shown in Figure FC4.3(a). The nodes represent the states, and the edges represent the containment of a state within another. Here we have also represented the transitions between states as arrows between these nodes at different levels. Shell state is represented as rectangle nodes and all other types of states represented by circle nodes. The subtree of the node G is given in Figure FC4.3(b). The sliced subtree* $\widehat{\mathbb{T}}(G, \{A, C\}, tr)$ *gives a tree rooted at G and each path leads to a leaf state in the set* $\{A, C\}$ *as shown in Figure FC4.3(c). The tree map function substitutes each node in the tree to the value provided by the mapping function* $s \rightarrow s.\mathcal{X}$ *as shown in Figure FC4.3(d).*

Based on this, we define *configuration state tree*, *transition state tree*, *source side tree*, *destination side tree*, *control flow graph tree*, *code tree* later. The Figure FC4.4 shows the dependencies between the concepts and terminologies used throughout this work. The dependencies are transitive.

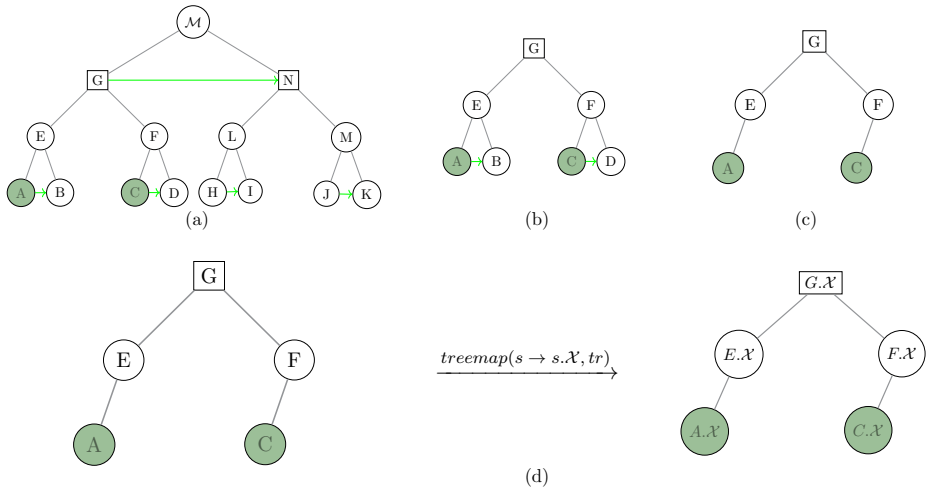


Figure FC4.3: (a) An equivalent state tree (tr) for the statechart in Figure FC4.2; (b) Subtree - $\mathbb{T}(G, tr)$ (c) Sliced Tree $\widehat{tr} = \widehat{\mathbb{T}}(G, \{A, C\}, tr)$ (d) $treemap(f, \widehat{tr})$ maps each node in given tree to the valuation of the function f. Here, $f = s \rightarrow s.\mathcal{X}$ outputs exit code $s.\mathcal{X}$ for any given state s.

4.3.2 Configuration

A configuration ($\mathbb{C}$) is a set of *atomic* states that are active at a given point. In active hierarchical sequential composition, $\mathbb{C}$ is a singleton set and in active concurrent composition, $\mathbb{C}$ is a set of atomic states (please note, here configuration stores only atomic states unlike the configuration we detailed in the chapter on StaBL).

Not all sets of atomic states are ***valid configurations***. A configuration can also be represented as Configuration State Tree (CST) - a tree st, with nodes from the root state to the states specified by the set $\mathbb{C}$. A configuration is valid iff:

1. For each composite state $s \in st$, s must have only one child in st corresponding to one of its substates.

2. For each shell state $s \in st$, s has a child node in st corresponding to each of its regions.

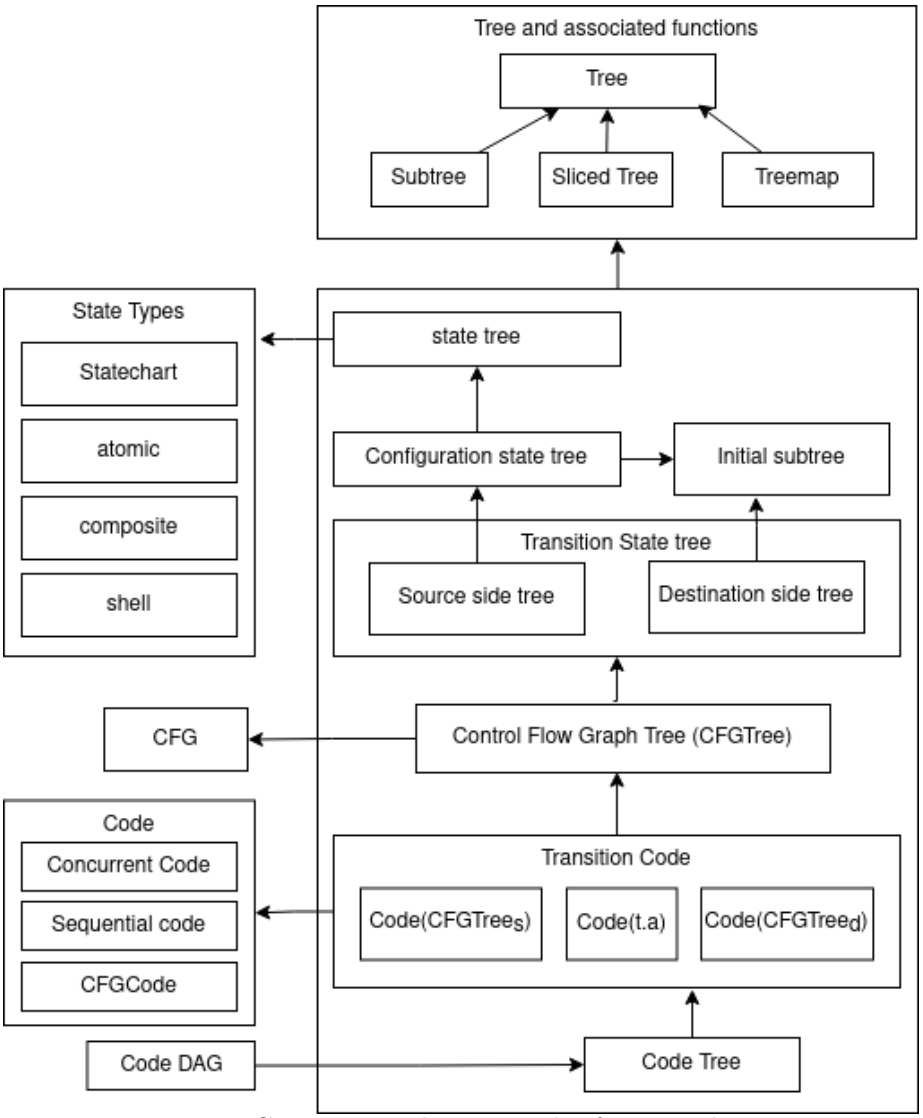


Figure FC4.4: Dependency graph of terminologies

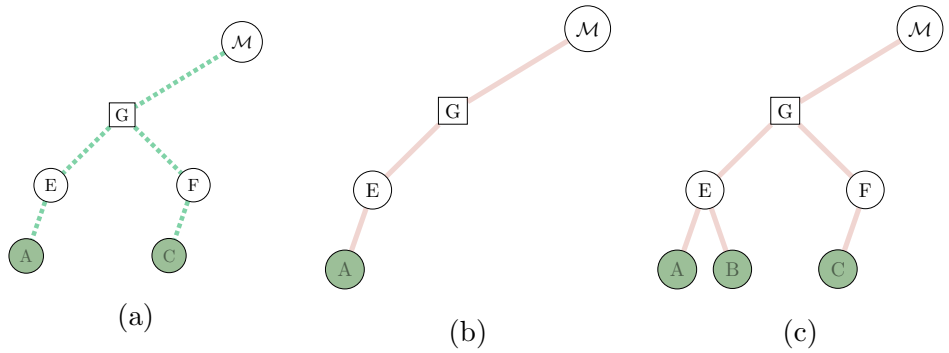


Figure FC4.5: Configuration state tree of statechart shown in Figure FC4.2; (a) A valid configuration state tree for $\mathbb{C} = \{A, C\}$. (b) An invalid configuration tree - shell state G does not have both its regions E and F in the state tree. (c) An invalid configuration tree - composite state E has both its substates in the state tree.

3. Each state in $\mathbb{C}$ is a leaf in st.

4. All ancestors of all states in $\mathbb{C}$ are contained in st.

5. No other state is included as a node in st.

Such a tree that represents valid configurations is called configuration state tree(CST) as shown in Figure FC4.5.

$$st = CST(\mathbb{C}) = \widehat{\mathbb{T}}(n, \mathbb{C}, tr)$$

here, n is the root state of the statechart model.

4.3.3 Transition state tree.

When a transition is fired in a configuration $\mathbb{C}$, the states corresponding to *source side tree* ($ST_s(t, \mathbb{C})$) should be exited and the states corresponding to *destination side tree* ($ST_d(t)$) are entered. It can be viewed as two trees corresponding to the containment hierarchy of the states that they are a part of.

Example 8. *Let us consider the representation of transition t_{GN} on the state tree. It is a combination of source side tree and destination side tree as shown in*

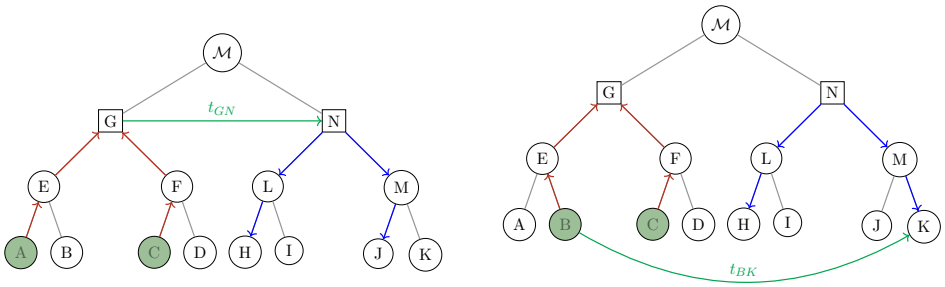


Figure FC4.6: (a)source side tree (ST_s) is marked in red arrows for the transition t_{GN} and destination side tree (ST_d) is marked with blue edges. The arrows in ST_s indicate that the states will be exited, and the arrows in ST_d indicate that the states will be entered when $\mathbb{C} = \{A, C\}$; (b)shows ST_s and ST_d for $\mathbb{C} = \{B, C\}$ and transition t_{BK}

Figure FC4.6. Figure FC4.6 shows a state containment hierarchy. The green line from state G to state N shows an enabled transition t. The current configuration is $\mathbb{C} = \{A, C\}$ and the corresponding states are shown filled with green colour. State $\mathcal{M} = G \sqcup N$. Therefore, state G is the last state to exit on the source side, and state N is the first state to enter on the destination side. The source side tree $ST_s(t, \mathbb{C})$ and the destination side tree $ST_d(t)$ are highlighted in red and blue edges, respectively. □

$ST_s(t, \mathbb{C})$ is the *source side tree*, that is computed based on the transition to be taken and the current configuration $\mathbb{C}$.

$$
\begin{aligned}
&ST_s(t, \mathbb{C}) = \\
&\qquad \textbf{let } l = t.s \sqcup t.d \textbf{ in} \\
&\qquad \textbf{let } s = \\
&\qquad\qquad \textbf{if } t.s \prec_1 l \textbf{ then } t.s \\
&\qquad\qquad \textbf{else } \widehat{\mathbb{T}}(s, \mathbb{C}) \textbf{ where } t.s \prec_* s \prec_1 l
\end{aligned}
$$

Now, we define the *destination side tree* (ST_d) as follows: Consider the list of states L from s to $t.d$. Here, the state s is a ancestor of $t.d$ such that $t.d \prec_* s \prec_1 lub$ where, $lub = t.s \sqcup t.d$. Let this list be $\text{L} = [d_1, d_2, ..., d_n]$ where $d_1 = s$ and $d_n = t.d$.

$$
\begin{aligned}
&ST_d(t) = f(d_1) \\
&\quad where, f(d_i) = \\
&\qquad \textbf{if } i = n \textbf{ then } ST_i(d_i) \\
&\qquad \textbf{if } i \neq n \wedge d_i.\tau = composite \textbf{ then} \\
&\qquad\quad \text{TREE}(d_i, f(d_{i+1})) \\
&\qquad \textbf{if } i \neq n \wedge d_i.\tau = shell \textbf{ then} \\
&\qquad\quad \text{TREE}(d_i, [f'(c_1), f'(c_2), ..., f'(c_m)]) \\
&\qquad\quad \textbf{where,} \\
&\qquad\quad d_i.childnodes = c_1, c_2, ..., c_m \\
&\qquad\quad f'(c) = \textbf{if } c \neq d_{i+1} \textbf{ then } ST_i(c) \\
&\qquad\qquad \textbf{else } f(c)
\end{aligned}
$$

Here, **Initial subtree (ST_i)** for any node n is given by the following:

$$
\begin{aligned}
&ST_i(n) = \\
&\qquad \text{TREE}(n, \{ST_i(n.I)\}) \textbf{ if } n.\tau \in \{Composite, Statechart\} \\
&\qquad \text{TREE}(n, \{\}) \textbf{ if } n.\tau = Atomic \\
&\qquad \text{TREE}(n, \{ST_i(c), \\
&\qquad \forall c | c \in childnodes(n)\}) \textbf{ if } n.\tau = Shell
\end{aligned}
$$

Example 9. *For the transition state tree shown in Figure FC4.6(b), when* $\mathbb{C} =$

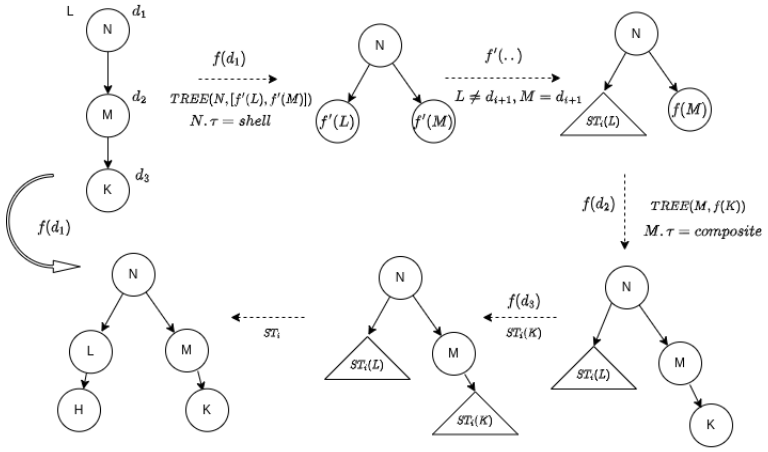


Figure FC4.7: Computation of destination side tree for transition t_{BK} as shown in Figure FC4.6(b)

$\{B, C\}$ and the transition t_{BK} to be taken, two state trees are to be computed. The source side tree (St_s) - a slice of the configuration state tree. The substeps in computing the destination side tree (St_d) as per our definition, is illustrated in Figure FC4.7. The input for the computing function is a list L*, that contains the nodes $[N, M, K]$ and the function $f(d1)$ is a recursive function that computes the destination side tree. Dotted arrows show the intermediate steps and the curved arrow shows the input with the final output on applying the function $f(d_1)$.*

4.3.4 Stages in operational semantics

Now we present the detail of the stages introduced in the beginning of this section. The algorithm that combines these stages to simulate the statechart is shown in algorithm 1. The next subsections will focus on explaining the internals of these stages that are integral to the operational semantics.

Algorithm 1: ConStaBL Simulator

Input: SC: Statechart, $\mathbb{Q}$: Event[]
Output: Configuration ($\mathbb{C}$), Environment (σ)

procedure SIMULATE($SC, \mathbb{Q}$)
▷ Initializing σ - environment, $\mathbb{C}$ - configuration
$\mathbb{C}, \sigma \leftarrow initialize(SC)$
while $e \in \mathbb{Q}$ **do**
▷ stage 1 : Find Transition
$\mathcal{T} \leftarrow$ `findEnabledTransitions(e);`
▷ Conflict Identification
`findNonDeterminism(`$\mathcal{T}$`);`
▷ stage 2 : Compute Code
if $\mathcal{T} \neq \emptyset$ **then**
▷ Here, $\mathcal{C} \rightarrow$ computeCode, ConcurrentCode=$[c_1|c_2|..|c_n]$
$\mathcal{C}(\mathcal{T}, \mathbb{C}) = [\mathcal{C}(t_1, \mathbb{C})|\mathcal{C}(t_2, \mathbb{C}), ...|\mathcal{C}(t_n, \mathbb{C})]$
▷ stage 3 : Execute Code
$\sigma \vdash$ `executeCode`$(\mathcal{C}(\mathcal{T}, \mathbb{C})) \rightarrow \sigma'$
▷ stage 4 : Update Configuration
$\mathbb{C}' = \bigcup_{t \in \mathcal{T}} leaves(ST_d(t))$
$\mathbb{Q} \leftarrow \mathbb{Q} \setminus$ `e`

procedure FINDENABLEDTRANSITIONS(e: Event)
▷ finding transitions that originate from any node of current CST - Configuration State Tree
`return` $\{t | t.s \in CST(\mathbb{C}) \wedge t.e = e \wedge \sigma \vdash t.g \Downarrow true\}$

procedure FINDNONDETERMINISM($\mathcal{T}$: Transition set)
▷ Identifying if ST_s - Source State Tree of two transitions overlap
if $\sigma, \mathbb{C} \vdash \forall t_1, t_2 \in \mathcal{T}, ST_s(t_1, \mathbb{C}) \cap ST_s(t_2, \mathbb{C}) \neq \phi$ **then**
`raise` $conflict(t_1, t_2)$

procedure COMPUTECODE(t:Transition, $\mathbb{C}$: Configuration)
▷ $CFGTree_s$ and $CFGTree_d$ - CFG Tree derived from ST_s and ST_d resp.
$CFGTree_s(t, \mathbb{C}) = treemap(s \rightarrow$ `cfg`$(s.\mathcal{X}), ST_s(t, \mathbb{C}))$
$CFGTree_d(t) = treemap(s \rightarrow$ `cfg`$(\langle init(s.\mathcal{V}_l), s.\mathcal{N}\rangle), ST_d(t))$
▷ $\mathcal{C}_s(t, \mathbb{C})$ and $\mathcal{C}_d(t, \mathbb{C})$ - Code derived from $CFGTree_s$ and $CFGTree_d$ resp.
$\mathcal{C}_s(t, \mathbb{C}) = \mathcal{C}(CFGTree_s(t, \mathbb{C}))$
$\mathcal{C}_d(t, \mathbb{C}) = rev(CFGTree_d(t))$
▷ `cfg`$(t.a)$ - Code derived from $t.action$, `cfg` - Control Flow Graph, SequenceCode = $\langle c_1, c_2, ..c_n\rangle$
$Code(t, \mathbb{C}) = \langle \mathcal{C}_s(t, \mathbb{C})$, `cfg`$(t.a)$, $\mathcal{C}_d(t, \mathbb{C})\rangle$
`return` $Code(t, \mathbb{C})$

4.3.4.1 Stage 1: Find Enabled Transitions

Enabled Transition. In a given configuration $\mathbb{C}$, when an event e has arrived, a transition t is enabled when all three of the following conditions hold:

1. $t.s \in CST(\mathbb{C})$, (i.e., t's source state is one of the vertices of the configuration

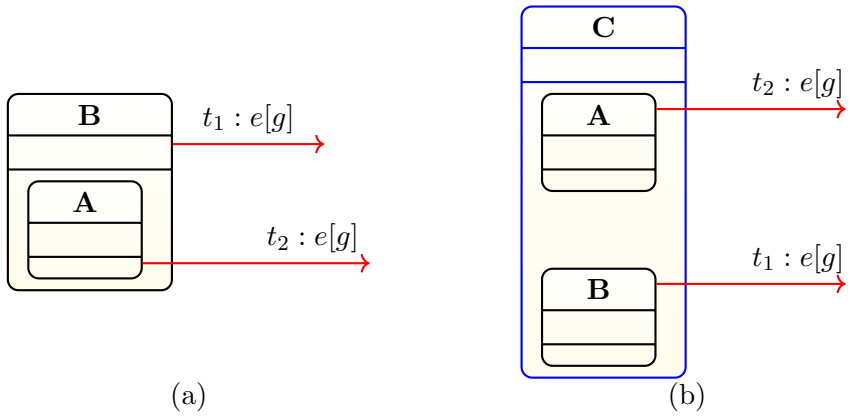


Figure FC4.8: Conflicting Transitions: (a) if $\mathbb{C}$={A}, then both $t_1.s = A$ and $t_2.s = B$ would cause A.$a_{\mathcal{X}}$ and B.$a_{\mathcal{X}}$ to execute; (b) Both t_1 and t_2 would cause C.$a_{\mathcal{X}}$ to execute.

state tree of $\mathbb{C}$).

2. $t.e = e$, (i.e., t's trigger event is the same as the one that has arrived).

3. $\sigma \vdash t.g \Downarrow true$, (i.e., in the given value environment σ, t's guard evaluates to $true$). Here, σ is the *environment* that maintains binding of variables to current values is given by σ.

It is possible that, for a given configuration $\mathbb{C}$ and event e, the number of enabled transitions is zero, one, or more than one. Hence, to perform the next step, we compute the set of enabled transitions $\mathcal{T}$. For example, in the example model in Figure FC4.2, if $\mathbb{C} = \{A, C\}$ and $e = e_1$, then $\mathcal{T} = \{t_{AB}, t_{CD}\}$.

The Source Side Tree helps to identify the transitions that conflicts to identify nondeterminism.

Conflicting transitions. For a given configuration $\mathbb{C}$ and event e, two enabled transitions are said to *conflict* if, fired concurrently, they lead to exiting the same state.

$$\sigma, \mathbb{C} \vdash \forall t_1, t_2 \in \mathcal{T},$$
$$conflict(t_1, t_2) =$$
$$\textbf{if } ST_s(t_1, \mathbb{C}) \cap ST_s(t_2, \mathbb{C}) \neq \phi$$

In the above, we use the set intersection ($\cap$) to indicate the intersection between the trees blocks in $ST_s(t_1, \mathbb{C})$ and $ST_s(t_2, \mathbb{C})$.

Example 10 (Conflicting transitions)**.** *Figure FC4.8 shows two scenarios in which transitions may conflict. Figure FC4.8(a) shows a case in a sequential (non-concurrent) scenario where two enabled transitions t_1 and t_2 emerge from two states that are in an ancestor-descendent relationship. This case, which we call* nondeterminism*, makes it impossible to determine which transition to take, unless we use an external conflict resolution rule (e.g. clockwise arrangement [41], outside-in [45] etc). We consider such conflict resolution undesirable, as it may sneak in undesirable behaviour without the knowledge of the modelling engineer. Instead, such cases should be flagged out wherever possible, and the engineer should be given the choice to resolve in a way as desirable to him/her. Figure FC4.8(b) shows a case of a concurrent model. Two enabled transitions emerge from the two regions A and B of a shell state C. Both transitions would lead to the execution of $C.a_{\mathcal{X}}$, which would be invalid.*

4.3.4.2 Stage 2: Compute code

We will now provide the specifics of how to identify the code to be executed. To compute the *transition code*, its corresponding *Source side tree* and *Destination side tree* are converted to the respective *control flow graph tree*. The transi-

tion code is the code composed from the respective CFGTree and transition code blocks. This section gives the details on composition of code from the identified non-conflicting transitions.

Control flow graph tree. The source side CFG tree for an enabled transition t in a configuration $\mathbb{C}$ is the tree of exit code blocks of the corresponding Source Side Tree $ST_s(t, \mathbb{C})$ of t. Similarly, the destination side CFG tree for an enabled transition t in a configuration $\mathbb{C}$ is the tree of exit code blocks of the corresponding Destination Side Tree $ST_d(t)$. Mathematically:

$$
\begin{aligned}
CFGTree_s(t, \mathbb{C}) = & \\
& treemap(s \rightarrow \\
& \textsc{cfg}(s.a_{\mathcal{X}}), ST_s(t, \mathbb{C})) \\
CFGTree_d(t) = & \\
& treemap(s \rightarrow \\
& \textsc{cfg}(\langle init(s.\mathcal{V}), s.a_{\mathcal{N}} \rangle), ST_d(t))
\end{aligned}
$$

Here, the constructor CFG takes a code block (in the form of its abstract syntax tree) and gives its control flow graph (CFG). Control flow graphs are presented in detail in Section 4.3.4.4 where it will be needed to discuss code execution. Here, it suffices to proceed with an informal understanding of control flow graphs. While computing the source side CFG tree $CFGTree_s(t, \mathbb{C})$, we include the CFGs of the exit code of each state in the Source Side Tree $ST_s(t, \mathbb{C})$. While computing the destination side CFG tree $CFGTree_d(t)$, we include the CFGs of the sequential composition of the variable initialisation code and entry code of each state in the Destination Side Tree $ST_d(t)$.

Code. The code executed during a simulation step is computed from $CFGTree$. The code composition can be either sequential or concurrent and is represented by the following datatype $Code$ as follows:

$$\begin{aligned} code : Code &= Seq([c_1, c_2, ..., c_n]) \text{ where } c_1, c_2, ..., c_n : Code \\ &= Conc(\{c_1, c_2, ..., c_n\}) \text{ where } c_1, c_2, ..., c_n : Code \\ &= CFGCode(cfg) \\ &\text{ where } cfg = \text{CFG}(b) \text{ and } b \text{ a code block} \end{aligned}$$

Here, Seq is an abbreviation of sequential code and $Conc$ is an abbreviation of concurrent code. Henceforth, we abbreviate $Seq([c_1, c_2, ..., c_n])$ as $\langle c_1, c_2, ...c_n \rangle$ and $Conc(\{c_1, c_2, ..., c_n\})$ as $[c_1|c_2|...|c_n]$.

Code reversal function done to a code $c : Code$ is defined as follows:

$$\begin{aligned} rev(c) = & \\ & \textbf{if } c \textbf{ is } CFGCode \textbf{ then } c \\ & \textbf{if } c \textbf{ is } \langle c_1, c_2, ..., c_n \rangle \textbf{then} \\ & \quad \langle rev(c_n), rev(c_{n-1})..., rev(c_2), rev(c_1) \rangle \\ & \textbf{if } c \textbf{ is } [c_1|c_2|...|c_n] \textbf{ then} \\ & \quad [rev(c_1)|rev(c_2)|...|rev(c_{n-1})|rev(c_n)] \end{aligned}$$

Code containment tree.

As is clear, the *code* type is recursive and can represent a hierarchical arrange-

ment of code blocks. We call this hierarchy as code hierarchy and represent the same by a tree called *code containment tree*. Sequence code and concurrent code form the internal nodes of this tree and CFG code form the leaf nodes of this tree. This tree is useful in describing the navigation of the code during code execution. Please note that this is different from the code tree mentioned in Section 4.3.4.2 which mimics the state hierarchy. As shown in the Figure FC4.9(a), for any event in a given configuration, the codes of the non-conflicting enabled transitions are concurrently composed. The transition code is further computed with the help of CFGTree introduced in the beginning of this section.

Transition code.

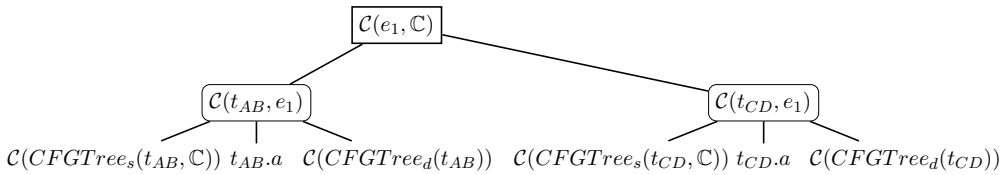
The code to be executed when a transition t gets fired in a configuration $\mathbb{C}$ is given by:

$$\begin{aligned}\mathcal{C}(t,\mathbb{C}) = \langle &\mathcal{C}(CFGTree_s(t,\mathbb{C})),\\ &\textsc{cfg}(t.a)\ , rev(\mathcal{C}(CFGTree_d(t)))\rangle\end{aligned}$$

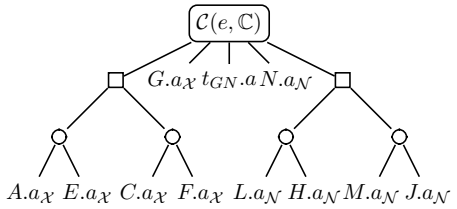
Note the code reversal done to compute code from CFGTree of destination.

The function $\mathcal{C} : CFGTree \rightarrow Code$ takes a CFG tree ct, and gives its code as follows:

$$\begin{aligned}\mathcal{C}(ct) &= CFGCode(s.cfg) \text{ if } ct = \textsc{tree}(s,\{\})\\ &= \langle \mathcal{C}(c)\ , CFGCode(s.cfg)\rangle \text{ if } ct = \textsc{tree}(s,\{c\})\\ &= CFGCode(s.cfg)\ [\mathcal{C}(ct_1)|\mathcal{C}(ct_2)|...|\mathcal{C}(ct_n)]\\ &\qquad \text{if } ct = \textsc{tree}(s,\{c_1,c_2,...,c_n\})\end{aligned}$$



(a)



(b)

Figure FC4.9: Example: Code containment tree of the code in Example 13.

Please note, that the function $\mathcal{C}$ is used wherever there is a conversion from any existing format to code form abstractly. For the sake of simplicity, we have refrained from creating a new function for each of its overloaded parameters. The *cfg* here refers to the control flow graph detailed in section 4.3.4.4.

Example 11 (Code containment tree)**.** *The code shown in Example 13 are pictorially illustrated in the form of code containment trees in Figure FC4.9. We use rectangular nodes with pointed vertices to show a concurrent code, rectangular nodes with rounded corners to show a sequential code and an unbordered node to show CFG codes.* □

Finally, the entire code to be executed during a simulation step is the concurrent composition of the codes of the enabled transitions. For the set of enabled transitions $\mathcal{T} = \{t_1, t_2, ..., t_n\}$, code is given by: $[\mathcal{C}(t_1, \mathbb{C})|\mathcal{C}(t_2, \mathbb{C}), ...|\mathcal{C}(t_n, \mathbb{C})]$

Example 12. *In Figure FC4.5(c), when transition t_{GN} is fired in configuration*

$\{A, C\}$, *the code that gets executed is:*

$$\begin{aligned}\mathcal{C}(t_{GN}) = Seq([&Conc(\{Seq([A.a_{\mathcal{X}}, E.a_{\mathcal{X}}]),\\ &Seq([C.a_{\mathcal{X}}, F.a_{\mathcal{X}}])\}), G.a_{\mathcal{X}},\\ &t_{GN}.a,\\ &N.a_{\mathcal{N}}, Conc(\{Seq([L.a_{\mathcal{N}}, H.a_{\mathcal{N}}]),\\ &Seq([M.a_{\mathcal{N}}, J.a_{\mathcal{N}}])\})])\end{aligned}$$

In abbreviated form, the above is written as:

$$\begin{aligned}\mathcal{C}(t_{GN}) = \langle[\langle A.a_{\mathcal{X}}, E.a_{\mathcal{X}}\rangle|\langle C.a_{\mathcal{X}}, F.a_{\mathcal{X}}\rangle], G.a_{\mathcal{X}},\\ t_{GN}.a,\\ N.a_{\mathcal{N}}, [\langle L.a_{\mathcal{N}}, H.a_{\mathcal{N}}\rangle|\langle M.a_{\mathcal{N}}, J.a_{\mathcal{N}}\rangle]\rangle\end{aligned}$$

□

4.3.4.3 Stage 3: code execution

After the transition code $\mathcal{C}(\mathcal{T}, \mathbb{C})$ is computed, the simulator will proceed to execute it. In this subsection, we present the details of this process.

Each step in code execution involves the interpretation of single statements in the code. The code can be concurrent. Unlike many existing implementations of Statecharts [41] [42] [26] [46] [47] [48], we allow arbitrary interleaving between code executed concurrently. We assume individual instructions to execute atomically. Though finer grained interleaving is possible in principle, we consider that not to be necessary feature at the statechart modelling level.

Example 13 (Transition Code)**.** *Consider the model shown in Figure FC4.2. Source configuration* $\mathbb{C} = \{A, C\}$*. For this example, assume that the code blocks in the model are as follows:*

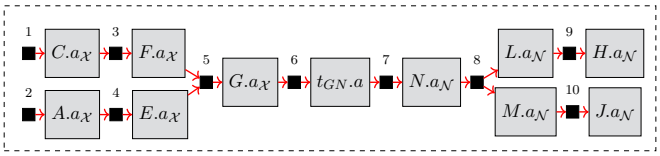
- ***Entry codes:*** $A.a_{\mathcal{N}} = \{i_{A\mathcal{N}_1}; i_{A\mathcal{N}_2};\}$, $B.a_{\mathcal{N}} = \{i_{B\mathcal{N}_1}; i_{B\mathcal{N}_2};\}$ *etc.*
- ***Exit codes:*** $A.a_{\mathcal{X}} = \{i_{A\mathcal{X}_1}; i_{A\mathcal{X}_2};\}$, $B.a_{\mathcal{X}} = \{i_{B\mathcal{X}_1}; i_{B\mathcal{X}_2};\}$ *etc.*
- ***Transition action codes:*** $t_{AB} = \{i_{AB_1}; i_{AB_2};\}$, $t_{CD} = \{i_{CD_1}; i_{CD_2};\}$ *etc.*

1. ***Case 1 (Concurrent and disjoint).*** *Suppose, the input event is* e_1*. The code to be executed in this case is*
   $[\mathcal{C}(t_{AB}, \mathbb{C})|\mathcal{C}(t_{CD}, \mathbb{C})] =$
   $[\langle A.a_{\mathcal{X}},\ t_{AB}.a,\ B.a_{\mathcal{N}}\rangle|\langle C.a_{\mathcal{X}},\ t_{CD}.a,\ D.a_{\mathcal{N}}\rangle]$*. A possible trace generated when the above code executes:* $i_{A\mathcal{X}_1}$, $i_{C\mathcal{X}_1}$, $i_{A\mathcal{X}_2}$, $i_{C\mathcal{X}_2}$, i_{AB_1}, i_{CD_1}, i_{AB_2}, i_{CD_2}, $i_{B\mathcal{N}_1}$, $i_{D\mathcal{N}_1}$, $i_{B\mathcal{N}_2}$, $i_{D\mathcal{N}_2}$.
2. ***Case 2 (Concurrent, joining and forking).*** *Suppose, the input event is* e*. The code to be executed is* $\mathcal{C}(t_{GN}, \mathbb{C}) = \langle[\langle A.a_{\mathcal{X}}, E.a_{\mathcal{X}}\rangle|\langle C.a_{\mathcal{X}}, F.a_{\mathcal{X}}\rangle], G.a_{\mathcal{X}}, t_{GN}.a, N.a_{\mathcal{N}}, [\langle L.a_{\mathcal{N}}, H.a_{\mathcal{N}}\rangle|\langle M.a_{\mathcal{N}}, J.a_{\mathcal{N}}\rangle]\rangle$*. A possible trace generated when the above code executes:* $i_{A\mathcal{X}_1}$, $i_{C\mathcal{X}_1}$, $i_{A\mathcal{X}_2}$, $i_{C\mathcal{X}_2}$, $i_{E\mathcal{X}_1}$, $i_{F\mathcal{X}_1}$, $i_{E\mathcal{X}_2}$, $i_{F\mathcal{X}_2}$, $i_{G\mathcal{X}_1}$, $i_{G\mathcal{X}_2}$, i_{GN_1}, i_{GN_2}, $i_{N\mathcal{N}_1}$, $i_{N\mathcal{N}_2}$, $i_{L\mathcal{N}_1}$, $i_{M\mathcal{N}_1}$, $i_{L\mathcal{N}_2}$, $i_{M\mathcal{N}_2}$, $i_{H\mathcal{N}_1}$, $i_{J\mathcal{N}_1}$, $i_{H\mathcal{N}_2}$, $i_{J\mathcal{N}_2}$.

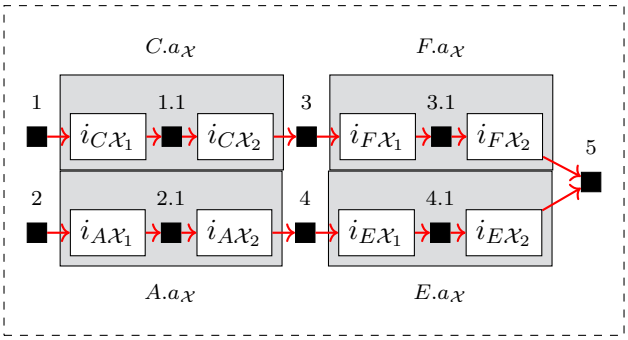
□

4.3.4.4 Control flow graph (CFG).

A control flow graph $G(V, E, \mathcal{N}, \mathcal{X})$ is a graph with a set of vertices/nodes V and (control flow) edges E. $V = V_I \cup V_D$. Here,



(a)



(b)

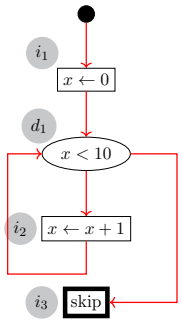
Figure FC4.10: Example: code and control points

- $\mathcal{N}$ is a unique node, called the entry node of the CFG g, referred to as $g.\mathcal{N}$.
- $\mathcal{X}$ is unique node $\mathcal{X}$, called the exit node of the CFG g, referred to as $g.\mathcal{X}$. The exit node has no successors.
- V_I is the set of instruction nodes, i.e., nodes with an instruction in them with a possible side-effect in the action language. An instruction node $v_I \in V_I$ has zero or one successor (referred to as $v_I.s$).
- V_D is the set of decision nodes. Each decision node $v_D \in V_D$ has a boolean expression in the action language called the condition, referred to as $v_D.c$. A decision node has two successors, namely the *then* successor (referred to as $v_D.t$) and the *else* successor (referred to as $v_D.e$).

Additionally:

- A CFG may have a single node, in which case, $\mathcal{N} = \mathcal{X}$.
- $\mathcal{N} \in V$. $V_{\mathcal{X}} \in V_I$.

```
1  x = 0;
2  while(x < 10)
3    x++;
```



$g.a_{\mathcal{N}} = i_1$, $g.a_{\mathcal{X}} = i_3$, $g.V_I = \{i_1, i_3\}$, $g.V_d = \{d_1\}$, $i_1.s = d_1$, $d_1.t = i_2$, $d_1.e = i_3$

Figure FC4.11: Control flow graph

An example of control flow graph is shown in Figure FC4.11.

Control points. To compute the sequence in which the code blocks get executed, we can think of a partial order (or the corresponding directed acyclic graph) of code blocks such that code blocks in the same sequence are in that order. The nodes of the code sequence DAG correspond to the leaf nodes of the code containment tree. The ordering relations between various nodes of this DAG can be directly derived from the structure of the code containment tree.

Example 14 (Code partial order)**.** *The model in Figure FC4.2(a) in configuration $\mathbb{C} = \{A, C\}$ with input event e causes t_{GN} to fire. The code partial order corresponding to this simulation step is shown in Figure FC4.10(a). As can be seen, the nodes of this DAG correspond to the leaves of the code containment tree shown in Figure FC4.9(b).*

□

As we have pointed out, instructions in concurrently composed pieces of code can interleave. Hence, we work with the idea of *control points* – points in the executing code which are visible to the simulator and are subject to interleaving in case of concurrent composition.

Example 15 (Control Points)**.** *A portion of the code partial order shown in Figure FC4.10(a) is shown in Figure FC4.10(b). The control points here are: 1, 1.1, 3,*

3.1, 2, 2.1, 4, 4.1, 5 and so on. Here, control points like 1, 2, 3, 4, 5 etc. are control points outside individual control flow graphs. However, control points like 1.1, 2.1, 3.1, 4.1 etc. are control points inside individual control flow graphs.

In Figure FC4.10(b), the two sequences – 1, 1.1, 3, 3.1 and 2, 2.1, 4, 4.1 – are concurrently composed and execute in two concurrent threads. As is clear from the figure, 1.1 is reached strictly after reaching 1; 3 after 1.1 etc. However, Any of 1.1 or 2.1 may be reached earlier depending on which of the two concurrent threads progresses first. At a point when the execution progresses beyond 3.1 and 4.1, the two threads join and give place to a single thread which is at control point 5. □

During code execution, there can be multiple active threads. The simulator keeps track of its progress through each running thread by maintaining a set of control points CP. Every code simulation step will cause the execution of one of the instructions immediately after one of the control points $cp \in CP$. Choice of cp is done randomly from CP. After this, cp gets removed from CP and gets replaced by zero, one or more control points, as the case may be, which succeed cp in the code partial order.

The function $nextCP$ computes the set of control points that replace the currently processed control point. An internal control point cp will be replaced by one of its successors in $cp.cfg$, the CFG it belongs to. However, if cp is an exit node of $cp.cfg$, then the next of control points to replace it in CP are the entry node of other CFGs. Such CFGs (and their unique entry node) can be identified using the functions $nextCFGCodes$ and $first$. Please note that these are computed through an implicit traversal of code containment tree, discussed in Section 4.3.4.2.

When a member $cp' \in nextCP(cp)$ happens to be a join point, (i.e., having mul-

$$
\begin{aligned}
parent(n) = n' \ &\textbf{if } n' = \text{Seq}([...n...]) \textbf{ or } \text{Conc}(\{...n...\}) \\
&nil \textbf{ otherwise.} \\
first(n) = \ &\textbf{if } \ n \textbf{ is a } CFGCode \textbf{ then } \{n.a_{\mathcal{N}}\} \\
&\textbf{if } n = \text{Seq}([c_1, c_2, ...c_n]) \textbf{ then } first(c_1) \\
&\textbf{if } n = Conc(\{c_1, c_2, ..., c_n\}) \textbf{ then } \bigcup_{c_i \in \{c_1, c_2, ..., c_n\}} first(c_i) \\
nextCFGCodes(n) = \ &\textbf{if } parent(n) = nil \textbf{ then } \{\} \\
&\textbf{if } parent(n) = Seq([c_1, c_2, ...c_m]) \wedge n = c_i \textbf{ then} \\
&\quad \textbf{if } c_i \neq c_m \textbf{ then } first(c_{i+1}) \\
&\quad \textbf{else } nextCFGCodes(parent(n)) \\
&\textbf{if } parent(n) = Conc(\{c_1, c_2, ..., c_m\}) \textbf{ then} \\
&\quad nextCFGCodes(parent(n)) \\
nextCP(n) = \ &\textbf{if } (n = n.cfg.a_{\mathcal{X}}) \textbf{ then } nextCFGCodes(n) \\
&\textbf{else } succ(n, n.cfg)
\end{aligned}
$$

Figure FC4.12: Functions - Operational semantics of code simulation

tiple predecessor control points), the situation needs to be dealt with separately. This is a point where multiple threads will collapse into one at an appropriate time. This is when all predecessors of cp' have been processed, thus bringing the control points of all these thread to cp'. This concludes their execution. Hence, when the first of these threads reaches cp', we start keeping track of the number of threads waiting to join at cp', and cp' is kept waiting. When, all these threads reach cp', we insert cp' into CP scheduling it for execution in the next code simulation step.

Note that due to the complex hierarchy in which the code blocks in a statechart may be arranged at runtime, it may appear that keeping track of them would be very complex. However, with the above approach, we are able to essentially collapse the control front (the control points of all the active threads) into a flat set.

Example 16 (Code Simulation Trace)**.** *Consider the code partial order shown in Figure FC4.10(b). We track the successive values of the current control point set*

CP and the join points JP in a typical code simulation.

$$\begin{aligned}(CP, JP) = &(\{\}, \{\}), (\{1, 2\}, \{\}), (\{1, 2.1\}, \{\}),\\ &(\{1, 4\}, \{\}), (\{1.1, 4\}, \{\})\\ &(\{3, 4\}, \{\}), (\{3.1, 4\}, \{\}),\\ &(\{4\}, \{5 \mapsto \{4.1\}\}),\\ &(\{4.1\}, \{5 \mapsto \{4.1\}\}),\\ &(\{5\}, \{\}), ...(\{7.1\}, \{\}), (\{8\}, \{\}),\\ &(\{8.1, 8.2\}, \{\}), ...(\{9.1, 10.1\}, \{\}),\\ &(\{9.1\}, \{\}), (\{\}, \{\})\end{aligned}$$

Following points in the above trace are noteworthy:

- *The set CP and map JP, both start off empty and end up empty over a simulation step.*
- *The join point* 5 *is reached when the control front passes control point* 3.1*. At this point, an entry is added to JP with* 5 *as key and* $\{4.1\}$ *as value, as* 4.1 *is the previous control point to* 5 *in the currently active threads (only one in this case, as the thread corresponding to* 3.1 *has already finished execution) which will eventually join at* 5*.*
- *As the control front passes* 4.1*, it gets removed from the values of key* 5 *in JP. This makes the value set of* 5 *empty in JP. Therefore,* 5 *is removed from JP, and added to CP.*
- *Control point* 8 *is fork point.* 8.1 *and* 8.2 *are the internal control points of $L.a_{\mathcal{N}}$ and $M.a_{\mathcal{N}}$ respectively. As soon as the control front moves past* 8*, it is replaced by* 8.1 *and* 8.2 *in CP.*

- *Code simulation for this simulation step concludes when CP becomes empty.*

Operational semantics – code simulation. During the code simulation of a simulation step, the code data structure `C` that gets executed remains constant. In this part of the discussion, we will omit mentioning it in most places. The first step of code simulation involves populating the control front CP with appropriate set of CFG Nodes in the code. These are nothing but the initial entry nodes of the initial CFGCodes of the code. In Figure FC4.13, we show this in rule CODE-SIM-INIT. Rule CODE-SIM-INTERNAL presents the operational semantics when the currently processed CFG node n is not an exit node of its CFG. Its successor node in the CFG n' is computed as $nextCFGNode(n, \sigma)$. This is a direct successor if n corresponds to an instruction node. However, if n is a decision node, n' depends on valuation of the condition expression $n.c$ of n. If $n.c$ evaluates to true in the current value environment σ, then n' is n's then-successor; otherwise, it is the else-successor of n.

The rule CODE-SIM-EXIT-NODE handles the case when the current CFG node n chosen randomly from the control front CP (using the function *any*) is the exit node of its CFG. As discussed above, this step could lead to joining (i.e., two threads collapsing into one) or forking (one thread giving place to multiple threads) of code execution. This could also lead us to the completion of code simulation when CP becomes empty again. In this step, all successors of n which are not join points (shown by set N), directly get added to CP. The other set of successors are join points, shown by set $J = J_i \cup J_c \cup J_o$. J_i is the set of new join points, i.e., for each member of J_i, n is the first of the predecessors to have got processed. In this step, all members of J_i get added to JP. J_c is the set of those join point successors, one of whose predecessors have already been processed in an earlier code simulation step, and hence, they had previously been added to JP. Even after the processing of n, there are other predecessors of the members

CODE-SIM-INIT

$$\frac{}{\sigma, \{\}, \{\} \longrightarrow \sigma', first(\mathtt{C}), \{\}}$$

CODE-SIM-INTERNAL

$$\frac{n = any(CP) \quad n' = nextCFGNode(n, \sigma) \quad \sigma \vdash n.inst \Downarrow \sigma' \quad CP' = CP \setminus \{n\} \cup \{n'\}}{\sigma, CP, \{\} \longrightarrow \sigma', CP', \{\}}$$

CODE-SIM-EXIT-NODE

$$\frac{\begin{array}{c} n = any(CP) \quad n = n.cfg.a_{\mathcal{N}} \quad \sigma \vdash n.inst \Downarrow \sigma' \\ next(n) = N \cup J_i \cup J_c \cup J_o \quad N = \{n' | pred(n') = \{n\} \wedge n' \in CP'\} \\ J_i = \{j |\ |pred(j)| > 1, j \notin JP \wedge JP'(j) = pred(j) \setminus \{n\}\} \\ J_c = \{j \mid |pred(j)| > 1 \wedge |JP(j)| > 1 \wedge JP'(j) = pred(j) \setminus \{n\}\} \\ J_o = \{j |\ JP(j) = \{n\}\} \quad CP' = CP \setminus \{n\} \cup N \cup J_o \quad JP' = JP \cup J_i \setminus J_o \end{array}}{\sigma, CP, JP \longrightarrow \sigma', CP', JP'}$$

Figure FC4.13: Operational semantics of code simulation

of J_c which await processing; hence all members of J_c continue to be part of JP. J_o are those join point successors, all of whose predecessors have completed their processing in this step, n being the last of them. Hence, all members of J_o are removed from JC and inserted to the control front CP.

Note that CODE-SIM-EXIT-NODE subsumes the case when the control front CP empties out, which means that the code execution halts.

Example 17 (Operational semantics – Code simulation)**.** *In Table TC4.1, we present an example of the trace shown in Example 16 to illustrate the role of sets J_i, J_c and J_o.*

□

4.3.4.5 Stage 4: update configuration

The new configuration $\mathbb{C}'$ is given by the following function:

Table TC4.1: Details of code simulation trace tracking the sets J_i, J_c and J_o as per the operational semantics.

CP	JP	n	N	J_i	J_c	J_o	CP'	JP'
{}	{}	-	{}	{}	{}	{}	{}	{}
$\{1, 2\}$	{}	2	$\{2.1\}$	{}	{}	{}	$\{1, 2.1\}$	{}
$\{1, 2.1\}$	{}	2.1	$\{4\}$	{}	{}	{}	$\{1, 4\}$	{}
$\{1, 4\}$	{}	1	$\{1.1\}$	{}	{}	{}	$\{1.1, 4\}$	{}
$\{1.1, 4\}$	{}	1.1	$\{3\}$	{}	{}	{}	$\{3, 4\}$	{}
$\{3, 4\}$	{}	3	$\{3.1\}$	{}	{}	{}	$\{3.1, 4\}$	{}
$\{3.1, 4\}$	{}	3.1	{}	$\{5\}$	{}	{}	$\{4\}$	$\{5 \mapsto \{4.1\}\}$
$\{4\}$	$\{5 \mapsto \{4.1\}\}$	4	$\{4.1\}$	{}	$\{5\}$	{}	$\{4.1\}$	$\{5 \mapsto \{4.1\}\}$
$\{4.1\}$	$\{5 \mapsto \{4.1\}\}$	4.1	{}	{}	{}	$\{5\}$	$\{5\}$	{}
$\{5\}$	...		...	...	...	...	...	...

$$\mathbb{C}' = \mathbb{C} \textbf{ if } \mathcal{T} = \{\}$$
$$\bigcup_{t \in \mathcal{T}} leaves(ST_d(t)) \textbf{ otherwise.}$$

If there are no enabled transitions, the configuration does not change. The event gets lost. If there are one or more enabled transitions, the new configuration is the union of the set of leaf nodes of the Destination Side Tree of each of these transitions.

We have implemented a simulator according to the operational semantics presented in section 4.3. Figure FC4.1 shows an overview of the integral units of the simulator. The *simulator* operates in two modes: *auto* and *interactive*. In *auto* mode the event queue is populated prior to the beginning of execution. In interactive mode, events are taken as input during execution. It works on the *AST*, and the simulation (procedure SIMULATE) consumes a sequence of events and proceeds with execution as shown in algorithm 1. The simulator maintains the current execution state, configuration, and environment at each execution point. Our simulator allows users to simulate the models written in *ConStaBL*

and is available in our GitHub repository[2].

4.4 Summary

This chapter examined concurrent statecharts' structural and operational semantics, particularly action language interleaving, to offer a new perspective. We can also simulate and manage nested concurrent system states with this way. This gives concurrent statechart language designers a new viewpoint on action languages. Understanding the action language-statechart connection helps language designers optimise and refine the language for nested concurrent states and do upstream analysis of interleaved execution behaviour. Our contribution was the development of a semantics and simulator that allow for the interleaved execution of action code, enabling the detection of concurrency-related issues at an early stage.

[2]Repo link: https://github.com/sujitkc/statechart-verification/tree/skc-simulator-test

CHAPTER 5

FUZZING STATECHARTS

5.1 Introduction

Fuzzing is a technique used to test programmes and applications by generating random inputs. Over the years, various methodologies and tools for fuzzing have evolved [49]. These methodologies and tools have become more sophisticated and effective in identifying vulnerabilities and weaknesses in software systems. Fuzzing has proven to be an invaluable technique for uncovering unexpected bugs, crashes, and security flaws that may not be easily detected through traditional testing methods [50, 51].

1. It aids in evaluating the robustness of a system by subjecting it to a diverse array of inputs, including unusual data, edge cases, and unexpected sequences. This verifies the system's ability to handle such scenarios effectively, without encountering crashes or displaying erroneous behaviour.

2. It enables rigorous testing and verification by systematically exploring an extensive number of paths (in statecharts, the paths can be practically infinite).

3. It can be automated, reducing the manual effort required in testing various

combinations of interleavings in a concurrent code. It facilitates comprehensive coverage which significantly enhances the detection of challenging concurrency-related bugs that might otherwise remain elusive.

4. The counterexample traces obtained during fuzz testing can be used to easily replicate and analyse the identified issues, further simplifying the debugging process.

We used fuzzing techniques for the statechart models developed in StaBL/ConStaBL that is part of ConStaBL statechart models[1].

5.2 Fuzzing statecharts

Statecharts are widely used in various domains [52], including safety-critical systems like avionics and automotive control systems. These systems often incorporate subsystems for navigation, obstacle detection, and vehicle communication, resulting in complex interactions and numerous possible execution scenarios. Manual simulation of such systems becomes challenging due to the sheer number of valuations and execution traces. To address this challenge, we have harnessed the advantages of fuzz testing specifically for statecharts. Fuzz testing involves generating a large volume of inputs and observing how the system responds, aiming to uncover software bugs and anomalies. Integrating fuzz testing with statecharts is a novel concept that has not been explored in existing literature.

In this study, we employ fuzzing for the following purposes:

[1]The content of this chapter is based on: Venkatesan, Karthika, and Sujit Kumar Chakrabarti. "ConStaBL–A Fresh Look at Software Engineering with State Machines." arXiv preprint arXiv:2307.03790 (2023), doi:https://doi.org/10.48550/arXiv.2307.03790 and "A fresh look on semantics of Concurrent State Based Language (ConStaBL)", Journal of Computer Languages, vol. 79, p. 101270, 2024. doi:https://doi.org/10.1016/j.cola.2024.101270

1. Simulating large models with thousands of events, a task that is cumbersome when performed manually. This verifies the capacity and capability of our simulator to handle extensive models and events.

2. Confirming the simulator's functionality with respect to as many potential transition combinations as possible, in accordance with the structural semantics.

3. Assessing the presence of both desired and undesired properties within the model.

5.2.1 Fuzzing techniques

Fuzzing comes in various flavours, each with its strengths. By input generation, fuzzing can be mutation-based (modifying existing inputs), generative (creating new ones from scratch), or template-based (using a format to fill with random data). Targeting can be black-box (no internal knowledge), grey-box (some knowledge like file formats), or white-box (full source code access). Coverage-guided fuzzing prioritizes exploring new code paths, while protocol and file format fuzzing targets specific communication or data structures. Mutation-based and coverage-guided fuzzing are generally the most effective for discovering the bugs in the code. It starts with valid inputs and modifies them to create unexpected scenarios, often uncovering logic errors and edge cases.

5.2.1.1 Jazzer

Several fuzzing engines are available for Java, including Jazzer, JCrasher, JavaFuzz, Java Quick Check, and KFuzzy. Among these tools, Jazzer and JCrasher excel at bug detection and provide counter-example traces when crashes occur.

However, Jazzer stood out due to its larger support community and integration with Google OSS-Fuzz. Since our goal was to identify bugs through fuzzing and obtain crash-causing input traces, Jazzer was suitable.

Google's adoption of Jazzer likely stemmed from its strengths in fuzzing Java programs. First, Jazzer's coverage-guided approach prioritizes exploring new code areas, leading to efficient bug discovery. Additionally, being built for the JVM, it enables deep inspection of Java programs, uncovering subtle bugs. These features, combined with Jazzer's heritage in libFuzzer and mutation capabilities, make it a powerful tool for Google to ensure the stability of their Java applications. If a crash occurs during fuzzing, Jazzer can record the specific input that caused it. This recorded input, called a counter-example trace, is invaluable for developers in pinpointing the location of the bug and understanding how to fix it.

The inputs are encoded in a serialized format and a reproducer file named $CrashFile.java$ is created as shown in Figure FC5.2. This file calls the $sampleMethod$ (in our case $simulate$ method from the $Simulator$ class) using the serialized object as input. By running the program again with input from the crash file, the issue can be reproduced. Please see that the $crashFilePath$ points to the serialised input.

5.2.2 Jazzer Integration

The operation of Jazzer is illustrated in Figure FC5.3. When a *property under test* is detected during simulation, we call $throw\ new\ FuzzerIssue()$ command to halt the fuzzer and record all the inputs generated by the fuzzer up to that point. Note that, we call $throw\ new\ FuzzerIssue()$ on the detection of both desired and undesired properties. We have integrated Jazzer [53], the java-based fuzzer to our simulator.

```
// ProgramUnderTest.java
public class ProgramUnderTest {
    public static void main(String[] args) {
        if (args.length > 0) {
            String input = args[0];
            sampleMethod(input);
        }
    }

    public static void sampleMethod(String input) {
        if (input.length() > 0 && input.charAt(0) == 'C') {
            throw new FuzzerIssue("Crash triggered!");
        }
    }
}
// FuzzTarget.java
import com.code_intelligence.jazzer.api.FuzzedDataProvider;
import com.code_intelligence.jazzer.junit.FuzzTest;

public class FuzzTarget {
    @FuzzTest
    void fuzzTest(FuzzedDataProvider data) {
        String input = data.consumeString(100);
        ProgramUnderTest.sampleMethod(input);
    }
}
```

Figure FC5.1: Sample program with fuzzer issue

```

public class CrashFile {
    public static void main(String[] args) throws Exception {
        // Load the crash file content
        String crashFilePath = "path/to/crash-12345";
        String crashFileContent = new String(Files.readAllBytes(
            ↪ Paths.get(crashFilePath)));

        // Parse the JSON content
        JSONObject crashData = new JSONObject(crashFileContent);
        String input = crashData.getString("input");

        // Reproduce the crash
        ProgramUnderTest.sampleMethod(input);
    }
}
```

Figure FC5.2: Reproducing the crash

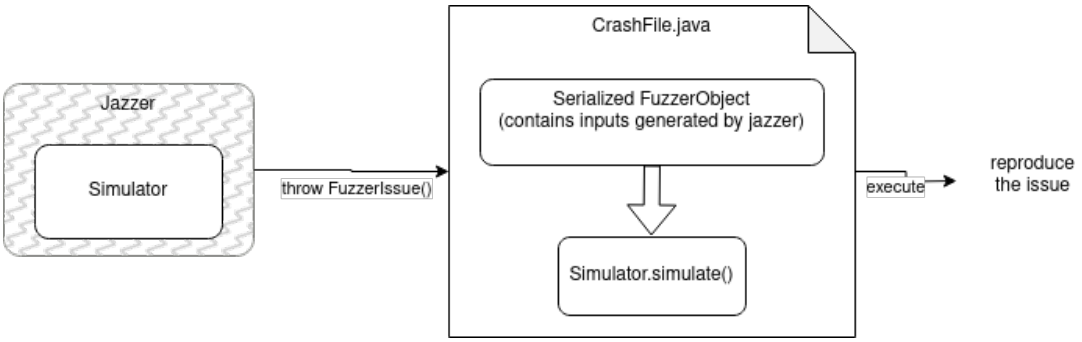


Figure FC5.3: Fuzzing statechart model with jazzer

5.2.3 Testing properties of statechart models

When employing fuzz testing on statecharts, we can uncover several *properties* in the statechart at runtime, such as:

1. **Conflicting transitions** - Although ConStaBL semantics does not employ priorities for transitions, it is still valuable for designers to be aware of the existence of conflicts (as explained in detail in section 4.3 - *conflicting transitions*). Detecting conflicts is a run-time phenomenon, as it depends on the run-time valuation of variables in the *guard* of the enabled transitions.

2. **Configuration reachability** - We intend to check if desired and undesired configurations are reachable. Undesired configurations can occur in concurrent statecharts when multiple regions enter a combination of states that, while individually valid, lead to undesired behaviour. For example, while it is acceptable for two traffic signals to be independently green, if they are at the same junction displaying opposite signals, it can cause accidents. Therefore, it is crucial to examine the orthogonal composition of traffic lights and determine if there are any scenarios where the configuration becomes $\{green, green\}$. When we test for the *reachability* as discussed in next section, a configuration can be desired as well.

We augment the algorithm 1 to throw the fuzzer issue as shown below:

$$\text{conflict}(t_1, t_2) \implies$$

throw new

FuzzerIssue("Conflicting transitions detected");

$$\text{if } \mathbb{C} = \langle c \rangle \text{ then } \implies$$

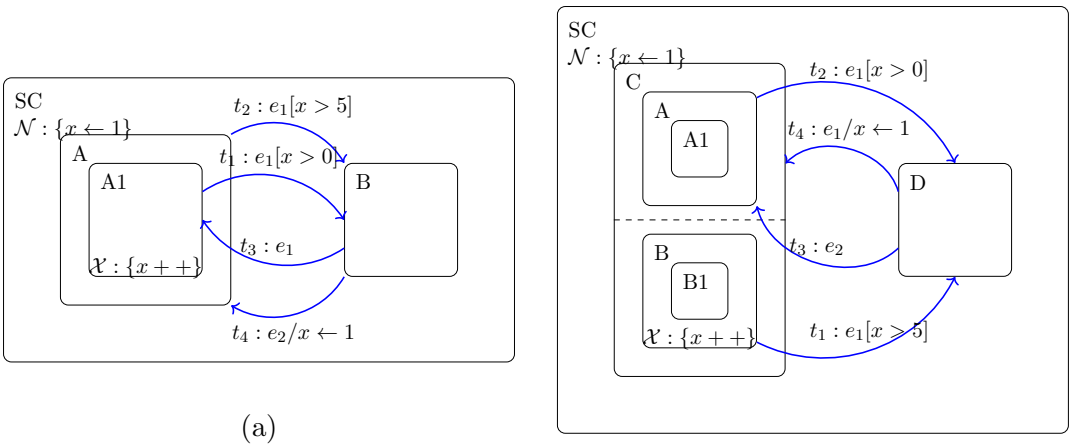
throw new

FuzzerIssue("Configuration detected");

Here $\langle c \rangle$, is a configuration (desired or undesired).

Example 18. *Consider the model in the Figure FC5.4(a) and (b), that are an extension of Figure FC4.8. It has two transitions* $t1$ *and* $t2$ *that can cause conflict as the value of* x *becomes greater than 5, however, it also contains transition* $t4$ *which resets the value of* $x \leftarrow 1$*. Identifying if there is a valuation of* x *that may cause both* $t1$ *and* $t2$ *to be enabled in large models is difficult. Even in this model if the event sequence is* $e1, e2, e1, e2, e1, e2, e1, ...$*, an alternating pattern between* $e1$ *and* $e2$ *or if* x *is carefully reset at appropriate action blocks, it may never result in conflict. Even when such issues are identified, reproducing the same in large models is difficult.*

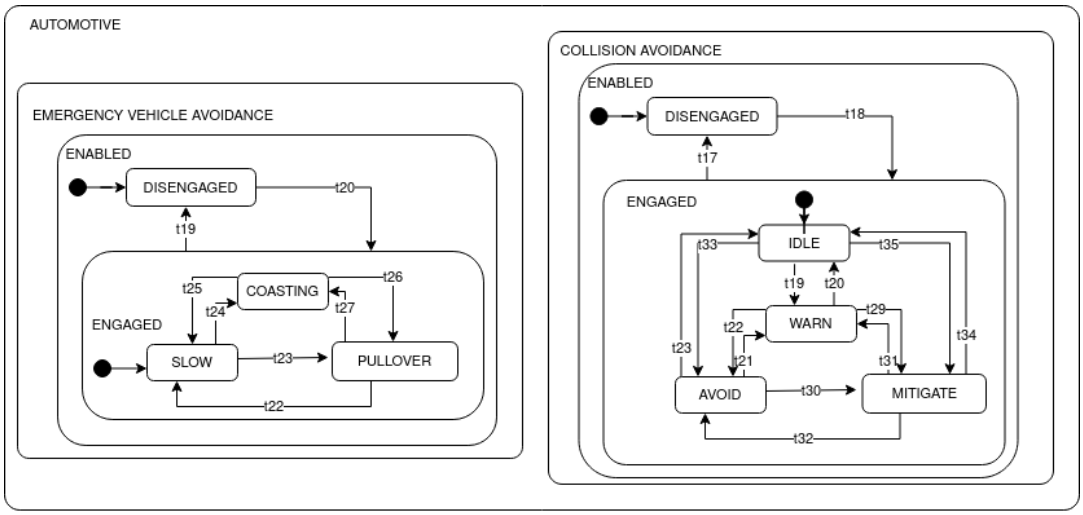
Example 19 (Undesired Configuration)**.** *Consider the model in Figure FC5.5 that depicts two concurrent subsystems in an automotive scenario: Emergency Vehicle Avoidance (EVA) and Collision Avoidance (CA). These systems process inputs from sensors and radars and control actuators such as throttle, steering,*



(a) (b)

Figure FC5.4: Conflicting Transitions: (a) if $\mathbb{C}$={A1}, then both $t_1.s = A$ and $t_2.s = B$ would conflict when $x > 5$; (b) if $\mathbb{C}$={A1,B1}, then both t_1 and t_2 would conflict when $x > 5$.

and brakes. For the purpose of this work, we have presented a simplified model in Figure FC5.5 extracted from the automotive dataset by Juarez et al. (2007) [54]. We have identified conflicts in which the two subsystems issue conflicting actions, such as simultaneous modifications to the speed of the vehicle (known as same-actuator conflicts). Additionally, these subsystems can result in undesired configurations, such as $\{Slow, Mitigate\}$*, where EVA instructs the system to slow down, and CA instructs the system to mitigate the risk of a collision by halting the vehicle. However, halting the vehicle may impede the movement of the emergency vehicle, making this an undesired configuration. As speed may range from* 0 *to* 100*, manual testing of all scenarios is a tedious task. To replicate the actual behaviour, we employ fuzzing of inputs, specifically varying the value of speed. During simulation, if the current configuration is* $\{Slow, Mitigate\}$*, the command throw new* $FuzzerIssue()$ *is triggered, to capturing the fuzzed data that caused to reach the configuration. While these models initially did not contain any inherent errors, we injected conflict into a few states by setting the guards of the outgoing transitions to true or causing the guards of one or more transitions to evaluate as true. For instance, in Figure FC5.5, the guard of EVA_t22 is changed from*



EVA_t19: [!siren||speed==0]/EVA_HVI=1;
EVA_t20: [siren && speed>0]/EVA_HVI=2;
EVA_t22:
[WayClear&&!DontStop]/set_Brake=60;
set_SteerOut=-1
EVA_t23:[!WayClear&&!DontStop]/set_Brake=30
EVA_t24: [DontStop]/set_Throttle=35;
EVA_t25:[!WayClear&&!DontStop]/set_Brake=30
EVA_t27:[DontStop]/set_Throttle=35;
EVA_t26:
[WayClear&&!DontStop]/set_Brake=60;
set_SteerOut=-1

CA_t17: [(Speed>0 && Speed<=25)]/CA_HVI=1;Warning=0
CA_t18: [(Speed>25]/CA_HVI=2;
CA_t19: [ThreatCA==1]/warning=1;
CA_t20: [ThreatCA==0]/warning=0;
CA_t21: [ThreatCA==1]/warning=1;
CA_t22: [ThreatCA==2]/warning=2;set_Brake=30;
CA_t23: [ThreatCA==0]/warning=0;
CA_t29: [ThreatCA==3]/warning=3;set_Brake=80;
CA_t30: [ThreatCA==3]/warning=3;set_Brake=80;
CA_t31: [ThreatCA==1]/warning=1;
CA_t32: [ThreatCA==2]/warning=2;set_Brake=30;
CA_t33: [ThreatCA==2]/warning=2;set_Brake=30;
CA_t34: [ThreatCA==0]/warning=0;
CA_t35: [ThreatCA==3]/warning=3;set_Brake=80;

Figure FC5.5: Collision Avoidance and Emergency Vehicle Avoidance subsystems $[wayclear \wedge !DontStop]$ *to* $[wayclear \wedge DontStop]$ *and wayclear is initialised to true. During simulation, when the configuration becomes* $\{pullover, X\}$ *(here,* X *can be any state in* CA*), conflict is detected between the transitions EVA_t22 and EVA_t27.*

5.2.4 Testing the simulator

Fuzzing contributes to thorough testing and verification by randomly traversing a vast number of paths within statecharts, which can be virtually limitless. It can also be employed to assess the *reachability* of all states and transitions, a common consideration in conventional testing. To guarantee the simulator's intended functionality, it is imperative that all transitions, in line with the structural semantics, remain executable, regardless of their depth within the model. The following list enumerates all potential source and destination states for these

transitions.

1. Possible source states as shown in Figure FC5.6(a):
   (a) case 1 - atomic
   (b) case 2 - composite state (active - atomic substate, composite substate, shell substate)
   (c) case 3 - substate of composite state (active - atomic, composite, shell)
   (d) case 4 - shell state (active - atomic state, shell state, composite state)
   (e) case 5 - region state (active - atomic state, shell state, composite state)
   (f) case 6 - substate of region (active - atomic state, shell state, composite state)
2. Possible destination states as shown in Figure FC5.6(b):
   (a) case 7 - atomic
   (b) case 8 - composite state (default - atomic substate, composite substate, shell substate)
   (c) case 9 - substate of composite state (default - atomic, composite, shell)
   (d) case 10 - shell state (default - atomic state, shell state, composite state)
   (e) case 11 - region state (default - atomic state, shell state, composite state)
   (f) case 12 - substate of region (default - atomic state, shell state, composite state)
3. All possible combinations within a shell state as shown in Figure FC5.6(c)

In Figure FC5.6, the hatched states can be replaced with either shell, composite, or atomic states. A model encompassing all possible transitions was created. This was used to randomly test the simulator for the reachability of all feasible

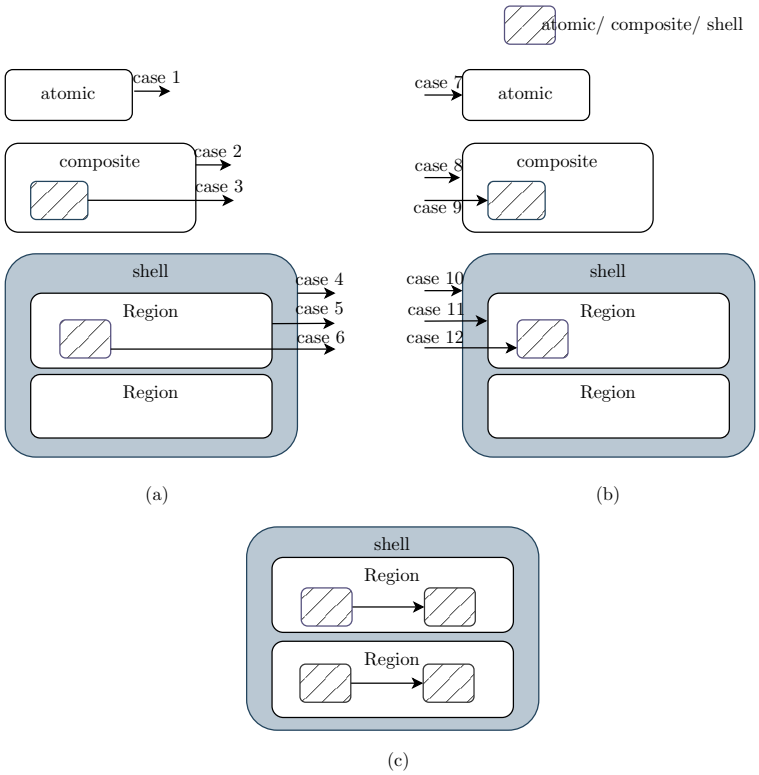


Figure FC5.6: Testing All Potential Transitions via Simulation

configurations by generating event sequences through fuzzing. we use the configuration test property (discussed in section 5.2.3) to accomplish this. We conducted the simulation implementation test - without action code (applicable to all statecharts) and with action code(specific to ConStaBL with local variables) to ensure that all the states in the model are reachable. The fuzzer integrated implementation is available in our GitHub repository[2].

5.2.5 Running extensive tests

Our experiments were conducted on an automotive dataset [1,54] that includes subsystems: Cruise Control (CC), Collision Avoidance (CA), Lane Guidance (LG), Emergency Vehicle Avoidance (EVA). We transformed the dataset from Statemate to ConStaBL semantics, omitting the transition priorities in our model intention-

[2]https://github.com/sujitkc/statechart-verification/tree/skc-simulator-test

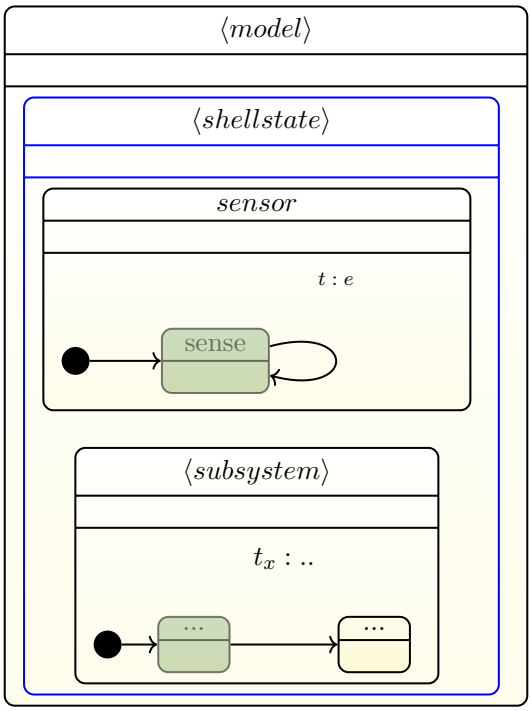


Figure FC5.7: A template shell composition in a configuration $\mathbb{C} = \{sense, X\}$ that combines sensor state and subsystem under test (like CA/EVA/CC/LG).

ally to test it in a non-prioritised system. The goal of this study was to check whether our simulator is able to flag *conflicts* when priorities were removed from transitions.

The automotive dataset takes its inputs from the environment and other subsystems. The inputs to the system, such as AccelPedal, BrakePedal, Speed, etc., were generated randomly within the provided range. For the purpose of our simulation, we have mimicked this by concurrently combining them with a sensor state, as shown in Figure FC5.7. The sensor state includes a self-loop on an event e, ensuring that all the inputs required by the subsystem under test are periodically initialised or randomised.

As fuzzing generates random input events, and since most of these models lacked outgoing transitions from the $Fail$ state, we refrained from generating the *error* event that could potentially result in the $Fail$ state.

The *simulate* method was called with event sequences generated by the fuzzer. We experimented with event queues of varying lengths, specifically 500, 10,000, 20,000, and 300,000 events. The fuzzed execution effectively detected instances of *conflicting transitions*. The number of events required for detection varied across runs, as we stopped and reported the issue upon detection. The execution duration ranged from 5 seconds to 5 minutes, depending on the size of the event sequence processed on a Ubuntu-based server with 16GB of RAM.

Jazzer produced crash files that replicated the runs, allowing us to reproduce the paths leading to *conflicting transitions*. When the simulator didn't detect *conflicting transitions* in prioritised pairs from the state machine model, we found that the guards of those transitions were mutually exclusive. Since our simulator halts and reports issues, we ran it multiple times, resolving the issue each time it was detected. We resolved conflicts in transitions by adding a variable into the guard with different valuations for the conflicting transitions. All cases that were detected by our simulator were true positives. When the simulator didn't detect *conflicting transitions* in prioritized pairs from the state machine model, we found that the guards of those transitions were mutually exclusive.

Example 20 (Conflicting transitions in CA)**.** *Collision Avoidance (CA) System [1, 54] can assist drivers by helping to prevent or mitigate accidents. Combining long and short range radars with a video camera, Collision Warning monitors the road ahead (including part of the side-fronts). In the event that a vehicle or obstacle approaches, collision warning system can notify the driver of a possible collision through audible or visual alerts and can also provide braking force as soon as an imminent collision is detected. The states Disabled, Override, Failed, and Enabled, are substates of Collision Avoidance subsystem. Enabled further encompasses substates, namely Disengaged, Engaged, and Halt, with Engaged containing substates Idle, Warn, Avoid, and Mitigate. The CA system comprises a total of 23*

transitions, with 20 of them defined with priorities. Conflicting transitions arose from transitions that existed at different hierarchical levels. For instance, the outgoing transition of Idle, ($t_x : e[ThreatCA = 2]$) conflicted with the outgoing transition of Engaged, ($t_y : e[((Speed > 0)\&\&(Speed <= 25))||(PRNDL\_In! = 3)]$), as their guards were evaluated to true when we ran the simulator by fuzzing with event sequence. But, all three outgoing transitions from the state Idle had guards of $ThreatCA = 1$, $ThreatCA = 2$, and $ThreatCA = 3$, which did not contribute to conflicts, but transition priorities were defined [54] in the actual dataset in stateflow. Thus, if the model does not have inherent conflicts, the semantics do not need to mandate a priority scheme, which could help in building more abstract and priority-independent system. We repeated the same analysis in other subsystems, such as Cruise Control, Emergency Vehicle Avoidance, and Lane Guidance, and discovered similar outcomes concerning transition priorities.

5.3 Summary

To sum up, we integrated a fuzzer:

1. To test the simulator itself and evaluate its use as a model exploration tool, we employed the simulator in models of various sizes and generated a large number of events.
2. To assess the reachability from one random configuration to another, we generated event sequences using fuzzing as part of our tool's coverage testing.
3. To verify properties within the statechart, like conflicting transitions and desired/undesired configuration.

This experimentation has provided us with confidence in the effectiveness of fuzz testing in statecharts. It has also opened up possibilities for employing fuzzing

to verify different properties, which could be an area for future research. Finally, the entire experiments we have conducted is applicable to detecting bugs in upstream models and can allow early validation and detection of bugs. In all these experiments, the models were developed using the semantics presented in this work, proving their practicality and applicability in the larger models. The experiments were run using our simulator, demonstrating its usefulness in performing bug detection in models.

CHAPTER 6

EXHAUSTIVE EXPLORATION OF CONSTABL

6.1 Introduction

This chapter discusses on execution of statecharts with concurrency to explore all possible interleavings, specifically focusing on our variant, ConStaBL4. The current semantics of ConStaBL computes the code for enabled transitions and executes the code for a single, arbitrarily chosen interleaving. To verify properties in all possible executions, one way is to explore all possible states of execution as in model-checking [35]. In this chapter we present an algorithm for exhaustive exploration in the context of statecharts. We provide examples to illustrate our discussions.

6.2 Scope of exploration

The methodology presented can explore all feasible interleavings in statecharts. While the algorithm is versatile and capable of operating at any depth, we restrict the exploration based on maxdepth parameter to ensure halting of exploration.

As per our semantics, the simulation proceeds in two phases: the INITIALISATION phase and PROCESS-EVENT phase. In the INITIALISATION phase, the

simulator executes the entry actions of the root state and its default states recursively until the atomic descendent state. In the PROCESS-EVENT phase, the corresponding exit and entry actions of the source and destination states of the enabled transitions are executed.

The code is composed accordingly for both of these scenarios. The code is a sequential/concurrent composition of CFGs of the entry and exit action blocks of the states of the enabled non-conflicting transitions. The execution of each statement in these CFGs changes the value environment.

The execution of the statechart is denoted by the formation of a node tree. We maintain two types of nodes (internal and external) during exploration that stores information regarding additional nodes that need to be explored. An *external* node contains the configurarion, value environment (σ) and the event (*event*) to be processed from that node. When the *event* is processed on an *external* node, the set of enabled transitions is obtained. *Internal* nodes is created during the execution of the code corresponding to the set of enabled transitions. That *internal* node saves the altered value environment (Δ_{stmt}) on execution of a statement and the $readyset(R)$ - the set of next control points that needs to be explored (control points are detailed in section 4.3.4.4).

The number of nodes created during exhaustive exploration can quickly explode when the code is in concurrent composition. Also, exploration can grow rapidly if we encounter an assignment statement with input (for instance, $var \leftarrow input()$). So we restrict processing up to the *maxdepth*—that gives the number of nodes to be processed in a path.

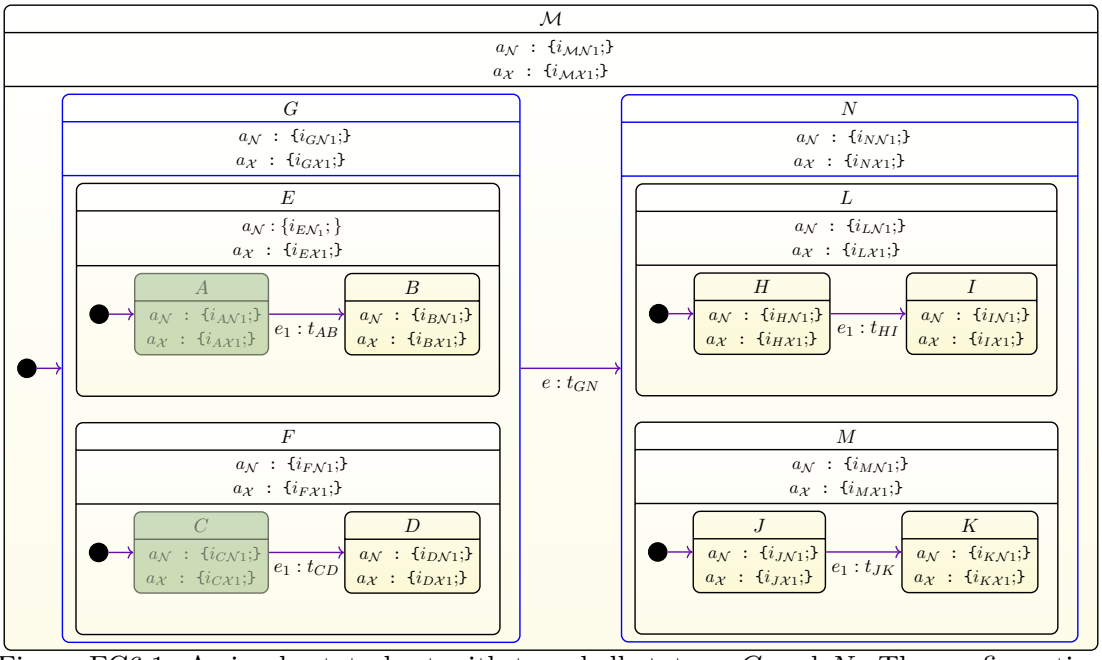


Figure FC6.1: A simple statechart with two shell states - G and N. The configuration is $\{A, C\}$

6.3 Exhaustive state space exploration

6.3.1 Processing external nodes.

An *External node*, maintains three values: the current stable configuration of the statechart, the value environment, and the next event to be processed. External nodes therefore represent the stable configurations from which the subsequent event can be processed. When processing an event, the next code to be executed is determined based on the current state and the value environment contained in the external node.

Example 21. *The simulation begins with the external node -* $E_1 : \langle \mathbb{C} : \emptyset, \sigma : \emptyset, event : init \rangle$ *for the model shown in Figure FC6.1. Let us assume that the event list consists of* e_1, e.

The statechart is first initialised based on $ST_i(\mathcal{M}) = \langle \mathcal{M}, G, [\langle E, A \rangle | \langle F, C \rangle] \rangle$. *Thus, the next stable configuration is* $\{A, C\}$. *The next event to be processed is* e_1.

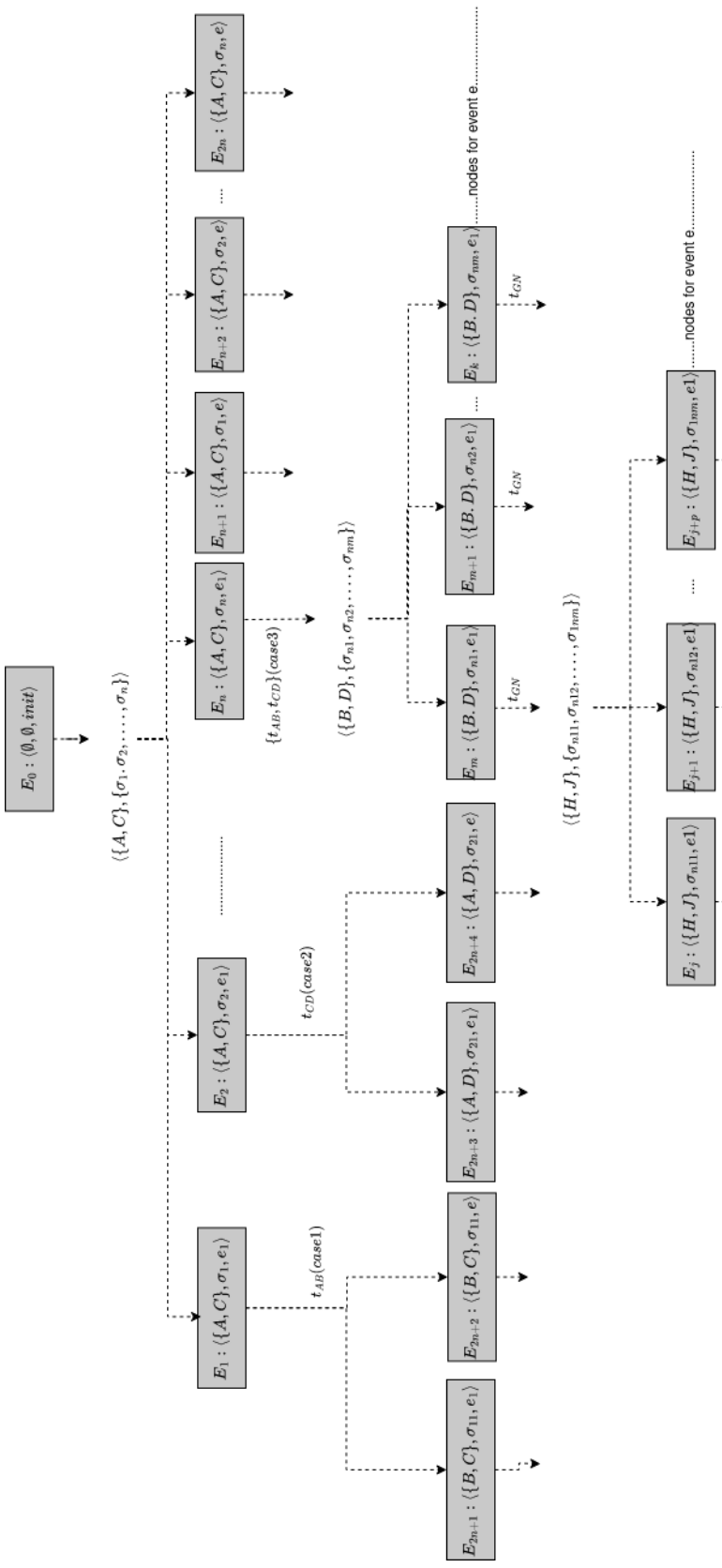


Figure FC6.2: External nodes for the statechart in Figure FC6.1

But, the next σ depends on the interleaving taken during code execution. Thus, multiple value environments due to interleaving (i.e. $\sigma_1, \sigma_2, ..., \sigma_n$) are created (here, n depends on the number of interleavings explored during code execution). Thus, n external nodes are created for each event in the event list (i.e., $(\mathbb{C}, \sigma_1, e_1)$, $(\mathbb{C}, \sigma_2, e_1)$, $(\mathbb{C}, \sigma_3, e_1)$,.....,$(\mathbb{C}, \sigma_n, e_1)$, $(\mathbb{C}, \sigma_1, e)$, $(\mathbb{C}, \sigma_2, e)$, $(\mathbb{C}, \sigma_3, e)$,.....,$(\mathbb{C}, \sigma_n, e)$).

An illustration of external nodes that can be created during exploration is depicted in Figure FC6.2. This figure shows the tree structure with all of the external nodes connected to the next external nodes.

The execution begins by processing node $E_0 : (\mathbb{C} = \emptyset, \sigma = \emptyset, init)$ (here, the event init is to denote the statechart initialisation). The corresponding code, $\langle \mathcal{M}.a_{\mathcal{N}}, G.a_{\mathcal{N}}, [\langle E.a_{\mathcal{N}},\ A.a_{\mathcal{N}} \rangle \mid \langle F.a_{\mathcal{N}}, C.a_{\mathcal{N}} \rangle] \rangle$, when executed creates n environments and the configuration becomes $\{A, C\}$. Thus we create the next possible set of external nodes for each event in the event list as $(\mathbb{C} = \{A, C\}, \sigma = \{\sigma_1.\sigma_2, ..., \sigma_n\}, e_1)$,$(\mathbb{C} = \{A, C\}, \sigma = \{\sigma_1.\sigma_2, ..., \sigma_n\}, e)$. The number of nodes here is n. This is dependent on the number of interleavings executed (each of them creates a distinct value environment). For each event in E_Q, next external node is created. The external nodes E_1, E_2 .. E_n are created for event e_1 and $E_{n+1}, E_{n+2}....E_{2n}$ are created for the event e. When processing the event e_1 from these external nodes, three cases of enabled transitions set are possible (assuming that the guard is evaluated to true in the respective value environment): $\{t_{AB}\}$(Case 1), $\{t_{CD}\}$(case 2), or $\{t_{AB}, t_{CD}\}$(case 3). We illustrate the exploration of case 1 on node E_1. When the enabled transition is t_{AB}, the code that will be executed will be sequential ($\langle A.a_{\mathcal{X}}, B.a_{\mathcal{N}} \rangle$) so it will create only one execution trace and one next value environment(σ_{11}). The corresponding next external nodes are $E_{2n+1} : (\{B, C\}, \sigma_{11}, e_1)$ and $E_{2n+2} : (\{B, C\}, \sigma_{11}, e)$ are created. Similarly, when the enabled transition is t_{CD}, the next external nodes $E_{2n+3} : (\{A, D\}, \sigma_{21}, e_1)$ and $E_{2n+4} : (\{A, D\}, \sigma_{21}, e)$ are created. In case 3, two transitions are enabled

$\{t_{AB}, t_{CD}\}$*(non-conflicting transitions), and the code corresponding to these transitions will be in concurrent composition. Thus, each execution trace corresponding to different interleaving will create a value environment* $(\sigma_{n1}, \sigma_{n2}, ..., \sigma_{nm})$ *and the corresponding external nodes* E_m *to* E_k *are created.*

Similarly, the transition t_{GN} *from external node* E_m *is will create a set of next nodes* $E_j, E_{j+1}, ..., E_{j+p}$ *for the event* e_1 *and further nodes* $E_{j+p+1}, ..., E_{j+2p}$ *with event* e*. The code processed here is* $\langle[\langle A.a_{\mathcal{X}},\ E.a_{\mathcal{X}}\rangle \mid \langle C.a_{\mathcal{X}},\ F.a_{\mathcal{X}}\rangle], G.a_{\mathcal{X}},\ t_{GN}.a,\ N.a_{\mathcal{N}},\ [\langle L.a_{\mathcal{N}},\ H.a_{\mathcal{N}}\rangle \mid \langle M.a_{\mathcal{N}},\ J.a_{\mathcal{N}}\rangle]\rangle$.

Exploration continues until all the nodes in the queue is explored or the maximum depth defined is reached on every path. To sum up, exhaustive exploration is with respect to a specific event list and maximum depth is defined in terms of the instructions to avoid infinite execution.

6.3.2 Processing internal nodes.

Between two external nodes, several *internal nodes* are created, that store the *ready set* (the next set of control points) and Δ_{stmt} - the environment that stores the variable and valuation modified by the statement executed (each control point corresponds to the end of execution of a statement).

Example 22. $\Delta_{iG\mathcal{N}1}$ *is an internal node that is created by executing the statement* $iG\mathcal{N}1$ *and stores the valuation of the variable modified by this statement. It also stores the next set of control points* $[CP_{iE\mathcal{N}1}, CP_{iF\mathcal{N}1}]$ *that corresponds to the execution of the statements* $iE\mathcal{N}1$ *and* $iF\mathcal{N}1$*. Though code execution is as per the operational semantics presented earlier in chapter 4, here it explores all the next control points for execution and creates two internal nodes -* $\Delta_{iE\mathcal{N}1}$ *and* $\Delta_{iF\mathcal{N}1}$ *respectively. Figure FC6.3 shows all internal nodes created in between two external nodes. Consider the case 2 in the exploration tree in FC6.3, the value*

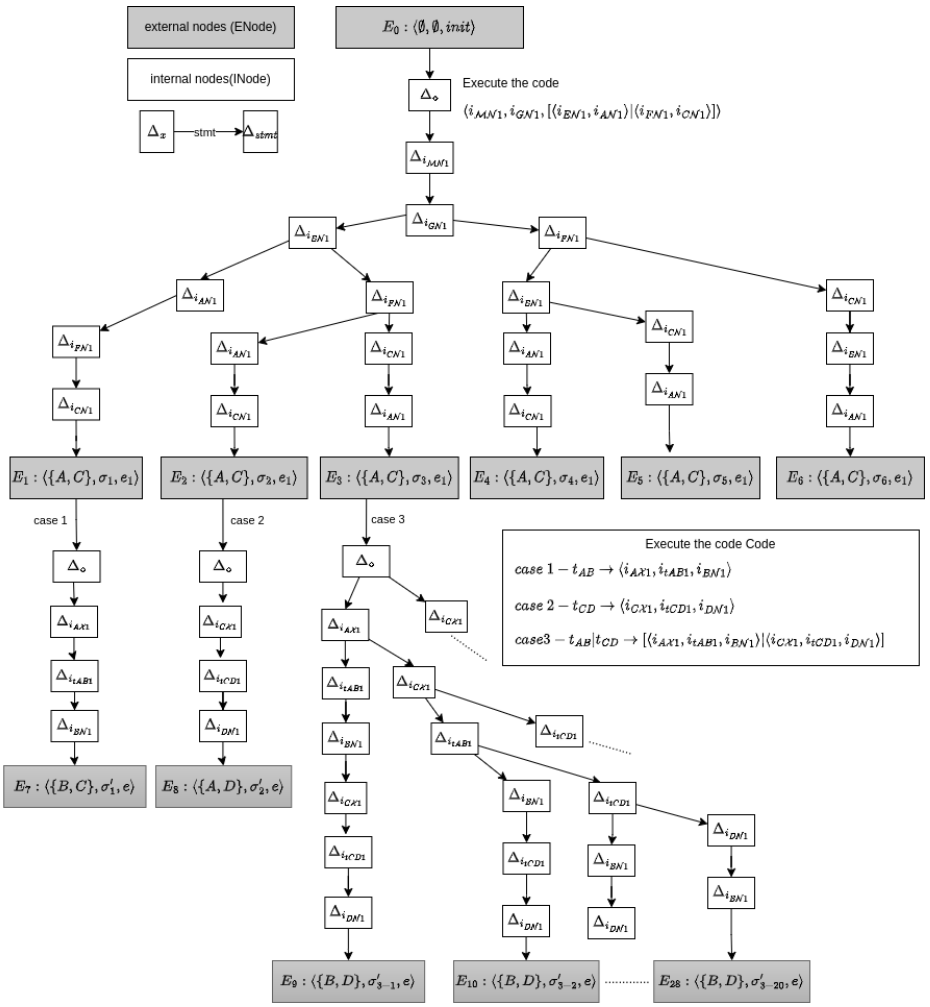


Figure FC6.3: Internal nodes created during exploration

environment needed for execution of i_{tCD1}, is given by $\Delta_{iC\mathcal{X}1} :: \Delta_{\diamond} :: \sigma_2$. External nodes do not interleave, so the effect of concurrency is confined between two external nodes. Here Δ is the partial environment in internal node and σ corresponds to the complete environment present in external node.

6.3.3 Tradeoff

Although our algorithm aims to traverse all possible paths, the sheer number of interleavings that need to be explored can rapidly escalate, resulting in state space explosion. This surge occurs due to the exponential increase in the number of nodes w.r.t to the number of interleavings, contributing to the overall complexity of the exploration process.

Example 23. *If the action blocks in Figure FC6.1 contain statements as shown below (2 statements per action block).*

- ***Entry codes:*** $A.a_{\mathcal{N}} = \{i_{A\mathcal{N}_1}; i_{A\mathcal{N}_2};\}$, $E.a_{\mathcal{N}} = \{i_{E\mathcal{N}_1}; i_{E\mathcal{N}_2};\}$ *etc.*
- ***Exit codes:*** $A.a_{\mathcal{X}} = \{i_{A\mathcal{X}_1}; i_{A\mathcal{X}_2};\}$, $E.a_{\mathcal{X}} = \{i_{E\mathcal{X}_1}; i_{E\mathcal{X}_2};\}$ *etc.*
- ***Transition action codes:*** $t_{AB} = \{i_{AB_1}; i_{AB_2};\}$, $t_{CD} = \{i_{CD_1}; i_{CD_2};\}$ *etc.*

Suppose, the input event is e and transition t_{GN} is enabled, then the code to be executed is

$$\begin{aligned}\mathcal{C}(t_{GN}, \mathbb{C}) = \langle [\langle A.a_{\mathcal{X}}, E.a_{\mathcal{X}}\rangle | \langle C.a_{\mathcal{X}}, F.a_{\mathcal{X}}\rangle],\\ G.a_{\mathcal{X}}, t_{GN}.a, N.a_{\mathcal{N}},\\ [\langle L.a_{\mathcal{N}}, H.a_{\mathcal{N}}\rangle | \langle M.a_{\mathcal{N}}, J.a_{\mathcal{N}}\rangle]\rangle\end{aligned}$$

A possible trace for this code is $i_{A\mathcal{X}_1}$, $i_{C\mathcal{X}_1}$, $i_{A\mathcal{X}_2}$, $i_{C\mathcal{X}_2}$, $i_{E\mathcal{X}_1}$, $i_{F\mathcal{X}_1}$, $i_{E\mathcal{X}_2}$, $i_{F\mathcal{X}_2}$, $i_{G\mathcal{X}_1}$, $i_{G\mathcal{X}_2}$, i_{GN_1}, i_{GN_2}, $i_{N\mathcal{N}_1}$, $i_{N\mathcal{N}_2}$, $i_{L\mathcal{N}_1}$, $i_{M\mathcal{N}_1}$, $i_{L\mathcal{N}_2}$, $i_{M\mathcal{N}_2}$, $i_{H\mathcal{N}_1}$, $i_{J\mathcal{N}_1}$, $i_{H\mathcal{N}_2}$,

$i_{J\mathcal{N}_2}$. *The portion highlighted in blue shows the code interleaved due to the concurrent composition of the action block. Here, for* $[\langle A.a_{\mathcal{X}}, E.a_{\mathcal{X}}\rangle|\langle C.a_{\mathcal{X}}, F.a_{\mathcal{X}}\rangle]$, *there can be 70 possible paths caused by interleaving. It is the same with* $[\langle L.a_{\mathcal{N}}$, $H.a_{\mathcal{N}}\rangle|\langle M.a_{\mathcal{N}}$, $J.a_{\mathcal{N}}\rangle]$ *as well. The entire code will have 70 * 70 possible combinations. The exploration of all potential interleavings will significantly expand in a larger statechart or a statechart with nested concurrency. Exploring all potential interleavings becomes necessary, as each of these interleavings might result in different outcomes. The objective is to ensure that none of the behaviours remains unexplored. Therefore, our algorithm is designed to explore all interleavings during a statechart simulation.*

Similarly, when the input event is e_1, *the code to be executed in this case is* $[\mathcal{C}(t_{AB}, \mathbb{C}), \mathcal{C}(t_{CD}, \mathbb{C})] = [\langle A.a_{\mathcal{X}},\ t_{AB}.a,\ B.a_{\mathcal{N}}\rangle, \langle C.a_{\mathcal{X}},\ t_{CD}.a,\ D.a_{\mathcal{N}}\rangle]$ *will result in 924 interleavings.*

Algorithm 2: Exhaustive exploration of statechart

procedure ExhaustiveExplore(SC : Statechart, E_Q : Events, $maxdepth$: number)

Q: **Queue** of **Nodes**

$initialNode \leftarrow$ ENode($\emptyset$, $\emptyset$, init)

Q.enqueue($initialNode$)

while Q *is not empty* **do**

  $current_n \leftarrow Q$.dequeue()

  if $current_n.depth < maxdepth$ **then**

    if $current_n$ *is an* ***ENode*** **then**

      code = computeCode($current_n$)

      Q.enqueue(new INode($current_n$, first(code), $\Delta_\diamond$))

    if $current_n$ *is an* ***INode*** **then**

      forall *child in* $current_n.R$ **do**

        $\Delta_{\text{child}} \leftarrow \sigma_{current_n} \vdash execute(child.stmt)$

        Q.enqueue(new INode($current_n$, next($child$), Δ_{child}))

      if $|current_n.R| = 0$ **then**

        $\forall e \in E_Q \bullet$ Q.enqueue(new ENode($current_n$, $\mathbb{C}'$, σ', e))

▷ Node can be of types INode or ENode

▷ INode (Internal node) is a tuple with parent, R, and Δ

▷ ENode (External node)is a tuple with parent, $\mathbb{C}$, σ, and event

▷ Here, $\sigma_{current_n} = \Delta_{current_n} :: \Delta_{current_n.p} :: .. :: \sigma_{initialNode}$

6.4 Algorithm

The algorithm 2 is described as follows:

- The procedure *ExhaustiveExplore* takes three inputs - statechart, event list and maximum depth.

- A queue of $Nodes$ is initialised. Here, *ENode* is an external node and *INode is the internal node*
- An initial external node is created where the parent is $\emptyset$, σ is $\emptyset$ and the event to be processed is *init* and it is added to the queue of nodes Q (the compute code method to compute the code corresponding to statechart initialisation when processing *init* event).
- The queue of nodes is processed until it becomes empty or reaches maximum depth
  - $current_n$ is the node selected for processing
  - if the depth of $current_n$ is greater than $maxdepth$ then stop exploration and move to the next node in the queue. Otherwise, proceed to next step.
  - if it is an $ENode$, then the code is computed by processing the *event*. Enabled transitions are picked based on the *event* and σ of $current_n$ to compute the code. Subsequently, an $INode$ is created with $current_n$ as parent and $readyset(R)$ given by $first(code)$ and an empty environment $\Delta_\diamond$. This marks the beginning of the interleaved execution of internal nodes.
  - if it is an $INode$, then for each *child* of R, the *execute* function is called with the statement and multiple $INode$ are created and added to the Q. When R is empty, a set of new *external node*s are added for each event in the event list.

As per ConStaBL semantics, we process only one event at a time (only one event is processed from an external node). The identification of transition conflicts is omitted here as the focus is on exploration when there are no conflicting transitions,

but non determinism due to conflicting transitions applies to exhaustive exploration as well. Unlike the arbitrary execution of the simulate function that calls the *execute* function only for one next statement, here we proceed with executing every next statement, thereby creating multiple internal nodes.

Here, let us consider the Figure FCFC6.1, the external nodes created is shown in table TC6.1. Initially, the $\mathcal{Q}$ has only one element ($\mathbb{C} = \emptyset, \sigma = \emptyset, event = init$). The code block to be executed is, $\langle \mathcal{M}.a_{\mathcal{N}}, G.a_{\mathcal{N}}, [\langle E.a_{\mathcal{N}}, A.a_{\mathcal{N}}\rangle, \langle F.a_{\mathcal{N}}, C.a_{\mathcal{N}}\rangle]\rangle$.The code to be executed can have 6 interleavings, thereby creating multiple external nodes ($[A, C], \{\sigma_1, \sigma_2, ..\sigma_6\}$). The internal nodes created for one of the explorations EC_0 are given in table TC6.2. EC_0 shows the nodes created for the code $\langle i\mathcal{MN}1, iG\mathcal{N}1, [\langle iE\mathcal{N}1, iA\mathcal{N}1\rangle | \langle iF\mathcal{N}1, iC\mathcal{N}1\rangle]\rangle$, here the initial node $node_0$ is created with initial ready set corresponding to the *first(code)*. A node, $n_0 = INode(par = E_0, readySet = \{CP_{i\mathcal{MN}1}\}, \Delta_\diamond)$ is created. This execution leads to 6 different exploration paths, leading to 6 value environments corresponding to the nodes n_{15} to n_{20}. For any node t the value environment σ_t denotes that of complete execution trace - from the current internal node to the parent up until the most recent external node in the current execution trace. Detailed trace is given in Appendix B.

6.4.1 Verification of properties

Multiple types of properties requiring verification within the statechart have been detailed in section 7.2.3. The discussion now revolves around the approach to accomplish similar verification within the exhaustive exploration process.

1. **Stuck specification**. When processing an external node, if the *code* is empty that implies the model is stuck in that configuration for the given environment and any of the events in the event list. An exploration is not

Table TC6.1: Glimpse of External nodes created during exhaustive exploration at statechart

event	$ENode_{in}$	$(\mathbb{C}, \sigma)$	$\mathcal{T}$	Code	EC	$ENode_{out}$
	$(\mathbb{C} = \emptyset, \sigma = \emptyset)$		-			
init		$E_0 : (\emptyset, \emptyset)$	ST_i	$\langle$ $\mathcal{M}.a_{\mathcal{N}}$,$G.a_{\mathcal{N}}$,[$\langle E.a_{\mathcal{N}}$, $A.a_{\mathcal{N}}\rangle$, | $\langle F.a_{\mathcal{N}}$, $C.a_{\mathcal{N}}\rangle$] $\rangle$	EC_0	$([A, C], \sigma_1)$, $([A, C], \sigma_2)$, $([A, C], \sigma_3)$, $([A, C], \sigma_4)$, $([A, C], \sigma_5)$, $([A, C], \sigma_6)$
e_1	$([A, C], \sigma_2)$, $([A, C], \sigma_3)$, $([A, C], \sigma_4)$, $([A, C], \sigma_5)$, $([A, C], \sigma_6)$	$E_1 : ([A, C], \sigma_1)$	t_{AB}	$\langle A.a_{\mathcal{X}}, t_{AB}.a$, $B.a_{\mathcal{N}}\rangle$	EC_1	$([B, C], \sigma'_1)$
	$([A, C], \sigma_3)$, $([A, C], \sigma_4)$, $([A, C], \sigma_5)$, $([A, C], \sigma_6)$	$E_2 : ([A, C], \sigma_2)$	t_{CD}	$\langle C.a_{\mathcal{X}}, t_{CD}.a$, $D.a_{\mathcal{N}}\rangle$	EC_2	$([A, D], \sigma'_2)$

Table TC6.2: Glimpse of Internal nodes created - EC_0

$INode$	$node_{new}$
-	$n_0 : (iM\mathcal{N}1, E_1, \Delta_\diamond)$
n_0	$n_1 : (iG\mathcal{N}1, n_0, \Delta_{i\mathcal{M}\mathcal{N}1})$
n_1	$n_2 : (\{iE\mathcal{N}1, iF\mathcal{N}1\}, n_1, \Delta_{iG\mathcal{N}1})$
n_2	$n_3 : (\{iA\mathcal{N}1, iF\mathcal{N}1\}, n_2, \Delta_{iE\mathcal{N}1}), n_4 : (\{iE\mathcal{N}1, iC\mathcal{N}1\}, n_2, \Delta_{iF\mathcal{N}1})$
n_3	$n_5 : (\{iF\mathcal{N}1\}, n_3, \Delta_{iA\mathcal{N}1}), n_6 : (\{iA\mathcal{N}1, iC\mathcal{N}1\}, n_3, \Delta_{iE\mathcal{N}1})$

stuck at an external node if $\exists e \in E \bullet \mathcal{T}(e) \neq \emptyset$.

2. **Undefined variable access**. In an $INode$, during execution, if $\sigma[v]$ returns $\diamond$, then the variable is undefined in its path.

3. **Conflicting transitions**. When processing an $ENode$, if there are conflicting transitions, it is flagged in the same manner as in the ConStaBL simulator algorithm

6.5 Summary

The idea to explore all possible nodes stems from our motive of verifying properties on the statecharts and ensuring that there are no paths in which they fail. Formal verification techniques like explicit state space model checking build on such exploration. Interleaving contributes to state space explosion, so we implemented our exploration algorithm and tested it on the toy example in this chapter. Future work will involve implementing partial order reduction-based techniques to optimise the exploration.

CHAPTER 7

VERIFICATION OF STATECHARTS

7.1 Introduction

Manual detection of errors stops scaling when the models become larger, as is likely with StaBL models. Hence, automatic detection of errors in the model is essential. This chapter presents an approach that focuses on the translation of statecharts to conduct symbolic execution to verify properties. We present the following contributions:

- Formal and Automated Translation: We present an approach to automatically translate the models written in StaBL to C and Java code. The symbolic execution engines available for these languages can be used to verify the translated code. The translation preserves the scopes of the variables as the typechecked and model verified for its scope is used for translation.
- Early detection of problems: We generate properties from the model to identify potential issues, such as undefined variables, conflicting transitions, and stuck states, and instrument the code with these properties.
- Symbolic execution: We use the symbolic execution engines KLEE [36] and JPF-Symbc [37] to check the satisfiability of these properties in the generated

code.

Symbolic execution is a programme analysis technique that uses symbolic values to explore multiple execution paths, allowing for better problem-solving and identifying potential issues. It works on the principle of constraint solving. As a symbolic execution engine explores different paths symbolically, it gathers constraints on program inputs along these paths. It uses constraint-solving techniques (often leveraging SMT solvers) to check for the satisfiability of these constraints, and only if the constraint is satisfied does the exploration continue along that path. We aim to use this ability to identify scenarios leading to certain conditions. All our properties (undefined variable access, stuck specification, and conflicting transitions) are instrumented as assertions into the code, and such assertion violations can be detected by constraint solvers during symbolic execution, and we get an execution trace that violates the assertion, which is useful in debugging the models.

KLEE is a widely-used symbolic execution tool designed primarily for analyzing and verifying software systems written in C language. JPF-Symbc, is a symbolic execution tool, that builds upon the foundation of Java PathFinder (JPF), a versatile framework for Java bytecode verification. JPF-Symbc interweaves symbolic execution capabilities into JPF's core, enabling it to explore a multitude of program paths and uncover potential vulnerabilities.

Key applications of symbolic execution include bug finding, test case generation, vulnerability discovery, and formal verification. Benefits include enhanced test coverage, automated analysis, and property checking, reducing manual effort and revealing hidden issues.

Figure FC7.1 shows our broad approach we use to verify StaBL models. In this context, the type-checked StaBL model is inputted to the flattener. The approach involves flattening the statechart model and translating it into an intermediate

representation. Our translator also inserts the specifications corresponding to the properties to be checked in the right place in the program. This intermediate representation (IR) is translated into a program (C or Java) with assertions based on the specifications. When the specification is not satisfied, the symbolic execution engines generate the counter-example trace. As our methodology is a systematic translation, the counter-example traces can be mapped back to the unflattened statechart models to be used with our simulator to reproduce the issue. For each step shown in Figure FC7.1, we have developed translators. Symbolic execution engines then process the resulting code to detect any violations.

7.2 Translation of statechart

The core of our work is to present a translation algorithm from StaBL to C and Java to leverage the capabilities of existing symbolic execution engines such as KLEE and JPF-Symbc. The translation procedure has 3 major steps: flattening, translation to IR, instrumentation and conversion.

The flattener follows a sequence of procedures that involve globalising the variables, states, and transitions. This process also encompasses reconstructing the action blocks for the transitions. The actions stemming from state entry and exit are amalgamated into translation actions. Subsequently, this flattened statechart is passed on to the Translation and Instrumentation section. Here, the flattened statechart undergoes conversion into an intermediate representation(IR)/ program, and pertinent assertions are inserted based on the properties intended for verification. This intermediate representation is then further transformed into code in the target language (e.g C/C++ for KLEE, Java for Jpf-Symbc).

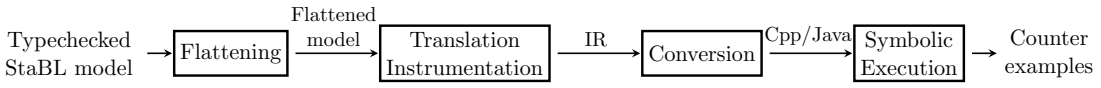


Figure FC7.1: Formal verification: Overall approach

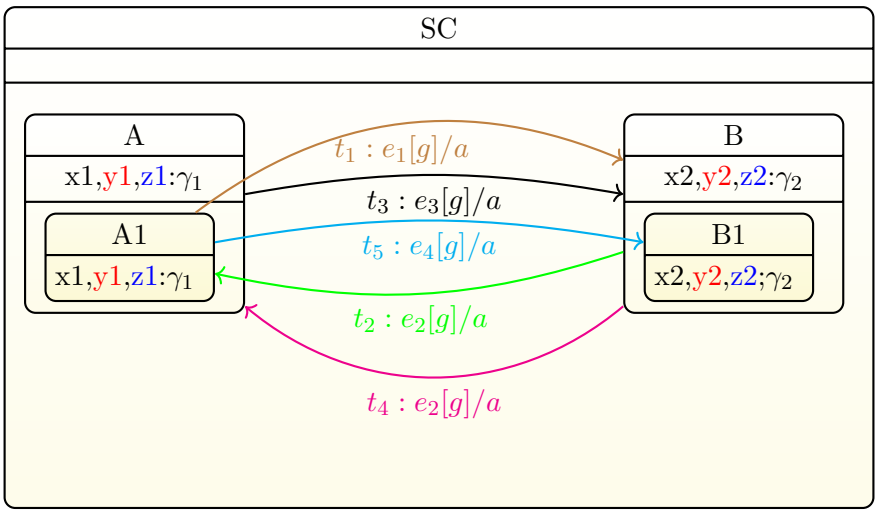


Figure FC7.2: Example for flattening statechart

7.2.1 Flattening

We translate the source statechart SC to its flattened equivalent SC' using a translation process X. We say $SC' = X(SC)$.

$X = X_{S2} \circ X_{S1} \circ X_{T3} \circ X_{T2} \circ X_{T1} \circ X_G$ where $\circ$ stands for function composition. X has the following steps:

1. X_G: Each variable v is lifted onto the global scope giving a new variable v', suitably renamed for disambiguation. We say: $v' = G(v)$. In a piece of program code C, if all variables v in C are replaced by $G(v)$, we get a new piece of code $G(C)$. We have 3 types of storage classes for variables - local, parameter and static. The translation process adds appropriate resetting of variables. As our type checker has already verified the usage scope of these variables, the action code is properly appended depending on the variable of use.

- For local variables, $clear(v)$ is appended to the action block of each outgoing transition of the state.
- If the state is marked with history, the clear function will store the value in *value history* before clearing it.

Example 24. *For instance, in Figure FC7.2, there are 3 variables in each state, the colour codes represents the storage class of the variables - local (black), parameter (red) and static (blue). Colours in the arrow is for representational, to just show the label and transition together. It does not have any semantic role. The first step is to globalise the variables. Here two variables are globalised, with the information of its containment hierarchy. For example, the variable* $x1, p1$ *and* $z1$ *contained in A are globalised as* A_x1*,* A_p1 *and* A_z1 *respectively. Similarly for all states, globalisation of variable is done. The variables contained inside A1 when globalised all of its ancestors are added to maintain uniqueness. Like* A_A1_x, A_A1_y, A_A1_z *respectively.*

In Figure FC7.2, $A1\_x1$ *is a variable in local scope and* $t1$ *is the outgoing transition, we append* $clear(A1\_x1)$ *to the action of* $t1$ *(Please note that the variable* $x1$ *is readable in outgoing transition, so we append the clear construct). The new action code will be* $\langle t1.a, clear(A1\_x1)$ *(please note :* $\langle a1, a2\rangle$ *is the sequential composition - recalling definition in chapter 4). As per our scoping policy, both A and* $t3$ *does not have access on* $A1\_x1$*, so we donot append clear in this transition. Similarly, clearance of* $p1$ *is appended to the exit action. So the new exit action will be* $\langle A1.a_{\mathcal{X}}, clear(A1\_x1)\rangle$.

2. X_{T1}: Translation $X_t(t)$ of every transition t such that $atomic(t.s) \wedge atomic(t.d)$ is to $\{t'\}$ such that $t' = X_{T1}(t)$, $t'.g = G(t.g)$ and action block with label, $t'.a = G(\langle t.s.a_{\mathcal{X}}, s^1.a_{\mathcal{X}}, ..., s^n.a_{\mathcal{X}}, t.a, d^m.a_{\mathcal{N}}, ..., t.d.a_{\mathcal{N}}\rangle)$. $t'.s.a_{\mathcal{X}} = t'.d.a_{\mathcal{N}} = \{\}$. Where $\sqcap\{t.s, t.d\} = s^{n+1} = d^{m+1}$.

Example 25. *In Figure FC7.2, the transition t5 is between A1 and B1 - two atomic states. A transition t5' is created with action composition -* $\langle A1\_a_{\mathcal{X}}$*,*$A.a_{\mathcal{X}}$*,t5.a*
$,B.a_{\mathcal{N}},B1.a_{\mathcal{N}}\rangle$.

3. X_{T2}: Translation $X_t(t)$ of every transition t such that $\neg atomic(t.s)$ is to transitions $\{t'_1, t'_2, ..., t'_n\}$ such that, if $t.s$ has n atomic descendant states s_1, s_2, ..., s_n, then $\forall i,\ 1 \leq i \leq n, s'_i = t'_i.s$. Also, $\forall i,\ 1 \leq i \leq n,\ t'_i.e = t.e$ and $t'_i.g = G(t.g)$. $\forall i,\ t'_i.a = G(\langle s^1.a_{\mathcal{X}}, s^2.a_{\mathcal{X}}, ..., s^m.a_{\mathcal{X}}, t.a\rangle)$ where $s^1,\ s^2,\ ...,\ s^m \in S(SC)$, $s^1 \prec_1 s^2 \prec_1 ... \prec_1 s^{m-1} \prec_1 s^m$, $s^m = t.s$ and $s^1 \prec_1 t.s \sqcup t.d$. Also, $t'.s.a_{\mathcal{X}} = \{\}$.

   Example 26. *In Figure FC7.2, the transition* $t2$ *has a non atomic source, so it maps to a set of transitions - one per substate* $\{t2'\}$ *with action* $\langle B1.a_{\mathcal{X}}, B.a_{\mathcal{X}}, t2.a\rangle$.

4. X_{T3}: Translation $X_t(t)$ of every transition t such that $\neg atomic(t.d)$ is to $\{t'\}$ such that $init^*(t.d) = X_t^{-1}(t'.d)$. $t'_i.e = t.e$. $t'_i.g = G(t.g)$. $t'.a = G(\langle t'.a, D^n.a_{\mathcal{N}}\ ,\ D^{n-1}.a_{\mathcal{N}}\ ,\ ...\ ,\ D^1.a_{\mathcal{N}}\rangle)$ where $D^1,\ D^2,\ ...,\ D^n \in S(SC)$, $D^1 \prec_1 D^2 \prec_1 ...D^{n-1} \prec_1 D^n$, $D^1 = t.d$ and $D^n \prec_1 t.s \sqcup t.d$. Also, $t'.d.a_{\mathcal{N}} = \{\}$.

   Example 27. *In Figure FC7.2, the transition* $t1$ *has a non atomic destination, so it maps to a set of transitions - one per substate of destination* $\{t1'\}$ *with action* $\langle t1'.a, B.a_{\mathcal{N}}, A1.a_{\S}\rangle$ *and guard* $t2'.g = G(t2.g)$.

5. X_{S1}: The initial state s' in SC' is such that $s = X_s^{-1}(s')$ where $atomic(s)$ and $s_1 \prec_1 s_2 \prec_1 ... \prec_1 s_n$, $s = s_1$, $SC = s_n$, and $\forall i, 1 \leq i < n, s_i = init(s_{i+1})$.
   We say: $s = init^*(SC)$

   Example 28. *In Figure FC7.2, the initial state, s' is A1 that must execute* $\langle init(SC)$*,* $init(A)$*,*$init(A1)\rangle$ *considering A is the default state of SC and A1 is the default state of A.*

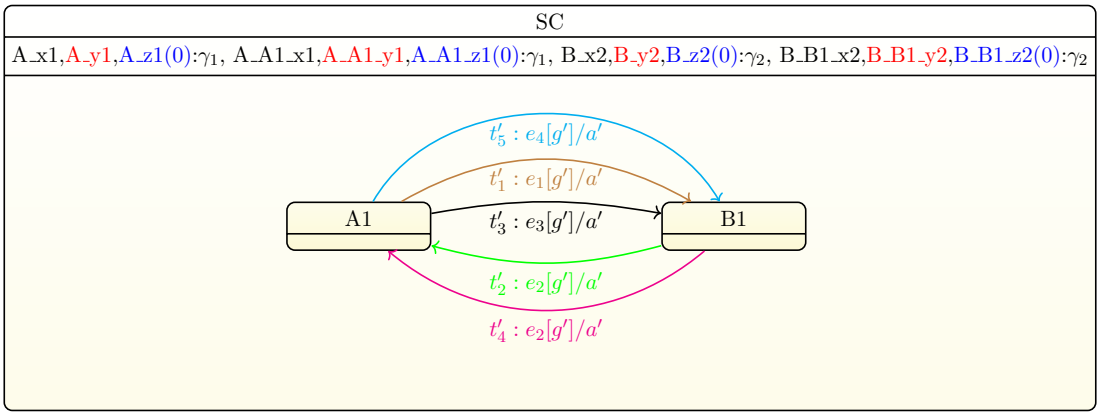


Figure FC7.3: Flattened statechart for Figure FC7.2

```
1  wait(event) {
2    match state with
3    ...
4    t.s ⇒
5      if(event = t.e)
6        if(t.g)
7          t.a
8          state ← t.d
9    ...
10 }
```

Figure FC7.4: Abstract representation of program equivalent of a statechart

6. X_{S2}: $\forall s \in S(SC)$ such that $atomic(s)$, s has a corresponding state s' in SC', given by $s' = X_s(s)$. $\forall s \in S(SC)$ such that $\neg atomic(s)$, $X_s(s)$ is not defined. In other words, all non-atomic states are done away with.

   Example 29. *The non atomic states A and B are removed.The resulting statechart model is shown in Figure FC7.3.*

7.2.2 Translation to Program

After flattening, we translate the flattened chart into a program that acts as an intermediate representation. For a transition t the generated code is as shown below:

A variable *state* keeps track of the current state. The transition actions are

implemented as branches on the value of the *state* variable and the *event*. Each of these branches contains code which is the action of the corresponding transition in the flattened chart. This logic is contained within an endless while-loop waiting over *event*. Each state transition now translates to one iteration of this while-loop. The pseudo-code of the program generated from the model in Figure FC7.5 is shown in listing. FC7.6.

Depending on what properties we wish to prove, we insert appropriate assert statements in the generated program at appropriate places. The program is then translated to languages such as C/Java to be used with symbolic execution engines such as KLEE and JPF-Symbc indicating the presence (or absence, as the case may be) of a property that we are trying to prove about the input specification.

7.2.2.1 KLEE

KLEE [55] is an automatic test generation tool that uses symbolic execution to systematically explore all possible paths a program can take. By treating inputs as symbols, KLEE can uncover hidden bugs triggered by specific edge conditions. It achieves high code coverage by efficiently representing program states and using smart search techniques to explore all conditional branches. This allows KLEE to identify not only basic errors like memory issues and assertion violations but also dangerous operations that could lead to problems under specific input scenarios.

KLEE operates symbolically, generating constraints that describe the possible values on different execution paths. When an error is detected or a path reaches an exit call, KLEE produces a test case based on the path's constraints. These tests can be rerun on an unmodified version of the program to ensure consistent behaviour. KLEE's ability to achieve high coverage efficiently makes it a valuable tool for software testing and verification.

KLEE operates by translating the program to LLVM bitcode, then executing it with symbolic inputs. It systematically explores each branch of the program by solving constraints to determine feasible paths. When a new path is encountered, KLEE forks the state, creating a new execution state for each possible branch outcome. The engine continues this process, keeping track of the constraints for each path and using an SMT (Satisfiability Modulo Theories) solver to check the feasibility of the paths. KLEE generates detailed logs and test cases that include concrete inputs for each path explored. These test cases can be rerun to verify the program's behavior under different conditions.

7.2.2.2 JPF-Symbc

JPF-symbc is an extension of the Java PathFinder (JPF) tool, designed to incorporate symbolic execution for more thorough analysis of Java bytecode. JPF, an open-source model checker developed by NASA, traditionally uses concrete values for input, which limits its ability to explore all potential program states. Symbolic execution, by treating inputs as symbolic variables, enables simultaneous exploration of multiple paths and reveals bugs that might not be encountered with specific input values. JPF-symbc enhances JPF by adding capabilities to handle symbolic variables, generate path conditions, and interface with constraint solvers to evaluate feasible execution paths. This integration extends JPF's analysis scope, making it a powerful tool for detecting bugs, identifying security vulnerabilities, formally verifying software properties, and generating comprehensive test cases. Despite its significant impact on software verification and testing, JPF-symbc faces challenges such as state explosion, scalability issues, and dependency on the performance of constraint solvers. Nonetheless, it remains a vital tool in advancing reliable and secure software development, facilitating both academic research and industrial applications.

7.2.3 Property Instrumentation

In this section, we discuss a set of common specification issues in statecharts, and present our approach to instrumenting the program to detect them using static analysis. The underlying analysis technique is symbolic execution, as discussed above. The way we harness symbolic execution to detect specification issues is to instrument the generated code in such a way that detection of the issues within the bounds of analysis will be intercepted and flagged using a counter-example trace. We use the following commands to instrument the code. The description below shows the how the symbolic execution engine responds to the commands:

1. `error`: The symbolic execution can be configured to either simply stop execution along the path where the `error` instruction is encountered, while continuing along other paths, or to abort execution and present the counter-example.

2. `assert(`p`)`: Invokes the SMT-solver to check if the predicate p is *satisfiable*.

3. `abortOn(`p`)`: Invokes the SMT-solver to check if the predicate p is *satisfiable*. If yes, the execution aborts, and the counter-example trace is presented.

4. `nassert(`p`)`: Invokes the SMT-solver to check if the predicate p is *unsatisfiable*. If yes, the execution aborts, and the counter-example trace is presented.

7.2.3.1 Undefined Variable Access

Consider the statechart fragment shown in Figure FC7.5. Suppose that A and B are variables defined in a global scope. Here, we are interested to know if any expression can be computed using variables that are not defined. Note that globals A and B are undefined in the beginning. Their values are accessed in state S. There

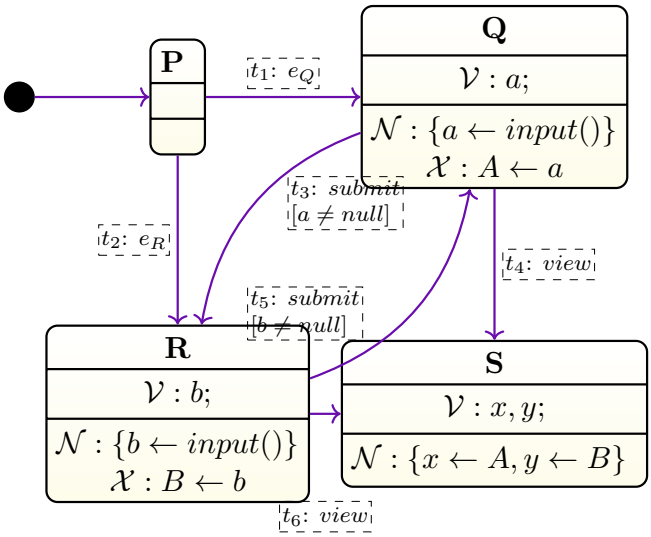


Figure FC7.5: Finding undefined variables

```
A, B, a, b, x, y
state ← P
A_d, B_d, a_d, b_d, x_d, y_d ← false
wait(event) {
  match state with
  P ⇒
    if(event = e_Q) a ← input(), a_d ← true , state ← Q
    if(event = e_R) b ← input(), b_d ← true , state ← R
  Q ⇒
    if(event = submit)
      abortOn(¬a_d) , if(a ≠ null)
        abortOn(¬a_d) , A ← a, A_d ← true
          b ← input(), b_d ← true , a_d ← false , state ← R
    if(event = view)
      abortOn(¬a_d) , A ← a, A_d ← true ,
      abortOn(¬A_d) , x ← A, x_d ← true ,
      abortOn(¬B_d) , y ← B, y_d ← true , a_d ← false ,state ← S
  R ⇒
    if(event = submit)
      abortOn(¬b_d) , if(b ≠ null)
        abortOn(¬b_d) , B ← b, B_d ← true ,
          a ← input(), a_d ← true , b_d ← false , state ← Q
    if(event = view)
      abortOn(¬b_d) , B ← b, B_d ← true ,
      abortOn(¬A_d) , x ← A, A_d ← true ,
      abortOn(¬B_d) , y ← B, y_d ← true , b_d ← false ,state ← S
}
```

Figure FC7.6: Pseudo-code for the statechart in Figure FC7.5

are several ways to reach state S from the initial state P: $PQRS$, $PRQS$, PQS, and PRS. Out of these, if the paths $PQRS$ or $PRQS$ are taken, then at the time A and B are used in S, they both would already have been defined. However, if PQS or PRS are taken, then B and A would be left undefined, respectively. Reaching definition analysis [56] can be used to detect undefined variables. However, in certain cases, it tends to give results which are too conservative. Def-Use-paths thus marked out may not be feasible paths in the program. We use symbolic execution to reveal paths to uses of variables when they are yet to be defined. We apply the following set of strategies:

1. For every variable v, we define an auxiliary boolean variable v_d. v_d is initialised to false to indicate that v has not been defined.

2. For each decision D on variable *event* that corresponds to transition t, $\forall v \in \sigma_{RO}(t), v_d \leftarrow false$ is executed immediately after action $t.a$. This is to preserve the local scope of variables(i.e, semantically variables get undefined outside its scope).

3. For each instruction I such that $v = lhs(I)$, $v_d \leftarrow true$ is executed immediately following I.

4. For every instruction I such that $\forall v \in rhs(I)$ and for every decision D such that $\forall v \in D$, the symbolic execution checks if $\neg v_d = true$. If yes, execution aborts and displays the counter-example trace.

As an example, see the code in Figure FC7.6 which shows the generated code for the statechart in Figure FC7.5. The checks done by the symbolic execution are shown as asserts and the mutations of the auxilliary variables have been shown as assignments, both shown in grey highlight.

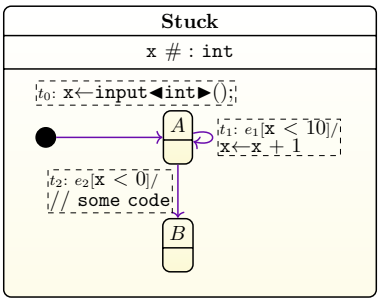


Figure FC7.7: Example for conflicting transitions and stuck specification

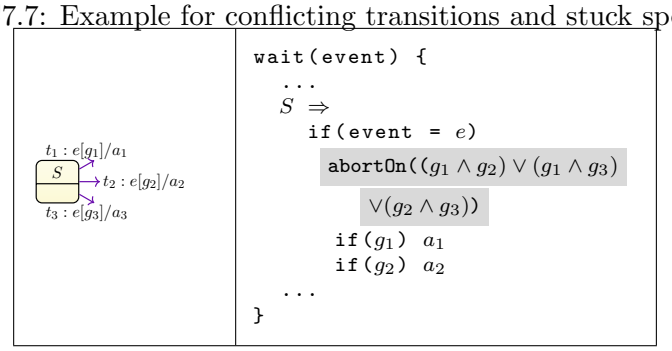


Figure FC7.8: Conflicting transitions

7.2.3.2 Conflicting transitions

Statechart permits conflicting transitions. When multiple outgoing transitions from the current state are enabled simultaneously, the system must make a non-deterministic choice between them. Different flavours of Statechart resolve conflicting transitions in different ways [57, 58].

Whether conflicting transitions are desired in the specification or not depends on the specific case. Figure FC7.8 shows a state of a flattened statechart. It has three outgoing transitions t_1, t_2 and t_3 all triggered by the same event e, and guarded by g_1, g_2 and g_3 respectively. The instrumentation needed to detect conflicting transitions is: *insert an* `abortOn` *in the matching case of all states with more than one outgoing transitions triggered with the same event, to check if the guards of any two outgoing transitions can becoming simultaneously true.* For example, if there are n outgoing transitions with guards g_1, g_2, ..., g_n, each triggered by same event e, we need to check any pair in this set evaluates to true

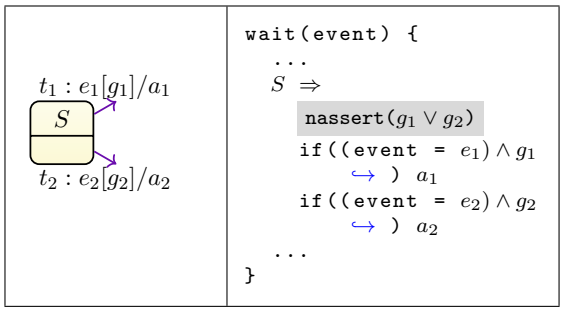


Figure FC7.9: Stuck specification

simultaneously given by:

$$\bigvee_{i,j \in \{1...n\},\ i \neq j} (g_i \wedge g_j)$$

The code corresponding to this piece of statechart is also shown in Figure FC7.8. The `abortOn` statement inserted within the matching case for the state s and event e checks if both g_1 and g_2 are simultaneously true. This assertion evaluating to true is evidence that there are conflicting transitions in the specification. Since this guarantee holds only within the bounds of exploration, not finding a true valuation of the assertion is not a proof of the absence of conflicting transitions. As an example, consider the chart in Figure FC7.9. The variable `x` is given a symbolic value X_1 on t_0. In state A, we do `abortOn`$(X_1 < 0 \wedge X_1 < 10)$. The SMT solver finds this satisfiable proving the presence of conflict.

7.2.3.3 Stuck Specification

In certain cases, once the system enters a particular state, there may not be any way for it to exit that state. This may happen if there is no outgoing transition from a given state. This can be detected by a structural inspection of the statechart. Such cases may well be intended. However, even when there are outgoing transitions from a state, the machine can get stuck in that state. Such cases are likely to be unintended. Consider the simple Statechart fragment in Figure FC7.9. If `x` ≥ 10 on t_0, there is no way the chart can then get out of state

A.

Our approach is to check for reachability of such a state where none of the outgoing guards from a state (in the flattened statechart) is satisfiable. Here, we define the command `nassert`(p): if the given predicate p is detected to be unsatisfiable by the SMT-solver, the symbolic execution aborts and the counter-example is presented. Otherwise, execution proceeds as usual. Consider a state with n outgoing transitions having guards g_1, g_2, ..., g_n. For this state, $p = g_1 \vee g_2 \vee ... \vee g_n$. Detection of unsatisfiability of p is a sure evidence of a stuck state. However, due to bounded depth of analysis, if we encounter no unsatisfiability, it does not guarantee absence of stuckness beyond the bounds of symbolic execution. The abstract program shown in Figure FC7.4 is instrumented with properties at appropriate locations.

7.2.4 Translation to C/Java

When translating this program to C/Java, we take the following steps to make it amenable to symbolic execution:

- A **makeSymbolic** routine is created, that creates a symbolic variable for each *input* variable(Please note: As our case study was derived from stateflow, it had clear set of input variables, so they were made symbolic. Otherwise, this step has to instrument the program by replacing all statements of type $var \leftarrow input()$, $var \leftarrow symvar$ where *symvar* is a symbolic variable created by either *klee_make_symbolic* function in C or *Debug.makeSymbolicX* method in Java (in *makeSymbolicX*, X is the datatype of the variable). The routine **makeSymbolic** is executed prior to all other functions.
- The variable *event* as it is input, is made symbolic. However, it has to be cleared for every iteration of *wait*, otherwise it will evaluate to the same

```
wait(event) {
  match state with
  ...
  s ⇒
  abortOn((g1 ∧ g2) ∨ (g1 ∧ g3)
          ∨(g2 ∧ g3)) //conflicting transitions
  nassert(g1 ∨ g2) //stuck-specification

    if(event = t.e)
      if(t.g)
        //undefined variable access - only for non symbolic variables
        ∀v ∈ V, v_d ← false
        foreach stmt in t.a
            ∀u ∈ lhs(stmt), u_d ← false  ,
            ∀w ∈ rhs(stmt) ,
                if w = input(), v_d ← true
                else abortOn(¬w_d)
            stmt;
            u_d ← true
        state ← t.d
  ...
}
```

Figure FC7.10: Program with instrumentation of properties

symbolic value. So instead we create an array of events - all of them are symbolic to be taken as input. N is the size of the array and it can be changed as required.

The representation of translated code is shown in Figure FC7.11

7.3 Example

We applied our technique to verify the subsystems from the Automotive dataset [1,54], all of which are specified in Stateflow [41]. Here, we focus on the Emergency Vehicle Avoidance (EVA) subsystem from this dataset.

The EVA function assists drivers by guiding the vehicle to a safe stop when an emergency vehicle using a siren requires the road to be cleared. It integrates long and short-range radars, a sound detector, and a GPS device to determine the appropriate time and location for the vehicle to pull over.

Upon detecting a siren, the vehicle initiates the following steps:

1. Deceleration: The vehicle begins to slow down smoothly.
2. Lane Shift: It cautiously manoeuvres towards the right-hand side of the road, ensuring safety and abiding by traffic regulations.
3. Stopping: If conditions permit, the vehicle comes to a complete stop, allowing the emergency vehicle to pass.
4. Coasting: In situations where stopping immediately would be unsafe, such as within an intersection, the vehicle coasts until it reaches a safe location to complete the stopping procedure.

```
#include <klee/klee.h>
#include <stdio.h>
#include<stdbool.h>
#include<assert.h>

void makeSymbolic(){
     ∀t ∈ T, stmt ∈ t.a
     if rhs(stmt) = input()
          ∀v ∈ lhs(stmt)
               klee_make_symbolic(&v, sizeof v, "v.name");
               //make the variable symbolic
}

void run(int events[], int N){
     for(int i=0;i<N;i++) {
       event=events[i];
       match state with
       ...
       s ⇒
       abortOn((g1&&g2)||(g1&&g3)||(g2&&g3)) //conflicting transitions
       nassert(g1||g2) //stuck-specification

         if(event = t.e)
           if(t.g)
                  stmt; //assert are carried over from program
                  state ← t.d
       ...
     }
}
int main () {
     int N = 5; int events[N];
     klee_make_symbolic(events, sizeof events, "events");
     makeSymbolic();
     run(events, N);
     return 0;
}
```

Figure FC7.11: Program in C with instrumentation of properties

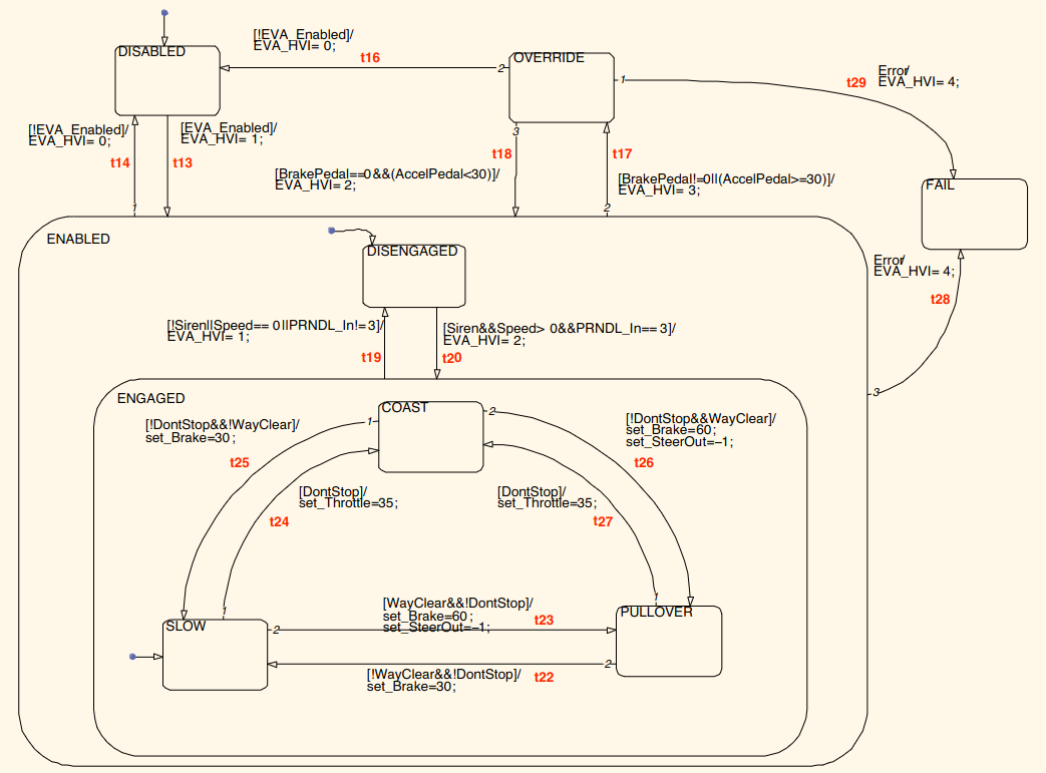


Figure FC7.12: Emergency Vehicle Avoidance subsystem in Stateflow from UWFMS dataset [1]

The EVA function prioritises safety throughout the process, ensuring that the vehicle responds appropriately to emergency situations while minimising risks to the driver, passengers, and other road users.

The statechart model of the EVA subsystem has 7 atomic states, 2 hierarchical states, and 15 transitions, as shown in Figure FC7.12.

It has input variables:

- *EVA_Enabled*: Specifies whether the subsystem should be enabled (input given by the driver via a switch).
- *Siren*: An input from an external component to alert to an approaching emergency vehicle (set to 0 or 1 based on sensors).
- *WayClear*: Denotes whether the vehicle can be pulled to the right lane (0

or 1).

- *DontStop*: Set to 1 if braking would cause the vehicle to reach an unsafe position.
- *BrakePedal* and *AccelPedal*: Inputs from the depression of the respective pedals in the vehicle (0 to 100 percent, with 0 indicating no depression and positive values indicating depression).
- $PRNDL\_IN$: Denotes the current gear selection (0: Park, 1: Reverse, 2: Neutral, 3: Drive, 4: Low).
- *Speed*: kilometer per hour (range 0 to 100)

It also has output variables:

- *set_Steerout*: Controls steering (-1 for turning right, 0 for center, and 1 for turning left).
- *set_Throttle* and *set_Brake*: Control throttle and brake, respectively (both in percentage).
- EVA_HVI: Provides output information to the driver (0: disabled, 1: enabled, waiting for detection of emergency vehicle, 2: engaged, executing action to move the vehicle out of the lane of the emergency vehicle, 3: overridden, 4: error, to be restarted).

7.3.1 Conversion to StaBL model

We have converted this model to a StaBL model and localised the variables based on their usage in the subsystem as shown in Table TC7.1. When modelling this subsystem in isolation, the variables *DontStop* and *WayClear* are local to the

Table TC7.1: Globalisation in StaBL model of EVA

Variable (v)	Storage Class	Globalised Variable (G(v))	exit action instrumentation	entry action instrumentation
EmergencyVehicleAvoidance				
$EVA\_Enabled$	local	$EVA\_Enabled$	clear(G(v))	$v \leftarrow input()$
EVA_HVI	local	EVA_HVI	clear(G(v))	-
AccelPedal	static	$AccelPedal$	-	$v \leftarrow input()$
BrakePedal	static	$BrakePedal$	-	$v \leftarrow input()$
Enabled				
Speed	static	$Enabled\_Speed$	-	computed based on AccelPedal and BrakePedal
$PRNDL\_In$	static	$Enabled\_PRNDL_In$	-	$v \leftarrow input()$
Siren	static	$Enabled\_Siren$	-	$v \leftarrow input()$
Engaged				
Set_Brake	local	$Enabled\_Engaged\_Set\_Brake$	clear(G(v))	-
$Set\_Throttle$	local	$Enabled\_Engaged\_Set\_Throttle$	clear(G(v))	-
$Set\_SteerOut$	local	$Enabled\_Engaged\_Set\_SteerOut$	clear(G(v))	-
WayClear	static	$Enabled\_Engaged\_WayClear$	clear(G(v))	$v \leftarrow input()$
DontStop	static	$Enabled\_Engaged\_DontStop$	clear(G(v))	$v \leftarrow input()$

Engaged State (i.e., they are not used in any of its super states). Therefore, we have scoped these variables within that specific state. The variables that are critical to maintain values across all subsystems are made *static* - like *Speed, Siren,* etc.

As we are currently considering only this subsystem, they are scoped under respective states based on their usage. However, this might change when the subsystems are combined. This example is included for demonstration purposes only. In reality, some of these variables may use the *param* storage class when they are used for data passing.

7.3.2 Translation

7.3.2.1 Globalisation

Table TC7.1 shows the variable globalisation stage, where the variables are uniquely named after the states that they are contained in, and their values are cleared based on their scoping.

7.3.2.2 Flattening

During the flattening phase, the hierarchy is purged. **Enabled** and **Engaged** states are purged, resulting in the creation of 7 atomic states (their names are globalised). Transitions are also flattened, generating new transitions for all states that were previously enclosed within composite states. Transitions to/from the composite states are incorporated into the flattened atomic states. The entry and exit action are moved to the transition action with sequential composition.

In this case study, the unflattened statechart had 10 states and 15 transitions, while the flattened statechart has 7 atomic states and 26 transitions. The the root state of the entire model, which, despite being of type statechart, is retained in the flattened model. The entry action can be prepended before the *wait(event)*, in the program. As the root state does not have any incoming or outgoing transition, it does not affect the approach.

The Table TC7.2 outlines the flattened states and transitions as per the translation steps X_{T1}, X_{T2}, X_{T3}, X_{S1} and X_{S2} described in section 7.2.1.

7.3.2.3 Property generation

The properties corresponding to stuck specification, conflicting transitions or undefined variable access are generated.

For instance, for the state $SLOW$, the property generated are:

- Conflicting transitions - The outgoing transitions are $\{t14, t18, t23, t24, t28\}$. Conjunction of guards each pair of these transitions is evaluated. If any of these is true then conflicting transitions exists

Table TC7.2: Flattening of EVA

State	Newly added Transitions
Disabled(D)	t13 to each of the state *Disengaged*, *Enabled_Engaged_Coast*, *Enabled_Engaged_Slow* and *Enabled_Engaged_PullOver*;
Fail(F)	nil
Override(O)	t18 to each of the state *Enabled_Disengaged*, *Enabled_Engaged_Coast*, *Enabled_Engaged_Slow* and *Enabled_Engaged_PullOver*;
Enabled_Engaged_Slow(S)	t19 to *Enabled_Disengaged*; t14 to *Disabled*; t17 to *Override*; t28 to *Fail*
Enabled_Engaged_Coast(C)	t19 to *Enabled_Disengaged*; t14 to *Disabled*; t17 to *Override*; t28 to *Fail*
Enabled_Engaged_PullOver(P)	t19 to *Enabled_Disengaged*; t14 to *Disabled*; t17 to *Override*; t28 to *Fail*
Enabled_Disengaged(DE)	t20 to *Enabled_Engaged_Coast*, *Enabled_Engaged_Slow* and *Enabled_Engaged_PullOver*; t14 to *Disabled*; t17 to *Override*; t28 to *Fail*
State	Retained Transitions
Override(F)	t16 to *Disabled*; t29 to *Fail*
Enabled_Engaged_Slow(S)	t24 to *Enabled_Engaged_Coast*; t23 to *Enabled_Engaged_Pullover*;
Enabled_Engaged_Coast(C)	t25 to *Enabled_Engaged_Slow*; t26 to *Enabled_Engaged_Pullover*;
Enabled_Engaged_PullOver(P)	t27 to *Enabled_Engaged_Coast*; t22 to *Enabled_Engaged_Slow*;

- Stuck specification - $!(t14.g||t18.g||t23.g||t24.g||t28.g)$ - if this evaluates to true then the specification is said to be stuck.

The model does not contain any read access except for the guard conditions, and all the variables in the guard are input variables, so they are symbolic. So the var_d - defined variables and check are not added.

7.3.3 Experimentation

Apart from the localisation of variables, the other major difference between stateflow and StaBL is: that by default, stateflow assigns priorities to outgoing transitions. StaBL, however, does not have such restrictions. Our intention was to detect the absence of conflicting transitions though there are transition priorities. By our translation-based symbolic execution approach in the subsystem **Engaged**

Table TC7.3: Subsystems of Automotive dataset

model	#states	#trans	#vars(#sym)	#fstates	#ftrans	#properties
Emergency Vehicle Avoidance	9	15	12(8#sym)	7	27	5#nd 6#ss
Collision Avoidance	10	23	9(6#sym)	9	44	7#nd 8#ss
Lane Guidance	11	19	9(7#sym)	9	29	7#nd 9#ss
Parking Assistance	14	18	14(10#sym)	13	22	8#nd 10#ss
Reversing Assistance	10	17	9(6#sym)	9	33	6#nd 7#ss
#nd - Conflicting transitions, #ss - stuck specification						

of the Emergency Vehicle Avoidance (EVA) case study, we could find that some of the priorities were not necessary as their guard can not result in conflict (but the guarantee is only within the exploration bounds). For instance, the outgoing transitions from the states *Coast*(t25,t26), *Pullover*(t22,t27) and *Slow*(t23, t24) - cannot become true at the same time, so priorities are not needed there at the same level though defined. It is enough to prioritise the transitions when they may lead to conflict and symbolic execution will be able to ascertain this. However, there can be conflicts flagged when outgoing transitions of engaged are also added to these states while flattening.

Experiments were conducted within a Ubuntu virtual machine with 16GB RAM. Their purpose was to ensure the model's correctness and relevance to various subsystems. Key findings are as follows:

- Stuck States: The model exhibits the potential for stuck states. For instance, if $EVA\_Enabled$ fails to reach a value of 1. This scenario was true for the $X\_Enabled$ variables in all subsystems.

- Transition Priority: The flattening process applied to the Stateflow model in Figure FC7.12 introduced additional outgoing transitions to the states **Slow, Coast**, and **Pullover**. Stateflow assigns default priorities to transitions, governed by various rules outlined in its operational semantics [25] including higher priority for guarded transitions, "12 o'clock rule" for ordering transitions based on their graphical positioning. Our translation process prioritises the structural elements of the statechart over graphical transition positioning. However, StaBL models inherently lack explicit transition priority mechanisms. To effectively identify potential conflicts arising from this disparity, we employ property instrumentation techniques. This approach successfully detected non-deterministic behaviour in the flattened model's outgoing transitions from the aforementioned states, directly resulting from the newly added transitions during flattening.

- Effect of localisation: We simplified the model by isolating and testing the **engaged** subsystem exclusively. Following translation and symbolic execution, no instances of conflicting transitions were detected. These findings present potential avenues for future research, particularly regarding the impact of localisation within the analysis process, as elaborated upon in section7.4.

- Symbolic Execution Compatibility: The model demonstrates compatibility with both KLEE and JPF-Symbc symbolic execution engines. The statecharts were converted to C/Java code structures and submitted to these tools for verification. Notably, both tools were able to identify certain stuck specifications and conflicting transitions as intended.

7.4 Future work

Here, we outline potential aspects to consider as future work to optimise the symbolic execution of statecharts with local variables.

7.4.1 Localisation of analysis

Statecharts can be analysed using a variety of approaches. Common methods involve either flattening the statechart or working directly with its unflattened version. Alternatively, a more nuanced approach leverages the principle of localisation to isolate and execute specific subsystems. Statecharts, by their inherent hierarchical nature, enable the clear separation of subsystems, facilitating their independent functional assessment. With the presence of local variables, statecharts become more modular, thereby contributing to the localisation of analysis. This modular approach allows for a structured assessment process, beginning with broader subsystems and progressively delving into finer levels of detail, or vice versa.

A model can be divided to sub-models representing subsystems: (1) when they access only the variables accessed exclusively within that particular subsystem. (2) these subsystems should contain transitions and interactions that solely occur within themselves, without connecting or interacting with different levels of the actual model. In other words, transitions are contained within the subsystem without any inter-level connections to other states in the actual statechart model.

When variables and interactions remain confined within the boundaries of the subsystem, it becomes feasible to limit the analysis strictly within that subsystem(i.e, the subsystems do not access any variables from its ancestor states, however, if they access a variable of an ancestor state - the inferences are different,

which is omitted from the scope of discussion now). While the primary goal of subsystem isolation is to streamline analysis by reducing the state space, it also proves valuable in assessing the functionality of individual components. By isolating a subsystem, analysts can zero in on its unique characteristics and performance, free from the interference or influence of external factors. However, this localised verification approach can also present its own trade-offs:

1. Conflicting transitions - The absence of conflicts within a subsystem's states does not guarantee its absence when those states are encapsulated within a larger model. This is because conflicts can arise between the guards of outgoing transitions from the encapsulating state and the previously deterministic states, even though localised analysis deemed those states deterministic in nature.

   Example 30. *Consider the transitions in Figure FC7.13, if the subsystem C is isolated and active state is A and the event e is triggered, if $g1 \wedge g2$ is not true in the subsystem, then there are no conflicts from state A in subsystem C. But this may not be the case when it is embedded within the system D. Though, $g1 \wedge g2$ is not true, either $g1 \wedge g3$ or $g1 \wedge g2$ may be true, which may lead to conflicts in a non-prioritised transition semantics (like StaBL). Thus, the absence of conflicting transitions in subsystem C does not guarantee its absence in subsystem D.*

2. Stuck specification - A state determined to not be stuck during localised analysis will maintain its non-stuck status even when encapsulated within another statechart. This is due to the localised model's exclusive operation on localised variables and transitions. The scoping policy prevents the encapsulating state from modifying these variables, ensuring that a state's non-stuck nature in localised analysis persists within larger statechart encapsulations.

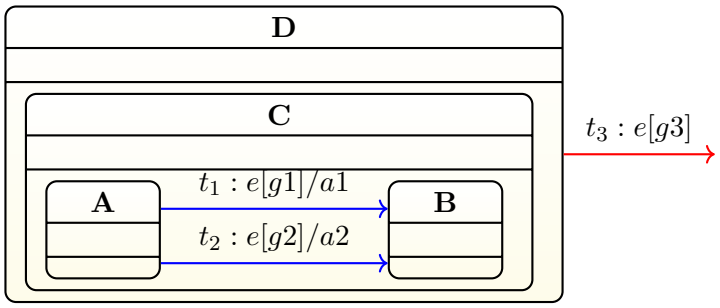


Figure FC7.13: Effect of localisation in statecharts

Example 31. *In Figure FC7.13, if the subsystem C is isolated and let us consider the state A is active, now if event e occurs, then the state is set to be not stuck if either of $g1$ and $g2$ will evaluate to true. So, if it was found that the subsystem cannot be stuck at A in subsystem, even if it is encapsulated within a system D, the property will be maintained. The guard of transition $t3$ being true or false, has no effect on the subsystem, when it is not stuck. However, if A is stuck in C, and it is encapsulated within D, then $t3$ can remove the stuckness of $g3$ will evaluate to true.*

3. Undefined variable access - The absence or presence of undefined variable access within a sub-model remains consistent when encapsulated within any other state that does not have undefined variable access issue.

   Example 32. *In Figure FC7.13, if the action code of the states, C, A or B has a statement with read access to undefined variable that is local to these states. It will persist when it is encapsulated within D as well, this is because - the scope rules of D restricts access of variables contained in C, A and B.*

These cases offer a glimpse of the benefits of localisation and hint at potential future directions. While the scenarios and examples presented here are not exhaustive, they can be expanded further, particularly when considering inter-component interaction and the context of local variables.

7.5 Summary

In summary, we provide a semantic-preserving translation algorithm that is amenable to symbolic execution without compromising expressiveness. We have demonstrated how StaBL specifications can be formally verified using symbolic execution via this automated translation approach. This has displayed promising potential for employing symbolic execution within the context of statecharts. We conducted tests of this approach using an automotive dataset [1, 54].

In future extensions of this work, we aim to employ symbolic execution directly on the statechart itself, rather than relying on a translated version.

CHAPTER 8

RELATED WORKS

The existing surveys [4–6,17] focus on the new features, new semantics, or their suitability to model checking and formal verification [35], but do not detail in how they differ on the usage of variables, data passing, action blocks, or variable-related optimisations; this is the core of our work.

8.1 Scope of the related works

For the sake of brevity, we focus only on existing works that discuss variables and action blocks that alter these variables in hierarchical statechart variants. Therefore, we omit those works that don't address state hierarchy altogether.

We critically review existing works through the lens of well-defined and distinct conceptual categories, ultimately highlighting how our work differs from them. In particular, we focus on the presence of variables: scopes, storage classes, action execution, concurrency, and transition priorities. We have excluded publications that contain minimal or no references to variables.

Our semantics processes events one at a time, ensuring that no other event is processed until the current event processing phase has finished. This sequential approach is one of the most widely used techniques for event processing, as

there exist multiple semantic variants, including instantaneous, parallel, and sequential approaches [43]). Therefore, we do not consider this sequential nature a distinguishing factor for comparison purposes.

This related work is divided into two main categories: language and model verification approaches.

8.2 Language

8.2.1 Usage of variables - scopes and storage classes

The variables in most statechart variants have either a *global* or *local* scope. Variables are in *global* scope when they can be accessed throughout the statechart model. They are in *local* scope when their access is restricted to certain parts of the statechart. Some variants also introduce additional scopes, such as *internal, external, input*, etc. however, they were majorly in *global* scope. In StaBL [38], three storage classes - local, static and param that are in *local* scope were introduced.

The works by Harel [2, 59] inclusive of STATEMATE [58, 60–62] consider the variables to be in global scope and mentions that shared variables can cause conflict in case of concurrent states. One of the earliest surveys on statechart by von deer beek [4], around 21 variants of statecharts have been surveyed. These variants either use global variables or no variable at all.

RSML [63],a statechart-based requirement specification language, defines subsystems as component state machines with *input* and *output* variables in *global* scope. In both Stateflow [25, 32] and Rhapsody [64], variables are part of the action language rather than the statechart model itself. This action language can

be any imperative programming language supported by the formalisms. Stateflow is a variant of statecharts used with MATLAB Simulink, while Rhapsody is a tool that implemented the first executable semantics for object-oriented statecharts, which has been widely adapted in UML. Although Stateflow does have input/output and data variables, their visibility and modifiability span the entire statechart, unlike the localised approach in StaBL semantics [38].

SCXML [65], a statechart XML notation, has provisions for declaring variables in both global and local scopes. Datamodels are used to declare variables (either common or within a state); however, these variables are visible and modifiable throughout the entire statechart. This data model leverages scoping to enable binding of values to variables, with the flexibility to employ either lazy (when the state becomes active) or early (at the beginning) binding approaches. It also has provisions for declaring variables within action blocks, thus providing a local scope within each block. Additionally, it has parameters for sending data to external components or services. However, the variables in data models are in global scope.

Circus [66] is a language that combines the stateflow [57] with Z and CSP, it has provision to declare local variables within a Z schema; the semantics on accessing local variables and visibility of the variables in parallel states are well discussed in this work - like resolving conflicting local variable names through namespaces; a generic approach in any programming language. The schemas are encapsulated inside a process and multiple processes communicate via channels. But the local variables in Z state schema is only local to the action statements of the schema [38].

There are multiple works on UML semantics [28,67]; Some of them handle variables but do not explain the intricacies of data passing during inter-level transitions and history. Others leave out aspects of action language and history constructs

due to their complexity [38].

Visualstate [68] distinguishes between external and internal variables, reflecting a two-layered data management approach. External variables, accessible by user routines, facilitate interaction and statechart control. In contrast, internal variables, confined to the system space, prioritise data integrity and operational stability within the statechart's core engine. This dichotomy caters to diverse needs, enabling flexible user interaction while safeguarding internal processes. However, it is worth noting that all variables in Visualstate are global in scope.

In a distinct work, Exelmans in 2020 [69], explores the use of input/output variables within the context of big-step semantic languages (BSML) specifically tailored for statecharts. Statecharts in this framework can feature input variables (read during event reactions), output variables (updated regularly but read by the environment), and interface variables for communication between components. Crucially, this work also defines a hierarchy of scopes governing variable access, including action scope, guard scope, transition scope, instance scope, and built-in scope. This work seems to have evolved concurrently with the work on StaBL [38, 39] as both were published in 2020. However, this work does not scope the variables based on the statechart hierarchical structure and StaBL leverages the power of hierarchical structure of statechart to implement localisation of variables and incorporate structured dataflow.

In these approaches, though there are various hierarchical statechart variants - the variable access methodology can be broadly classified to be: (1)declared global; accessed globally [2, 58–62] (2) global; scoped as per action language semantics [25,32,65,66,69] (3) accessible from outside the system for interfacing with external components [41, 65, 69] (4) local - action scope, guard scope, transition scope, instance scope, built-in scope [65, 69]

The works that consider local variables consider action language as a distinct entity from the statechart itself or scopes the variables within action blocks unaffected by the statechart structure. In contrast to these works the StaBL semantics, which tightly couples variables with states, ensuring that they are not directly accessible or modifiable from anywhere in the action blocks, that does not have access scope to these variables. This design consideration has lead to improved modularity in statecharts and a *scopescore* metric3.4 has been proposed that shows these statecharts modelled in StaBL are more modular than traditional statechart models.

Unlike Harel's statecharts with history pseudostates, StaBL represents history through a marker variable within a state that specifies whether history needs to be maintained for that state. Based on this, the value history is maintained. We have defined our semantics for shallow history, and can replicate deep history in a state by setting the history marker to true in all its descendants.

While most statechart variants, delve into the concept of history and offer mechanisms for clearing it, their focus is primarily on state history. This means they track the previous states a system has been in, enabling a return to those states under certain conditions. Our approach, however, distinguishes itself by extending the concept of history to encompass variables and their values. The notion of value history in StaBL arises primarily due to local variables and their scopes. For variables in the global scope, they must always be accessible, so the history of a single state does not impact them. However, this is not the case with local variables. Their valuations need to be stored and retrieved, necessitating the maintenance of value history. This unique approach is further enhanced by our differential treatment of variables based on their storage classes, ensuring a fine-grained and adaptable approach to value history management.

8.2.2 Action

In statechart semantics, a "step" denotes a fundamental unit of execution, encapsulating the transition from one configuration to another in response to an event [70]. A step comprises executing set of actions. The action execution can be either simultaneous or sequential in order with valuation from previous step or sequential in order with valuation from current step [71, 72] based on step semantics.

For instance, in the *action* : $\langle x \leftarrow x+1, x \leftarrow x+2 \rangle$, let us assume the value of x was v_0 before execution of the action block, and variables are updated on the semantics chosen:

- The execution can be sequential, where $x \leftarrow x+1$ and $x \leftarrow x+2$ are executed in the same order, and the valuation of x is taken as per the statements executed in the ordering. So the value of x will be $v_0 + 3$.
- The execution is sequential and the ordering is the same, but for executing each statement, the value of x is considered as it was at the beginning of the code execution. Thus, at the end of the step, x is $v_0 + 2$.
- The execution is simultaneous; it can execute in any order, in which case x can be either $v_0 + 1$ or $v_0 + 2$.

All these action execution approaches have valid semantics within different statechart variants. As observed, each approach has a distinct impact on statechart execution, as the evaluation of variables yields varying results for the same action block depending on the chosen semantics.

Pnueli's [70] work briefly addresses variables and reinforces the definition of consistency within action code. It deems an action block inconsistent if any two

atomic actions within it involve assignments to the same variable. This strict constraint stems from the intention to execute all actions within a step simultaneously, thus precluding potential conflicts arising from concurrent writes or reads to the same variable.

Harel [2] introduced a classification of actions in statecharts into two distinct categories: actions and activities.

- Activities: Actions that unfold over a non-zero time period, such as beeping, displaying information, or performing complex computations.
- Actions: Immediate actions that occur instantaneously.

Each activity is associated with two special actions, "start(X)" and "stop(X)," where X denotes the activity itself. While this comprehensive report outlined various features of statecharts as a visual formalism, it did not delve into the details of variable modifications within actions.

Harel's subsequent work on formal semantics [59] proposed a simultaneous semantics for actions. Within this framework, all statements within an action block are considered atomic, meaning they execute as a single, indivisible unit. If two statements, a_i and a_j, attempt to modify the same variable, this is flagged as an inconsistency. STATEMATE semantics [42] align with this approach, defining both actions and activities and adopting the simultaneous execution and inconsistency detection principles.

Huizing [31] proposes future work on incorporating variables within their statechart model, where variables would be shared among orthogonal components. However, to prevent conflicts, their proposal would disallow simultaneous writes to the same variable within a single step, a restriction necessitated by the model's simultaneous execution behaviour.

In STATEMATE the action language as the statements in action block are executed in parallel/simultaneously; Whereas in UML [28, 67], SCXML [65] and Rhapsody's [73] action language is sequential [24].

Action language holds a prominent role in the semantics of most statechart variants. It may be *state entry*, *state exit*, *state do*, *transition* actions, etc. Also, certain works speak about other features like timed actions [2, 42], some of them restrict usage of action only to generate events [31, 61]. StaBL and ConStaBL semantics currently supports *state entry*, *state exit* and *transition* actions. They follow action execution in sequential order with valuation from current step.

Our statechart variant adheres to sequential execution in a consistent order, ensuring that actions always consider the most recent variable valuations during processing. Additionally, we introduce variables with differentiated scopes, which offers a distinct advantage: it controls the accessibility of variables within action blocks, restricting unintended modifications and enhancing overall model modularity. Also even in case of concurrent composition of action code, ConStaBL follows interleaved fashion of execution (more in section 8.2.4 .

However, our current semantics exclude do actions, timed actions, and the ability to raise events within actions. Exploring the effects of these features on localised variables presents a promising avenue for future research.

8.2.3 Transition priorities

In statecharts, a state can possess multiple outgoing transitions, introducing the potential for conflicts during event processing. When an event occurs, all transitions emanating from the current configuration are evaluated to determine the set of enabled transitions. A transition is considered enabled if its event ($t.e$) matches the incoming event and its guard condition ($t.g$) evaluates to true within

the current configuration and value environment.

If multiple transitions are enabled, priorities dictate which transition will be executed, resolving conflicts. Transition prioritisation approaches in existing literature can be categorised as below:

1. No priorities:
    - No explicit priorities are defined between transitions
    - During simulation when multiple enabled transitions are detected, this approach either flags the presence of conflicting transitions or chooses a transition arbitrarily.
2. Explicit priorities:
    - Priorities are explicitly defined during model construction for each state's outgoing transitions.
    - The enabled transition with the highest priority is executed.
3. Priorities based on hierarchy: When multiple transitions are enabled across different hierarchy levels,
    - Inner-first: the transition originating from the innermost state within the current configuration is prioritised.
    - Outer-first: The enabled transition whose source is the outermost state in the configuration takes precedence.
    - Both *inner-first* and *outer-first* have been combined with either *priority* or *no-priority* schemes in different works.
4. Priorities based on ordering: either the state order, document order or positioning define the priorities for the transitions.

A UML statechart variant [67] prioritises the transition originating from the most deeply nested state. However, flexibility exists for equally deep transitions.

YAKINDU [47] statecharts ensure predictable behaviour through two explicit priority mechanisms:

1. Numbered priorities for outgoing transitions from a state, ensuring a specific firing order.
2. Region firing order based on left-to-right, top-to-bottom layout, guaranteeing region prioritisation.

Visualstate [68] employs a top-down priority mechanism for actions and states. Actions higher in the hierarchy execute first, and conflicting transitions are resolved using a priority function that discards the lower-priority transition.

In SCXML [65], when confronting conflicting transitions (T1 and T2), T1 takes precedence over T2 under two conditions:

1. Innermost Transition Precedence: T1's source state is a descendant of T2's source state within the state hierarchy.
2. Document Order Precedence: T1's source state precedes T2's source state in the document's linear structure.

Sismic [48], an SCXML execution engine with extensive UML feature support, employs a distinct approach:

1. Decreasing Depth Order: Enabled transitions are processed based on the decreasing depth of their source states, aligning with inner-first and source-state semantics. Transitions originating from deeper states are prioritised over those from less nested states.

2. Lexical: In cases of equal depth, the lexicographic order (alphabetical sequence) of the source state names determines priority.

3. Conflicting transitions Flagging: Sismic prefers to signal conflicts rather than strictly enforcing priority-based resolution.

Stateflow [25, 32] defines transition priorities based on two distinct rules:

1. Guarded Transition Priority: Transitions with guards (conditions that must be met for execution) have higher priority than unguarded transitions.

2. 12'O Clock Rule: Transitions are evaluated clockwise from the source state's position in its graphical representation.

Inter-level transitions, forks, merges, and transitions between orthogonal regions pose challenges for statechart semantics as it becomes crucial to define priorities in these cases. Compositional semantics [30] disallow them altogether, while traditional statechart formalism by Harel [2] permits them but emphasizes the need for precise semantic definitions.

In our work, no priorities are defined for transitions as we felt it will restrict the formalism. However, we provided methods to detect conflicting transitions by simulation and verification.

8.2.4 Concurrency

Orthogonal states within statecharts offer diverse execution schemes, catering to various modelling needs:

1. Ordering: The substates of an orthogonal state follow a predetermined order specified during modelling.

2. Arbitrary: No ordering undefined, allowing for flexible execution paths.

3. Parallel: Irrespective of the ordering scheme, substates execute concurrently, enabling independent processing streams.

4. Interleaving: No ordering defined, the sequences of substates are interleaved within an event processing step, fostering a fine-grained interweaving of their behaviours.

Variants like VisualState [68] and Yakindu [47] employ a total ordering mechanism for orthogonal states, governed by a prioritisation function. The highest priority value (1) is assigned to the most critical state, ensuring a consistent and predictable execution flow within the statechart. This prioritisation approach is prevalent across numerous statechart variants.

Our statechart variant, ConStaBL[1], embraces the interleaving scheme, embracing arbitrary interleaving of concurrently executing action code. We believe that, this design choice empowers more precise system modelling and facilitates enhanced upstream analysis. Notably, ConStaBL does not define ordering between states contained within a shell state (our term for semantically distinct AND/orthogonal states). Furthermore, we establish criteria based on our semantics for defining conflicting transitions and valid simulations, and we extend the work on modular statecharts by presenting updated operational semantics that incorporate concurrency.

As research on the execution semantics of statecharts progressed [3] [64] [74] [75] [47] [71], the topic of concurrency-related semantics became significant. Two major divisions of statechart semantics, namely interleaving and true concurrency [4],

[1]The content of this chapter is based on: Venkatesan, Karthika, and Sujit Kumar Chakrabarti. "ConStaBL–A Fresh Look at Software Engineering with State Machines." arXiv preprint arXiv:2307.03790 (2023), doi:https://doi.org/10.48550/arXiv.2307.03790 and "A fresh look on semantics of Concurrent State Based Language (ConStaBL)", Journal of Computer Languages, vol. 79, p. 101270, 2024. doi:https://doi.org/10.1016/j.cola.2024.101270

were predominant. Interleaving semantics use priorities to ensure determinism and resolve data races. Transition priorities determine the execution preference in cases of conflicts, while sequential ordering of substates within an AND composition determines the execution order in the presence of concurrent enabled transitions. This mechanism, though widely used in commercial tools like Yakindu [47], Statemate [3], and Stateflow [41], may limit the analysis of more realistic behaviour in concurrent systems that use multiprocessing architectures. The semantics of UML 2.5.1 mentions that the order in which the enabled transitions in an orthogonal region are executed is left undefined. Whereas, certain variants of statecharts and tools [47] that follow UML semantics, requires the addition of priorities and sequential ordering during design. These design decisions percolate into implementation, as these tools generate sequential code from statecharts (to imperative languages like C, Java, and TypeScript etc.). This may not align with the original intention of designing and analysing concurrent systems. Rhapsody [64], an execution semantics of UML, allows modelling of non-prioritised orthogonal states and acknowledges the undetermined execution order among orthogonal siblings. The arbitrary order of execution is achieved by "locking" each AND state once a transition is triggered within one of its components [76]. Rhapsody insists that tools should permit to design without defining priorities for transitions. It is intractable to statically determine if the guards of two enabled transitions may evaluate to true, so many variants of statecharts insists on defining these priorities. SCXML [65, 77] resolves the execution order in parallel states by its document order. Each substate of a parallel state has a final state. When one of the substates reaches final state, it waits for all other states to reach final state in the subsequent event processing before processing an outgoing transition from the state. This serial processing of events ensures that no race conditions occur. As far as we are aware, none of these semantic approaches provide specific details on how to interleave the action code. The template semantics [34, 78] mentions about variables

but does not scope the variables, they are global. Also, it operates on hierarchical transition systems (HTS)—a variant of statecharts without inherent concurrency. However, it does state that concurrency can be simulated using composition operators. Six such operators are provided: parallel, interleaving, synchronization, sequence, choice and interrupt. However, in the interleaving scheme, only one component can execute a step at a time. Additionally, the model lacks details regarding interlevel transitions for entering or exiting orthogonal states. This limitation arises because the work solely considers concurrent execution of two or more HTS. A factor that necessitates more intricate semantics, as found in ConStaBL.

Our focus is primarily on analysing concurrency behaviour in ConStaBL statecharts with local variables in a priority-agnostic manner. The presence of local variables in our language makes it important to delve into the detailed semantics at the action code level. ConStaBL provide a detailed perspective on how action code within AND compositions is executed in the interleaved manner at the statement level. While our approach may appear close to token based mechanism of Petri nets [79], there is a significant distinction. Each node in the *code graph* is a *control flow graph* on its own. Also, nesting an AND state within another, the composition of action blocks combines sequential and concurrent patterns which needs a special attention. Our methodology formalises this process and enables the early detection and analysis of concurrency-related issues and also allows use of ConStaBL as a rapid prototyping language.

8.3 Model Verification Approaches

8.3.1 Fuzz testing

Fuzz testing applications and protocols have been an area of active research in the last decade. Many fuzzing tools have evolved with focus on security testing [50, 51, 80]. Numerous approaches [49] use fuzzing and model-based testing methods to generate inputs by mutating the models. For instance, Schneider's [81] work creates a classification of fuzzing operators to create invalid message sequences; it provides guidelines on how messages are to be fuzzed, such as by removing messages, adding and modifying them, or changing their behaviour during loops for UML sequence diagram and has claimed to be applicable for message sequence charts as well. Our aim is to test the statechart model itself through fuzzing. This work presents a preliminary setup to carry out fuzz testing of statechart models, an idea that does not have precedent in the literature to the best of our knowledge. ConStaBL and StaBL simulators in conjunction with a well-known fuzzer to do fuzz testing of statechart models of non-trivial sizes and have found issues in them that would have been hard to find through inspection [40]. While various fuzzing techniques, such as grammar-based fuzzing or mutation-based fuzzing, can be employed, our primary objective is to demonstrate the compatibility of the tool-chain we have developed with existing fuzzing engines. For this purpose, we have integrated Jazzer [53]. Our fuzzing approach can also be extended for integration with other fuzzing engines and methodologies, which requires further investigation to determine the most appropriate techniques for the specific property or system under test. This constitutes one of our future research areas[2].

[2]The content of this chapter is based on: Venkatesan, Karthika, and Sujit Kumar Chakrabarti. "ConStaBL–A Fresh Look at Software Engineering with State Machines." arXiv preprint arXiv:2307.03790 (2023), doi:https://doi.org/10.48550/arXiv.2307.03790 and "A fresh look on semantics of Concurrent State Based Language (ConStaBL)", Journal of Computer Languages, vol. 79, p. 101270, 2024. doi:https://doi.org/10.1016/j.cola.2024.101270

8.3.2 Model Verification

Another dimension of our work was on applying verification methods on models written in StaBL language. Majority of prior research have used statechart models in design phase to generate code, test cases and in verification [17]. Our aim is to make sure to leverage such techniques for StaBL as well and make sure, they seamlessly work in presence of local variables. In the existing works, various properties has been verified on statecharts. The statecharts have been either translated or as such taken for verification.

De Oliveira [20] proposed a method to model Hypermedia applications(HMBS) using hierarchical statecharts and emphasised the advantage of using state based models as it enables verification of useful properties like page reachability and valid paths. Statechart's features like hierarchy and parallelism is used for specifying display characteristics, browsing semantics, parallel interactions of hypermedia - action language, data-flow that are prominent in StaBL have not been used in this model. Han [82] presented a FarNav approach which uses transition diagrams without hierarchy to model adaptive, history sensitive navigation and subsequently verified properties like dead end, reachability, access control and other custom properties related to web pages using SMV [83] and CTL [84]. WAVer [85] is a tool to convert a web application design to finite state machines and uses model checking to identify inconsistencies like unreachable pages, violations of access privileges and also verifies application specific properties. These works are similar to ours in principle but differs at technical level as these works are applied in design phase to specify pages and its navigation [38, 39].

There has been substantial amount of work in developing methods for converting code and other software artifacts to models in order to use formal verification techniques. Kubo's [86] is one such effort, where a struts2spin tool that auto-

matically extracts page transitions from the source code to generate a finite state automaton is proposed and SPIN model checker [87] is used to verify properties of a web application. Another work by Choi [88] ensures consistency between page flow diagrams, class/method specifications and behaviour diagrams by modelling them as parallel composition of transition systems in UPPAAL [89] and model checks the navigational behaviour with properties written in CTL [84]. Miao [90] verifies implementation model (the external observable behaviour is represented as kripke structure) by automatically generating the safety and liveness properties like node coverage, edge coverage and edge composition coverage from design model (Object relation diagram). Grumberg's [91] work uses UML statechart to model message passing between objects with event queues and uses BMC method to identify the presence of livelock, but this work does not allow inter level transitions, controlled data-flow and local variables like StaBL. Some more works attempts to find missing/spurious relationships between behavioural models [92], implementation models, test cases etc. Our investigation shows that most of the approaches which uses state based models uses model checking and exploits the capabilities of CTL to verify desired properties. Alpuente [93] provides a rewriting logic approach to specify web applications - this work explicitly specifies the DB interactions, session interactions and server-client interactions for each operation. These implementation and design aspects are out of our scope as we concentrate on abstract formal specifications at the requirements phase. In one of the recent works, TK Das [94] discusses the smells that can occur due to design choices when using UML-RT for embedded systems and re factoring solutions for it, and mentions automatic verification as their future work. We share a common view with this work by providing an automated verification approach to the common bugs that can occur in specifications.

Ours is an attempt to perform verification to investigate all these semantic

features like action language, hierarchy, variable scopes and data passing during verification and provide automated verification. Model checking [35] and program analysis are the commonly used techniques on these models/software artifacts to detect inconsistencies - whereas we found that when using StaBL for specification with features like hierarchy, action code with abstract variables, local scoping and data flow, using symbolic execution for verification adds credibility to the abstract nature of the specification language. Though all other part of the works were building simulator on hierarchical/concurrent statechart itself, for verification we have converted the model to the languages (C/Java) that is compatible to the existing symbolic execution engines - KLEE and JPF. The generic issues for verification - conflicting transitions, stuck specification and undefined variable access have been taken. The methodology proposed is extensible because, the translation is generic and we provide an approach to auto generate the properties that corresponds to these issues and instrument them into the code. So, it is possible to extend it to other properties as well provided, they are amenable to formal verification.

We have also progressed in building model verification engine for ConStaBL, our concurrent variant of StaBL. We have developed of an exhaustive exploration algorithm specifically designed for interleaved execution semantics. While exhaustive exploration has been implemented in many other tools [35, 95, 96], ours is one of the first to implement it natively on a well defined statechart semantics. This algorithm holds strong potential for the field of model verification, as it lays a critical foundation for the following:

- Rigorous Analysis of Interleaved behaviour: The algorithm ensures that all possible execution paths and potential interactions among the action blocks of concurrent states are explored. This capability is crucial for uncovering subtle errors or unintended behaviours that might otherwise go undetected,

particularly within complex systems characterised by concurrency and hierarchy.

- Property Verification: By systematically exploring the entire state space of a statechart model, the algorithm sets the stage for the verification of properties. These properties could encompass safety (ensuring the system never enters undesirable states), liveness (guaranteeing the eventual occurrence of desirable events), or other essential characteristics that must be upheld for the model's correctness and reliability.

In essence, this algorithm serves as an important component in our ongoing endeavour to establish ConStaBL as a powerful platform for model verification, specifically tailored for statecharts with interleaved execution semantics. The development of this algorithm represents a significant step towards the construction of comprehensive verification tools within the ConStaBL framework.

8.4 Summary

This chapter analyses the differences between existing methodologies and works on statecharts concerning local variables. We categorised the works into two main areas: language aspects and verification methods. Within each subsection, we have highlighted how our work stands out in comparison to related works.

CHAPTER 9

CONCLUSION AND FUTURE WORK

9.1 Conclusion

Statecharts is a widely adapted tool for formal analysis, design, and implementation of complex systems by capturing multi-layered abstractions and intricate inter-component interactions. The introduction of local variables into the statechart further enhances the language's flexibility and expressiveness in the modelling process. The specific behaviours and conditions could be modelled within individual states, thereby improving the overall functionality and modularity of the statechart model. This deeper dive into local variables and action language, alongside their interplay with hierarchy, concurrency, and events, revealed the insights on the dynamics of the system and its components.

We proposed two languages—StaBL and ConStaBL—that provides tighter integration of the action language with the traditional statecharts and add rich features of modern programming languages, giving greater power in the hands of the software engineer. The tool chain has been fully developed (for both StaBL and ConStaBL), encompassing a parser, typechecker, and simulator to simulate the statechart models. Furthermore, a simulator has been developed to identify prevalent issues like conflicting transitions, stuck specifications, undefined vari-

able access, and configuration reachability. Similar issues have been identified by translation-based verification. Additionally, within this tool chain, a module for fuzz testing has been implemented to assess the robustness of statechart models and uncover potential vulnerabilities. For concurrent statecharts, exhaustive exploration techniques were applied to analyse all feasible combinations of states and transitions, ensuring a comprehensive understanding of the system's behaviour and identifying any potential issues. Case studies were created to showcase the effectiveness of the language and methods in terms of expressiveness, scalability, completeness, and correctness.

We have built simulators and verification engines for each of our contributions[1]. We carried out experiments using our own models and automotive dataset [54]. We aim to leverage the fine-grained nature of our statechart semantics to detect a wider range of defects in systems, and we have provided an overview of those extensions in the future work section.

9.2 Future Work

Our research is placed in the broader context of developing specification and verification methods using Statecharts. Our variant of statecharts with local variables can be extended in the following directions

9.2.1 Additional Features:

- Event Raising from Action Code: The actions within a statechart can be extended to trigger events directly, and redefine the semantics to detail when these events will be processed.

[1] https://github.com/sujitkc/statechart-verification

- More Types: Introducing a richer set of data types (e.g., real numbers, arrays, custom structures) to model complex systems more accurately and enhance expressiveness.
- Timed Events: Enabling events to occur automatically after specified time intervals, facilitating the modelling of time-dependent behaviour, timeouts, deadlines, and real-time systems.
- Handling Loops: Implementing mechanisms to manage loops within statecharts, preventing infinite execution, and ensuring deterministic behaviour is crucial for avoiding deadlocks and livelocks. The loops can be cause because of non terminating *while* or *for* constructs within the action code block or can be from the cyclic/self transitions between the states in the statechart model itself.

9.2.2 More Verification Methods:

The verification methods can be built on the hierarchical statechart itself.

- Symbolic Execution: Analysing statecharts by executing them with symbolic inputs, enabling the exploration of a wide range of possible execution paths and the detection of potential errors or unintended behaviours.
- Program Analysis: Applying static analysis techniques to statecharts to uncover potential issues without executing them, enabling early identification of errors and design flaws.
- Methods for containing state space explosion:
  - Utilising Partial Order Reduction (POR) techniques to reduce the state space when verifying concurrent statecharts, making verification more efficient and scalable.

  - Statechart Slicing: Focusing verification efforts on specific parts of a statechart by creating smaller, more manageable slices, improving efficiency, and targeting specific concerns.

- Model checking and abstraction to cover larger classes of specification issues, e.g. liveness and user-defined properties.

- Testcases: Generating executable test cases directly from Statechart models to test the implementation.

9.2.3 Design Specific:

Tailoring statecharts to specific design necessitates the creation of specialised models.

- Models for Accessibility: Developing statechart-based models specifically designed to capture and analyse accessibility requirements, ensuring that systems and applications are usable by people with disabilities.

- Regulations: Creating statecharts to model and verify compliance with regulations, standards, and policies, ensuring adherence to industry guidelines and best practices.

- UI-intensive applications: Modelling individual UI elements and their interactions (e.g., buttons, menus, forms). Focusing on user tasks and their associated UI flows. Combining widget-based and task-based views for comprehensive modelling of user inputs, error handling, etc.

These specialised models ensure that statechart systems are finely tuned to the requirements and intricacies of particular domains, enhancing their effectiveness and applicability within those contexts.

Bibliography

[1] Juarez Dominguez, Alma L. *Detection of Feature Interactions in Automotive Active Safety Features*. PhD thesis, 2012. URL: http://hdl.handle.net/10012/6701.

[2] David Harel. Statecharts: a visual formalism for complex systems. *Science of Computer Programming*, 8(3):231–274, 1987. URL: https://www.sciencedirect.com/science/article/pii/0167642387900359, doi:10.1016/0167-6423(87)90035-9.

[3] David Harel and Amnon Naamad. The statemate semantics of statecharts. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 5(4):293–333, 1996.

[4] Michael Von der Beeck. A comparison of statecharts variants. In *Formal Techniques in Real-Time and Fault-Tolerant Systems: Third International Symposium Organized Jointly with the Working Group Provably Correct Systems—ProCoS Lübeck, Germany, September 19–23, 1994 Proceedings 3*, pages 128–148. Citeseer, 1994.

[5] Étienne André, Shuang Liu, Yang Liu, Christine Choppy, Jun Sun, and Jin Song Dong. Formalizing uml state machines for automated verification – a survey. *ACM Comput. Surv.*, jan 2023. Just Accepted. doi:10.1145/3579821.

[6] Purandar Bhaduri and S. Ramesh. Model checking of statechart models: Survey and research directions. *CoRR*, cs.SE/0407038, 2004. URL: `http://arxiv.org/abs/cs.SE/0407038`.

[7] Simon Van Mierlo and Hans Vangheluwe. *Statecharts: A Formalism to Model, Simulate and Synthesize Reactive and Autonomous Timed Systems*, pages 155–176. Springer International Publishing, Cham, 2020. `doi:10.1007/978-3-030-43946-0_6`.

[8] Yonit Kesten and Amir Pnueli. Timed and hybrid statecharts and their textual representation. In *Formal Techniques in Real-Time and Fault-Tolerant Systems: Second International Symposium Nijmegen, The Netherlands, January 8–10, 1992 Proceedings 2*, pages 591–620. Springer, 1991.

[9] Adriano Peron and Andrea Maggiolo-Schettini. Transitions as interrupts: A new semantics for timed statecharts. In *Theoretical Aspects of Computer Software: International Symposium TACS'94 Sendai, Japan, April 19–22, 1994 Proceedings*, pages 806–821. Springer, 1994.

[10] Junyan Qian and Baowen Xu. Model checking for timed statecharts. In *Formal Techniques for Networked and Distributed Systems-FORTE 2005: 25th IFIP WG 6.1 International Conference, Taipei, Taiwan, October 2-5, 2005. Proceedings 25*, pages 261–274. Springer, 2005.

[11] Jinhyun Kim, Inhye Kang, Jin-Young Choi, and Insup Lee. Timed and resource-oriented statecharts for embedded software. *IEEE Transactions on Industrial Informatics*, 6(4):568–578, 2010.

[12] Carsta Petersohn and Luis Urbina. A timed semantics for the statemate implementation of statecharts. In *FME'97: Industrial Applications and Strengthened Foundations of Formal Methods: 4th International Symposium*

of Formal Methods Europe Graz, Austria, September 15–19, 1997 Proceedings 4, pages 553–572. Springer, 1997.

[13] Xinxiu Wenand Huiqun Yu. Real-time systems modeling and verification with aspect-oriented timed statecharts. *International Journal of Hybrid Information Technology*, 5(2), 2012.

[14] David N Jansen, Holger Hermanns, and Joost-Pieter Katoen. A probabilistic extension of uml statecharts: specification and verification. In *Formal Techniques in Real-Time and Fault-Tolerant Systems: 7th International Symposium, FTRTFT 2002 Co-sponsored by IFIP WG 2.2 Oldenburg, Germany, September 9–12, 2002 Proceedings 7*, pages 355–374. Springer, 2002.

[15] Kangfeng Ye, Ana Cavalcanti, Simon Foster, Alvaro Miyazawa, and Jim Woodcock. Probabilistic modelling and verification using robochart and prism. *Software and Systems Modeling*, pages 1–50, 2022.

[16] Bojan Nokovic and Emil Sekerinski. pstate: A probabilistic statecharts translator. In *2013 2nd Mediterranean Conference on Embedded Computing (MECO)*, pages 29–32. IEEE, 2013.

[17] Alalfi Manar H., Cordy James R., and Dean Thomas R. Modelling methods for web application verification and testing: state of the art. *Software Testing, Verification and Reliability*, 19(4):265–296, 2009. URL: `https://onlinelibrary.wiley.com/doi/abs/10.1002/stvr.401`, `arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1002/stvr.401`, `doi:10.1002/stvr.401`.

[18] Grégoire Hamon and John Rushby. An operational semantics for stateflow. In Michel Wermelinger and Tiziana Margaria-Steffen, editors, *Fundamental Approaches to Software Engineering*, pages 229–243, Berlin, Heidelberg, 2004. Springer Berlin Heidelberg.

[19] W. Chan, R. J. Anderson, P. Beame, S. Burns, F. Modugno, D. Notkin, and J. D. Reese. Model checking large software specifications. *IEEE Transactions on Software Engineering*, 24(7):498–520, Jul 1998. doi:10.1109/32.708566.

[20] Maria Cristina Ferreira de Oliveira, Marcelo Augusto Santos Turine, and Paulo Cesar Masiero. A statechart-based model for hypermedia applications. *ACM Trans. Inf. Syst.*, 19(1):28–52, January 2001. URL: http://doi.acm.org/10.1145/366836.366869, doi:10.1145/366836.366869.

[21] Anneliese A. Andrews, Jeff Offutt, and Roger T. Alexander. Testing web applications by modeling with fsms. *Software & Systems Modeling*, 4(3):326–345, Jul 2005. doi:10.1007/s10270-004-0077-7.

[22] Sonia Flores, Salvador Lucas, and Alicia Villanueva. Formal verification of websites. *Electronic Notes in Theoretical Computer Science*, 200(3):103 – 118, 2008. Proceedings of the 3rd International Workshop on Automated Specification and Verification of Web Systems (WWV 2007). URL: http://www.sciencedirect.com/science/article/pii/S1571066108003101, doi:10.1016/j.entcs.2008.04.095.

[23] Michael Whalen. A parametric structural operational semantics for stateflow, uml statecharts, and rhapsody. 2010.

[24] Michelle L. Crane and Juergen Dingel. Uml vs. classical vs. rhapsody statecharts: not all models are created equal. *Software & Systems Modeling*, 6(4):415–435, Dec 2007. doi:10.1007/s10270-006-0042-8.

[25] Grégoire Hamon and John Rushby. An operational semantics for stateflow. pages 229–243. Springer Berlin Heidelberg, 2004.

[26] Michael von der Beeck. A structured operational semantics for uml-statecharts. *Software and Systems Modeling*, 1, 2002. doi:10.1007/s10270-002-0012-8.

[27] Tao Zhang, Shaobin Huang, and Hongtao Huang. An operational semantics for uml rt-statechart in model checking context. 2009. doi:10.1109/ICICSE.2009.15.

[28] Michael von der Beeck. A structured operational semantics for uml-statecharts. *Software and Systems Modeling*, 1(2):130–141, 2002.

[29] Bence Graics and Vince Molnár. Formal compositional semantics for yakindu statecharts. In *24th PhD Mini-Symposium (Minisy@ DMIS 2017)*, pages 22–24. Budapest University of Technology and Economics, 2017.

[30] Gerald Lüttgen, Michael Von der Beeck, and Rance Cleaveland. A compositional approach to statecharts semantics. *ACM SIGSOFT Software Engineering Notes*, 25(6):120–129, 2000.

[31] Cornelis Huizing, Rob Gerth, and Willem P de Roever. Modelling statecharts behaviour in a fully abstract way. In *Colloquium on Trees in Algebra and Programming*, pages 271–294. Springer, 1988.

[32] Grégoire Hamon. A denotational semantics for stateflow. In *Proceedings of the 5th ACM international conference on Embedded software*, pages 164–172, 2005.

[33] Mass Soldal Lund, Atle Refsdal, and Ketil Stølen. 4 semantics of uml models for dynamic behavior: A survey of different approaches. In *Dagstuhl Workshop on Model-Based Engineering of Embedded Real-Time Systems*, pages 77–103. Springer, 2007.

[34] Ali Taleghani and Joanne M Atlee. Semantic variations among uml statemachines. In *International Conference on Model Driven Engineering Languages and Systems*, pages 245–259. Springer, 2006.

[35] Purandar Bhaduri and S Ramesh. Model checking of statechart models: Survey and research directions. *arXiv preprint cs/0407038*, 2004.

[36] Cristian Cadar, Daniel Dunbar, Dawson R Engler, et al. Klee: Unassisted and automatic generation of high-coverage tests for complex systems programs. In *OSDI*, volume 8, pages 209–224, 2008.

[37] Corina S Păsăreanu and Neha Rungta. Symbolic pathfinder: symbolic execution of java bytecode. In *Proceedings of the 25th IEEE/ACM International Conference on Automated Software Engineering*, pages 179–180, 2010.

[38] Sujit Kumar Chakrabarti and Karthika Venkatesan. Stabl: Statecharts with local variables. In *Proceedings of the 13th Innovations in Software Engineering Conference on Formerly Known as India Software Engineering Conference*, ISEC 2020, New York, NY, USA, 2020. Association for Computing Machinery. doi:10.1145/3385032.3385040.

[39] Karthika Venkatesan and Sujit Kumar Chakrabarti. StaBL - State Based Language for Specification of Web Applications. *arXiv e-prints*, page arXiv:1901.02188, January 2019. arXiv:1901.02188.

[40] Karthika Venkatesan and Sujit Kumar Chakrabarti. A fresh look on semantics of concurrent state based language (constabl). *Journal of Computer Languages*, 79:101270, 2024. URL: https://www.sciencedirect.com/science/article/pii/S2590118424000133, doi:10.1016/j.cola.2024.101270.

[41] Mathworks. Semantics of stateflow. URL: http://www.ece.northwestern.edu/local-apps/matlabhelp/toolbox/stateflow/semantic.html.

[42] David Harel and Amnon Naamad. The statemate semantics of statecharts. *ACM Trans. Softw. Eng. Methodol.*, 5:293–333, 10 1996. URL: http://doi.acm.org/10.1145/235321.235322, doi:10.1145/235321.235322.

[43] Rik Eshuis. Reconciling statechart semantics. *Science of Computer Programming*, 74(3):65–99, 2009. URL: https://www.sciencedirect.com/science/article/pii/S0167642308001020, doi:10.1016/j.scico.2008.09.001.

[44] Karthika Venkatesan and Sujit Kumar Chakrabarti. A fresh look on semantics of concurrent state based language (constabl). *Journal of Computer Languages*, 79:101270, 2024. URL: https://www.sciencedirect.com/science/article/pii/S2590118424000133, doi:10.1016/j.cola.2024.101270.

[45] P. Croll, P.-Y. Duval, R. Jones, S. Kolos, R.F. Sari, and S. Wheeler. Use of statecharts in the modelling of dynamic behaviour in the atlas daq prototype-1. *IEEE Transactions on Nuclear Science*, 45(4):1983–1988, 1998. doi:10.1109/23.710975.

[46] paul-j lucas. Github - paul-j-lucas/chsm: Concurrent hierarchical finite state machine language system., Jul 2018. URL: https://github.com/paul-j-lucas/chsm.

[47] Itemis. Yakindu statecharts. URL: https://www.itemis.com/en/yakindu/state-machine/documentation/user-guide.

[48] Alexandre Decan and Tom Mens. Sismic—a python library for statechart execution and testing. *SoftwareX*, 12, 7 2020. doi:10.1016/J.SOFTX.2020.100590.

[49] Xiaogang Zhu, Sheng Wen, Seyit Camtepe, and Yang Xiang. Fuzzing: A survey for roadmap. *ACM Comput. Surv.*, 54(11s), sep 2022. doi:10.1145/3512345.

[50] Daniel S Fowler, Jeremy Bryans, Siraj Ahmed Shaikh, and Paul Wooderson. Fuzz testing for automotive cyber-security. In *2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W)*, pages 239–246. IEEE, 2018. doi:10.1109/DSN-W.2018.00070.

[51] Valentin JM Manès, HyungSeok Han, Choongwoo Han, Sang Kil Cha, Manuel Egele, Edward J Schwartz, and Maverick Woo. The art, science, and engineering of fuzzing: A survey. *IEEE Transactions on Software Engineering*, 47(11):2312–2331, 2019. doi:10.1109/TSE.2019.2946563.

[52] David Harel. Statecharts in the making: a personal account. In *Proceedings of the third ACM SIGPLAN conference on History of programming languages*, pages 5–1, 2007.

[53] Code Intelligence. Codeintelligencetesting/jazzer: Coverage-guided, in-process fuzzing for the jvm. URL: https://github.com/CodeIntelligenceTesting/jazzer.

[54] Alma L Juarez-Dominguez, Nancy A Day, and Richard T Fanson. A preliminary report on tool support and methodology for feature interaction detection. Technical report, Technical Report CS-2007-44, University of Waterloo, 2007.

[55] Cristian Cadar, Daniel Dunbar, and Dawson Engler. Klee: unassisted and automatic generation of high-coverage tests for complex systems programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation*, OSDI'08, page 209–224, USA, 2008. USENIX Association.

[56] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools (2Nd Edition)*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA, 2006.

[57] Grégoire Hamon and John Rushby. An operational semantics for stateflow. In Michel Wermelinger and Tiziana Margaria-Steffen, editors, *Fundamental Approaches to Software Engineering*, pages 229–243, Berlin, Heidelberg, 2004. Springer Berlin Heidelberg.

[58] David Harel and Amnon Naamad. The statemate semantics of statecharts. *ACM Trans. Softw. Eng. Methodol.*, 5(4):293–333, October 1996. URL: http://doi.acm.org/10.1145/235321.235322, doi:10.1145/235321.235322.

[59] David Harel, Amir Pnueli, Jeanette P. Schmidt, and Rivi Sherman. On the formal semantics of statecharts (extended abstract). In *Logic in Computer Science*, 1987. URL: https://api.semanticscholar.org/CorpusID:5226046.

[60] D. Harel, H. Lachover, A. Naamad, A. Pnueli, M. Politi, R. Sherman, A. Shtull-Trauring, and M. Trakhtenbrot. Statemate: a working environment for the development of complex reactive systems. *IEEE Transactions on Software Engineering*, 16(4):403–414, 1990. doi:10.1109/32.54292.

[61] Erich Mikk, Yassine Lakhnech, Carsta Petersohn, and Michael Siegel. On formal semantics of statecharts as supported by statemate. In *Proceedings of the 2nd BCS-FACS Conference on Northern Formal Methods*, 2FACS'97, page 12, Swindon, GBR, 1997. BCS Learning amp; Development Ltd.

[62] Michelle L. Crane and Juergen Dingel. Uml vs. classical vs. rhapsody statecharts: Not all models are created equal. In Lionel Briand and Clay Williams, editors, *Model Driven Engineering Languages and Systems*, pages 97–112, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.

[63] N. G. Leveson, M. P. E. Heimdahl, H. Hildreth, and J. D. Reese. Requirements specification for process-control systems. *IEEE Transactions on Software Engineering*, 20(9):684–707, Sep. 1994. doi:10.1109/32.317428.

[64] David Harel and Hillel Kugler. The rhapsody semantics of statecharts (or, on the executable core of the uml). In *Integration of Software Specification Techniques for Applications in Engineering*, pages 325–354. Springer, 2004.

[65] Scxml. https://www.w3.org/tr/scxml/. URL: https://www.w3.org/TR/scxml/.

[66] Ana Cavalcanti, Phil Clayton, and Colin O'Halloran. From control law diagrams to ada via circus. *Formal Aspects of Computing*, 23(4):465–512, 2011.

[67] Diego Latella, Istvan Majzik, and Mieke Massink. Towards a formal operational semantics of uml statechart diagrams. In *International conference on formal methods for open object-based distributed systems*, pages 331–347. Springer, 1999.

[68] Andrzej Wasowski and Peter Sestoft. On the formal semantics of visualstate statecharts. Technical report, Technical Report TR-2002-19, IT University of Copenhagen, 2002.

[69] Joeri Exelmans. *A Study and Implementation of Semantic Variability in Statecharts*. PhD thesis, Master's thesis. University Of Antwerp, 2020.

[70] Amir Pnueli and Michal Shalev. What is in a step: On the semantics of statecharts. In *International Symposium on Theoretical Aspects of Computer Software*, pages 244–264. Springer, 1991.

[71] Rik Eshuis. Reconciling statechart semantics. *Science of Computer Programming*, 74, 2009. doi:10.1016/j.scico.2008.09.001.

[72] Nancy Ann Day. *A model checker for statecharts (linking case tools with formal methods)*. PhD thesis, University of British Columbia, 1993.

[73] David Harel and Hillel Kugler. *The Rhapsody Semantics of Statecharts (or, On the Executable Core of the UML)*, pages 325–354. Springer Berlin Heidelberg, Berlin, Heidelberg, 2004. doi:10.1007/978-3-540-27863-4_19.

[74] Andrew C Uselton and Scott A Smolka. A process algebraic semantics for statecharts via state refinement. In *PROCOMET*, volume 94, pages 262–281. Citeseer, 1994.

[75] Xuede Than, Huaikou Miao, and Ling Liu. Formalizing the semantics of uml statecharts with z. In *The Fourth International Conference onComputer and Information Technology, 2004. CIT '04.*, pages 1116–1121, 2004. doi:10.1109/CIT.2004.1357344.

[76] IBM. Semantics of transition selection algorithm in ibm engineering lifecycle management suite: Design rhapsody. https://www.ibm.com/docs/en/engineering-lifecycle-management-suite/design-rhapsody/9.0.1?topic=semantics-transition-selection-algorithm. Accessed on: [17-July-2023].

[77] Jim Barnett. *Introduction to SCXML*, pages 81–107. Springer International Publishing, Cham, 2017. doi:10.1007/978-3-319-42816-1_5.

[78] Jianwei Niu, J.M. Atlee, and N.A. Day. Template semantics for model-based notations. *IEEE Transactions on Software Engineering*, 29(10):866–882, 2003. doi:10.1109/TSE.2003.1237169.

[79] James L. Peterson. Petri nets. *ACM Comput. Surv.*, 9(3):223–252, sep 1977. doi:10.1145/356698.356702.

[80] Sanoop Mallissery and Yu-Sung Wu. Demystify the fuzzing methods: A comprehensive survey. *ACM Comput. Surv.*, 56(3), oct 2023. doi:10.1145/3623375.

[81] Martin Schneider, Jürgen Großmann, Nikolay Tcholtchev, Ina Schieferdecker, and Andrej Pietschker. Behavioral fuzzing operators for uml sequence diagrams. In *System Analysis and Modeling: Theory and Practice: 7th International Workshop, SAM 2012, Innsbruck, Austria, October 1-2, 2012.*

Revised Selected Papers 7, pages 88–104. Springer, 2013. URL: https://doi.org/10.1007/978-3-642-36757-1_6.

[82] Minmin Han and Christine Hofmeister. Modeling and verification of adaptive navigation in web applications. In *Proceedings of the 6th International Conference on Web Engineering*, ICWE '06, pages 329–336, New York, NY, USA, 2006. ACM. URL: http://doi.acm.org/10.1145/1145581.1145645, doi:10.1145/1145581.1145645.

[83] A. Cimatti, E. Clarke, F. Giunchiglia, and M. Roveri. Nusmv: A new symbolic model verifier. In Nicolas Halbwachs and Doron Peled, editors, *Computer Aided Verification*, pages 495–499, Berlin, Heidelberg, 1999. Springer Berlin Heidelberg.

[84] E. Clarke and H. Schlingloff. Model checking. *Handbook of Automated Reasoning*, 1996. doi:10.1145/1592761.1592781.

[85] D. Castelluccia, M. Mongiello, M. Ruta, and R. Totaro. Waver: A model checking-based tool to verify web application design. *Electronic Notes in Theoretical Computer Science*, 157(1):61 – 76, 2006. Proceedings of the Third International Workshop on Software Verification and Validation (SVV 2005). URL: http://www.sciencedirect.com/science/article/pii/S1571066106002350, doi:10.1016/j.entcs.2006.01.023.

[86] A. Kubo, H. Washizaki, and Y. Fukazawa. Automatic extraction and verification of page transitions in aweb application. In *14th Asia-Pacific Software Engineering Conference (APSEC'07)*, pages 350–357, Dec 2007. doi:10.1109/ASPEC.2007.57.

[87] Gerard J. Holzmann. The model checker spin. *IEEE Transactions on software engineering*, 23(5):279–295, 1997.

[88] E. H. Choi and H. Watanabe. Model checking class specifications for web applications. In *12th Asia-Pacific Software Engineering Conference (APSEC'05)*, pages 9 pp.–, Dec 2005. doi:10.1109/APSEC.2005.79.

[89] Kim G Larsen, Paul Pettersson, and Wang Yi. Uppaal in a nutshell. *International journal on software tools for technology transfer*, 1:134–152, 1997.

[90] H. Miao and H. Zeng. Model checking-based verification of web application. In *12th IEEE International Conference on Engineering Complex Computer Systems (ICECCS 2007)*, pages 47–55, July 2007. doi:10.1109/ICECCS.2007.30.

[91] Orna Grumberg, Yael Meller, and Karen Yorav. Applying software model checking techniques for behavioral uml models. In Dimitra Giannakopoulou and Dominique Méry, editors, *FM 2012: Formal Methods*, pages 277–292, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.

[92] Shoji Yuen, Keishi Kato, Daiju Kato, and Kiyoshi Agusa. Web automata: A behavioral model of web applications based on the mvc model. *Information and Media Technologies*, 1(1):66–79, 2006. doi:10.11185/imt.1.66.

[93] María Alpuente, Demis Ballis, and Daniel Romero. A rewriting logic approach to the formal specification and verification of web applications. *Science of Computer Programming*, 81:79 – 107, 2014. Automated Verification of Critical Systems 2010 (AVoCS 2010). URL: http://www.sciencedirect.com/science/article/pii/S0167642313001883, doi:10.1016/j.scico.2013.07.014.

[94] T. K. Das and J. Dingel. State machine antipatterns for uml-rt. In *2015 ACM/IEEE 18th International Conference on Model Driven Engineering Languages and Systems (MODELS)*, pages 54–63, Sept 2015. doi:10.1109/MODELS.2015.7338235.

[95] Gerard J Holzmann. Explicit-state model checking. *Handbook of Model Checking*, pages 153–171, 2018.

[96] Stefan Edelkamp, Stefan Leue, and Alberto Lluch-Lafuente. Directed explicit-state model checking in the validation of communication protocols. *International journal on software tools for technology transfer*, 5:247–267, 2004.

APPENDIX A

TYPES

A.1 Structures

A `struct` type declaration[1]looks like as follows. we allow recursive types too.

```
struct Duration = {
  startTime : int;
  endTime : int;
}
```

```
struct Request {
    TicketID: int;
    creation_date : Date;
    requester : User;
    team : list<|User|>;
    }
```

A.2 Functions

Functions map a list of input types to an output type. For example: sum : $fun(int \times int \rightarrow int)$ declares a function sum which takes two ints and returns an

[1]The content of this chapter is based on: Venkatesan, Karthika, and Sujit Kumar Chakrabarti. "StaBL-State Based Language for Specification of Web Applications." arXiv preprint arXiv:1901.02188 (2019). doi: https://doi.org/10.48550/arXiv.1901.02188

int. In our current implementation of StaBL compiler, we allow function declarations, but function definition is not implemented. The semantics of a set of library functions acting on our container types are defined, which are used in the symbolic simulation of StaBL specifications for static analysis purposes. Functions are not allowed to cause external side-effects, i.e., if a function contains an assignment to any variable which is not local to the function, it will not typecheck. However, assignments are allowed at the local level, i.e., the value returned from a function can be assigned to a variable at the call site.

A.3 Polymorphic Types

A polymorphic type can be thought of as a function (*type list* $\rightarrow$ *type*) which takes a list of types as parameters and returns a type. This type of polymorphism, specifically called *parametric polymorphism* in programming languages terminology, is a key feature of StaBL which allows creation of specifications which are flexible and re-usable on the one hand, and type-safe on the other.

A polymorphic type encapsulates a type expression inside it. The inner type expression is allowed to be any non-polymorphic type expression where type variables may take the place of types. Here are a few examples of polymorphic type declarations allowed in StaBL:

- Polymorphic structures:

```
struct◀A► S1{ ... }
struct◀A, B► S2 {
  s1 : S1◀B►;
  s2 : S1◀A►;
  s3 : S1◀int►;
  s4 : string;
}
```

- Polymorphic functions:

```
f◀A, B▶(p1 : B, p2 : S1◀A▶, p3 : S1◀int▶, p4 : string) : S1◀S2◀A,B▶▶
```

A.4 Substantiation of Polymorphic Types.

When a variable is declared as a polymorphic, it is really a *substantiation* of a polymorphic type. By substantiation, we mean getting a concrete type from a polymorphic type by supplying the required type arguments. For example:

```
p : pair◀ int, boolean ▶;
```

A.5 Containers.

Currently, we have provided containers like lists, sets and maps. All of them are polymorphic types. There are library functions declared which allow interacting with these containers. For instance, *put_map* allows adding/updating a key value pair to a map, *get_map* allows retrieving a value corresponding to a given key from a map. All these functions are side-effect free, i.e. a call to *put_map* does not modify the map passed as argument; rather it return a new map object. The decision to make functions side-effect free may have performance implications if translated directly into a running program. However, as StaBL statecharts are really specifications, their objective is analysis and not implementation. Side-effect free functions ease many difficult issues for static analysis.

APPENDIX B

EXHAUSTIVE EXPLORATION - NODES

The nodes created in exhaustive exploration all nodes is shown in the Table B and Table TA2.2:

Table TA2.1: External nodes created during exhaustive exploration at statechart

event	$ENode_{in}$	$\langle \mathbb{C}, \sigma \rangle$	$\mathcal{T}$	Code	EC	$ENode_{out}$
	$\langle \mathbb{C} = \emptyset, \sigma = \emptyset \rangle$		-			
init		$E_0 : \langle \emptyset, \emptyset \rangle$	ST_i	$\langle$ $\mathcal{M}.a_{\mathcal{N}}$,$G.a_{\mathcal{N}}$,[$\langle E.a_{\mathcal{N}}$, $A.a_{\mathcal{N}}\rangle$ | $\langle F.a_{\mathcal{N}}$, $C.a_{\mathcal{N}}\rangle$] $\rangle$	EC_0	$\langle [A, C], \sigma_1 \rangle$, $\langle [A, C], \sigma_2 \rangle$, $\langle [A, C], \sigma_3 \rangle$, $\langle [A, C], \sigma_4 \rangle$, $\langle [A, C], \sigma_5 \rangle$, $\langle [A, C], \sigma_6 \rangle$
e_1	$\langle [A, C], \sigma_2 \rangle$, $\langle [A, C], \sigma_3 \rangle$, $\langle [A, C], \sigma_4 \rangle$, $\langle [A, C], \sigma_5 \rangle$, $\langle [A, C], \sigma_6 \rangle$	E_1 : $\langle [A, C], \sigma_1 \rangle$	t_{AB}	$\langle A.a_{\mathcal{X}}, t_{AB}.a$, $B.a_{\mathcal{N}}\rangle$	EC_1	$\langle [B, C], \sigma_1' \rangle$

	$\langle[A,C],\sigma_3\rangle$, $\langle[A,C],\sigma_4\rangle$, $\langle[A,C],\sigma_5\rangle$, $\langle[A,C],\sigma_6\rangle$	E_2 : $\langle[A,C],\sigma_2\rangle$	t_{CD}	$\langle C.a_{\mathcal{X}}, t_{CD}.a, D.a_{\mathcal{N}}\rangle$	EC_2	$\langle[A,D],\sigma'_2\rangle$
	$\langle[A,C],\sigma_4\rangle$, $\langle[A,C],\sigma_5\rangle$, $\langle[A,C],\sigma_6\rangle$	E_3 : $\langle[A,C],\sigma_3\rangle$	t_{AB}, t_{CD}	$[\langle A.a_{\mathcal{X}}, t_{AB}.a, B.a_{\mathcal{N}}\rangle \mid \langle C.a_{\mathcal{X}}, t_{CD}.a, D.a_{\mathcal{N}}\rangle]$	EC_3	$\langle[B,D],\sigma'_{3-1}\rangle$... $\langle[B,D],\sigma'_{3-20}\rangle$
	$\langle[A,C],\sigma_5\rangle$, $\langle[A,C],\sigma_6\rangle$	E_4 : $\langle[A,C],\sigma_4\rangle$	t_{AB}	$\langle A.a_{\mathcal{X}}, t_{AB}.a, B.a_{\mathcal{N}}\rangle$	EC_4	$\langle[B,C],\sigma'_4\rangle$
	E_5 : $\langle[A,C],\sigma_6\rangle$	$\langle[A,C],\sigma_5\rangle$	t_{CD}	$\langle C.a_{\mathcal{X}}, t_{CD}.a, D.a_{\mathcal{N}}\rangle$	EC_5	$\langle[A,D],\sigma'_5\rangle$
	$\emptyset$	E_6 : $\langle[A,C],\sigma_6\rangle$	t_{AB}, t_{CD}	$[\langle A.a_{\mathcal{X}}, t_{AB}.a, B.a_{\mathcal{N}}\rangle \mid \langle C.a_{\mathcal{X}}, t_{CD}.a, D.a_{\mathcal{N}}\rangle]$	EC_6	$\langle[B,D],\sigma'_{6-1}\rangle$... $\langle[B,D],\sigma'_{6-20}\rangle$
e	$\langle[A,C],\sigma_2\rangle$, $\langle[A,C],\sigma_3\rangle$, $\langle[A,C],\sigma_4\rangle$, $\langle[A,C],\sigma_5\rangle$, $\langle[A,C],\sigma_6\rangle$	E_{11} : $\langle[A,C],\sigma_1\rangle$	t_{GN}	$\langle[\langle A.a_{\mathcal{X}}, E.a_{\mathcal{X}}\rangle \mid \langle C.a_{\mathcal{X}}, F.a_{\mathcal{X}}\rangle], G.a_{\mathcal{X}}, t_{GN}.a, N.a_{\mathcal{N}}, [\langle L.a_{\mathcal{N}}, H.a_{\mathcal{N}}\rangle \mid \langle M.a_{\mathcal{N}}, J.a_{\mathcal{N}}\rangle]\rangle, B.a_{\mathcal{N}}\rangle$	EC'_1	$\langle[H,J],\sigma''_1\rangle$
	$\langle[A,C],\sigma_3\rangle$, $\langle[A,C],\sigma_4\rangle$, $\langle[A,C],\sigma_5\rangle$, $\langle[A,C],\sigma_6\rangle$	E_{12} : $\langle[A,C],\sigma_2\rangle$	t_{GN}	$\langle[\langle A.a_{\mathcal{X}}, E.a_{\mathcal{X}}\rangle \mid \langle C.a_{\mathcal{X}}, F.a_{\mathcal{X}}\rangle], G.a_{\mathcal{X}}, t_{GN}.a, N.a_{\mathcal{N}}, [\langle L.a_{\mathcal{N}}, H.a_{\mathcal{N}}\rangle \mid \langle M.a_{\mathcal{N}}, J.a_{\mathcal{N}}\rangle]\rangle$	EC'_2	$\langle[H,J],\sigma''_2\rangle$

	$\langle[A,C],\sigma_4\rangle$, $\langle[A,C],\sigma_5\rangle$, $\langle[A,C],\sigma_6\rangle$	E_{13} : $\langle[A,C],\sigma_3\rangle$	t_{GN}	$\langle[\langle A.a_{\mathcal{X}}, E.a_{\mathcal{X}}\rangle\|$ $\langle C.a_{\mathcal{X}}, F.a_{\mathcal{X}}\rangle], G.a_{\mathcal{X}}$, $t_{GN}.a$, $N.a_{\mathcal{N}}, [\langle L.a_{\mathcal{N}}, H.a_{\mathcal{N}}\rangle\|$ $\langle M.a_{\mathcal{N}}, J.a_{\mathcal{N}}\rangle]\rangle$	EC'_3	$\langle[H,J],\sigma''_{3-1}\rangle$... $\langle[H,J],\sigma''_{3-20}\rangle$
	$\langle[A,C],\sigma_5\rangle$, $\langle[A,C],\sigma_6\rangle$	E_{14} : $\langle[A,C],\sigma_4\rangle$	t_{GN}	$\langle[\langle A.a_{\mathcal{X}}, E.a_{\mathcal{X}}\rangle,$ $\langle C.a_{\mathcal{X}}, F.a_{\mathcal{X}}\rangle], G.a_{\mathcal{X}}$, $t_{GN}.a$, $N.a_{\mathcal{N}}, [\langle L.a_{\mathcal{N}}, H.a_{\mathcal{N}}\rangle\|$ $\langle M.a_{\mathcal{N}}, J.a_{\mathcal{N}}\rangle]\rangle$	EC'_4	$\langle[H,J],\sigma''_4\rangle$
	$\langle[A,C],\sigma_6\rangle$	E_{15} : $\langle[A,C],\sigma_5\rangle$	t_{GN}	$\langle[\langle A.a_{\mathcal{X}}, E.a_{\mathcal{X}}\rangle\|$ $\langle C.a_{\mathcal{X}}, F.a_{\mathcal{X}}\rangle], G.a_{\mathcal{X}}$, $t_{GN}.a$, $N.a_{\mathcal{N}}, [\langle L.a_{\mathcal{N}}, H.a_{\mathcal{N}}\rangle\|\langle M.a_{\mathcal{N}}, J.a_{\mathcal{N}}\rangle]\rangle$	EC'_5	$\langle[H,J],\sigma''_5\rangle$
	$\emptyset$	E_{16} : $\langle[A,C],\sigma_6\rangle$	t_{GN}	$\langle[\langle A.a_{\mathcal{X}}, E.a_{\mathcal{X}}\rangle\|$ $\langle C.a_{\mathcal{X}}, F.a_{\mathcal{X}}\rangle], G.a_{\mathcal{X}}$, $t_{GN}.a$, $N.a_{\mathcal{N}}, [\langle L.a_{\mathcal{N}}, H.a_{\mathcal{N}}\rangle$ $\| \langle M.a_{\mathcal{N}}, J.a_{\mathcal{N}}\rangle]\rangle$	EC'_6	$\langle[H,J],\sigma''_{6-1}\rangle$... $\langle[H,J],\sigma''_{6-20}\rangle$
e	$\langle[A,D],\sigma'_2\rangle$, $\langle[B,D],\sigma'_{3-1}\rangle$... $\langle[B,D],\sigma'_{3-20}\rangle$, $\langle[B,C],\sigma'_4\rangle$, $\langle[A,D],\sigma'_5\rangle$, $\langle[B,D],\sigma'_{6-1}\rangle$... $\langle[B,D],\sigma'_{6-20}\rangle$	$\langle[B,C],\sigma'_1\rangle$	t_{GN}	$\langle[\langle B.a_{\mathcal{X}}, E.a_{\mathcal{X}}\rangle$ $\| \langle C.a_{\mathcal{X}}, F.a_{\mathcal{X}}\rangle$], $G.a_{\mathcal{X}}$, $t_{GN}.a$, $N.a_{\mathcal{N}}$, $[\langle L.a_{\mathcal{N}}, H.a_{\mathcal{N}}\rangle \|$ $\langle M.a_{\mathcal{N}}, J.a_{\mathcal{N}}\rangle]\ \rangle$	EC_7	$\langle[H,J],\sigma'_{6-1}\rangle$... $\langle[H,J],\sigma'_{6-20}\rangle$

Table TA2.2: Internal nodes created - EC_0

$Process(INode)$	$node_{new}$
-	$n_0 : (iM\mathcal{N}1, E_1, \Delta_{\diamond})$
n_0	$n_1 : (iG\mathcal{N}1, n_0, \Delta_{i\mathcal{M}\mathcal{N}1})$
n_1	$n_2 : (\{iE\mathcal{N}1, iF\mathcal{N}1\}, n_1, \Delta_{iG\mathcal{N}1})$
n_2	$n_3 : (\{iA\mathcal{N}1, iF\mathcal{N}1\}, n_2, \Delta_{iE\mathcal{N}1}), n_4 : (\{iE\mathcal{N}1, iC\mathcal{N}1\}, n_2, \Delta_{iF\mathcal{N}1})$
n_3	$n_5 : (\{iF\mathcal{N}1\}, n_3, \Delta_{iA\mathcal{N}1}), n_6 : (\{iA\mathcal{N}1, iC\mathcal{N}1\}, n_3, \Delta_{iE\mathcal{N}1})$
n_4	$n_7 : (\{iA\mathcal{N}1, iC\mathcal{N}1\}, n_4, \Delta_{iE\mathcal{N}1}), n_8 : (\{iE\mathcal{N}1\}, n_4, \Delta_{iC\mathcal{N}1})$
n_5	$n_9 : (\{iC\mathcal{N}1\}, n_5, \Delta_{iF\mathcal{N}1})$
n_6	$n_{10} : (\{\{iA\mathcal{N}1\}, n_6, \Delta_{iC\mathcal{N}1}), n_{11} : (\{\{iC\mathcal{N}1\}, n_6, \Delta_{iA\mathcal{N}1})$
n_7	$n_{12} : (\{iC\mathcal{N}1\}, n_7, \Delta_{iA\mathcal{N}1}), n_{13} : (\{iA\mathcal{N}1\}, n_7, \Delta_{iC\mathcal{N}1})$
n_8	$n_{14} : (\{iA\mathcal{N}1\}, n_8, \Delta_{iE\mathcal{N}1})$
n_9	$n_{15} : (\{\}, n_9, \Delta_{iC\mathcal{N}1})$
n_{10}	$n_{16} : (\{\}, n_{10}, \Delta_{iA\mathcal{N}1})$
n_{11}	$n_{17} : (\{\}, n_{11}, \Delta_{iC\mathcal{N}1})$
n_{12}	$n_{18} : (\{\}, n_{12}, \Delta_{iC\mathcal{N}1})$
n_{13}	$n_{19} : (\{\}, n_{13}, \Delta_{iA\mathcal{N}1})$
n_{14}	$n_{20} : (\{\}, n_{14}, \Delta_{iA\mathcal{N}1})$
n_{15}	new ENode($n_{15}, [A, C], \sigma_{n_{15}}, e), newENode(n_{15}, [A, C], \sigma_{n_{15}}, e_1)$
n_{16}	new ENode($n_{16}, [A, C], \sigma_{n_{16}}, e), newENode(n_{16}, [A, C], \sigma_{n_{16}}, e_1)$
n_{17}	new ENode($n_{17}, [A, C], \sigma_{n_{17}}, e), newENode(n_{17}, [A, C], \sigma_{n_{17}}, e_1)$
n_{18}	new ENode($n_{18}, [A, C], \sigma_{n_{18}}, e), newENode(n_{18}, [A, C], \sigma_{n_{18}}, e_1)$
n_{19}	new ENode($n_{19}, [A, C], \sigma_{n_{19}}, e), newENode(n_{19}, [A, C], \sigma_{n_{19}}, e_1)$
n_{20}	new ENode($n_{20}, [A, C], \sigma_{n_{20}}, e), newENode(n_{20}, [A, C], \sigma_{n_{20}}, e_1)$

APPENDIX C

CASE STUDIES

The model file for the example model illustrated in Figure FC3.1 is shown below. [1]

```
statechart Student {
  types {
    type t1<| |>; // no type parameters
    type list<| a |>; // with one type parameter
    type map<| a, b |>; // with multiple type parameters
  }
  events {
    eLogin;
    eLogout;
    eSetRoom;
    eViewDetails;
    eDoneSetting;
    eDoneViewing;
  }
  students : map<| int, string |> : static;
  rooms : map<| int, int |> : static;
  entry :
  {
    students :=  put_map<|int, string |>(students, 100, "p1");
    students :=  put_map<|int, string |>(students, 101, "p2");
  }
  exit  : {}
  functions {
```

[1] The other models used in this work are available in the github link: https://github.com/karthiv/ConstaBL-Fuzz/blob/main/statechart-verification/src/dfa/data/

```
  put_map<| A, B |>(amap : map<| A, B |>, k : A, v : B) : map<| A, B |>;
  get_map<| A, B |>(amap : map<| A, B |>, k : A) : B;
}
state LoggedOut {
  user      : int : param;
  password  : string : param;
  entry : {}
  exit  : {}
}
state LoggedIn {
  loggedinUser : int : param;
  entry : {}
  exit  : {}
  state Dashboard {
  }
  state SetRoom {
    room  : int : local;
    entry : {}
    exit  : { rooms := put_map <| int, int |>(rooms, loggedinUser, room); }
  }
  state ViewDetails {
    student : int : local;
    room    : int : local;
    entry : {
      student := loggedinUser;
      room    := get_map <| int, int |>(rooms, loggedinUser);
        // possible access to undefined value: It is possible for this call
        // to get_map to fail if there is no entry in
        // rooms (map<| int, int |>) with key = loggedinUser.
    }
    exit : {}
  }
  transition tsetroom {
    source      : Student.LoggedIn.Dashboard;
    destination : Student.LoggedIn.SetRoom;
    trigger     : eSetRoom;
    guard       : true;
    action      : {}
  }
  transition tviewdetails {
    source      : Student.LoggedIn.Dashboard;
```

```
      destination : Student.LoggedIn.ViewDetails;
      trigger     : eViewDetails;
      guard       : true;
      action      : {}
    }
    transition tsetroom_done {
      source      : Student.LoggedIn.SetRoom;
      destination : Student.LoggedIn.Dashboard;
      trigger     : eDoneSetting;
      guard       : true;
      action      : {}
    }
    transition tviewdetails_done {
      source      : Student.LoggedIn.ViewDetails;
      destination : Student.LoggedIn.Dashboard;
      trigger     : eDoneViewing;
      guard       : true;
      action      : {}
    }
  }
  transition tlogin {
    source      : Student.LoggedOut;
    destination : Student.LoggedIn;
    trigger     : eLogin;
    guard       : get_map <| int, string |>(students, Student.LoggedOut.user) =
        ↪ Student.LoggedOut.password;
    action      : Student.LoggedIn.loggedinUser := Student.LoggedOut.user;
  }
  transition tlogout {
    source      : Student.LoggedIn;
    destination : Student.LoggedOut;
    trigger     : eLogout;
    guard       : true;
    action      : {
      Student.LoggedOut.user := 0;
      Student.LoggedOut.password := "";
    }
  }
}
```